FINAL PROJECT

Hayden Schrauff, Elena Koung, Yifan Qin, and Kevin Ma

2021-05-04

The data cleaning and report write-up were done by Elena Koung, classification of location data was completed by Hayden Schrauff, regression and tree analysis was done by Yifan Qin, and Kevin Ma created our recommender system.

We would like to thank Yelp for providing the business data this project used. We would also like to thank Dr. Ho for teaching us how to use the tools to study and analyse the data.

Introduction

With the increasingly racially and ethnically diverse population of the U.S., differences between those groups are becoming all the more noticeable in everyday life. One of the most important differences between these groups is their ethnic cuisines, which is a part of their identity and culture. One major way that their cultures are becoming widespread in the U.S. is through businesses that specialize in these ethnic food types, such as Asian, Mexican or even fusion cuisines. In particular, as the Asian population has grown in recent years, so has the demand for Asian cuisine. In this project, we analyze data about Asian restaurant businesses in recent years and Asian demographics in those areas using a combination of Yelp business data and US Census data. The data that Yelp provided was limited in scope, only covering a select number of states (Arizona, Wisconsin, and Nevada) and regions in Canada. We were specifically interested in American states and cities only; therefore, we had to find matching census data to these regions.

Cleaning the data

Our first task in analyzing and creating our recommendation system was to clean up the data set. The original dataset was huge. It had 42,153 rows and 105 columns, so it was impossible to look through the data to understand exactly what was required to analyze. To clean the data, we need to gain understanding of what exactly the data contains and choose only what is necessary for analysis. By using dplyr functions to filter and group the data by state, it was clear that we had five specific regions, including two we did not recognize. The two we did not recognize were areas in Canada, so it was important for us to remove all instances of these regions from the data. The table below summarizes the observations of each business in each state of the data.

State	Number of Observations
AZ	22181
CA	1
EDH	2841
ELN	8
FIF	3

GA	1
KHL	1
MA	1
MLN	102
NC	1
NTH	1
NV	14583
NY	2
ON	305
SCB	2
WI	2118
XGL	1

It is clear that not all of the 50 states in the U.S. are represented in this data, so it will be hard to draw any major conclusions. Therefore, our hope is that by doing exploratory research on this data, the cultural landscape of these states will be known to us. From the table, we see that the major states are AZ (Arizona), NV (Nevada), and WI (Wisconsin). There are a few other states in the dataset, but there are not enough observations per state for it to be useful in our analysis later on. Therefore, we choose to only look at the three states with the largest number of businesses. After removing data that are not in the U.S. or in the three chosen states, the dataset shrunk to 38,882 rows and 105 columns.

Next, we wanted to decide what attributes of the businesses are important. The overall approach was to find out which attributes align with attributes of a restaurant and make that into its own data frame in an effort to reduce the number of columns. We hoped that by doing this, it would make it to apply different types of models onto this dataframe.

Then, we needed to look at the categories of the businesses. By creating a separate categories data frame, it is easier to see exactly what each business is classified as. We select categories and the business' name to a new data frame, so we can separate the categories for each business into their first 3 categories. This means that for every business in the categories dataframe, there are at most three types of categories we can look at. The reason that three types of categories were chosen was because most restaurants were described with three categories with a few exceptions that had up to 10 categories listed. Since the majority of the data was around three, it would be easier to look at just the first three categories and will not affect our analysis with lesser specificity.

To determine what categorizes a business as a “Restaurant”, we manually went through a list by using dplyr functions to group and summarize the businesses by categories. We only looked at the businesses by their first instance of categorization. For example, a business could be considered “American (New)” but also “Burgers”, so we were only interested in looking at the first category, aka type1, to see if it indicates that the business is a restaurant. By doing this, we can clearly account for all restaurants in the dataset. The table below illustrates a few categories and their respective counts.

Category	Number of Observations
"American (New)"	416
"American (Traditional)"	654
"Asian Fusion"	132
"Bakeries"	419
"Barbeque"	159
"Bars"	1248
"Breakfast & Brunch"	420
“Buffets”	204
"Burgers"	753
"Chinese"	572
“Delis”	225
"Dim Sum"	12
"Fast Food"	859
"Hawaiian"	53
“Hot Dogs"	84
"Indian"	72
"Italian"	263
"Japanese"	156
"Korean"	102
"Mexican"	1269

Once the attributes and categories data frames were created, we needed to use this information to filter out the data set that contains only the businesses from AZ, WI, NV. By matching the names from categories onto attributes and the original data set, we now have data sets that describe attributes and categories and contain only restaurants in AZ, WI, NV (cleaned data sets).

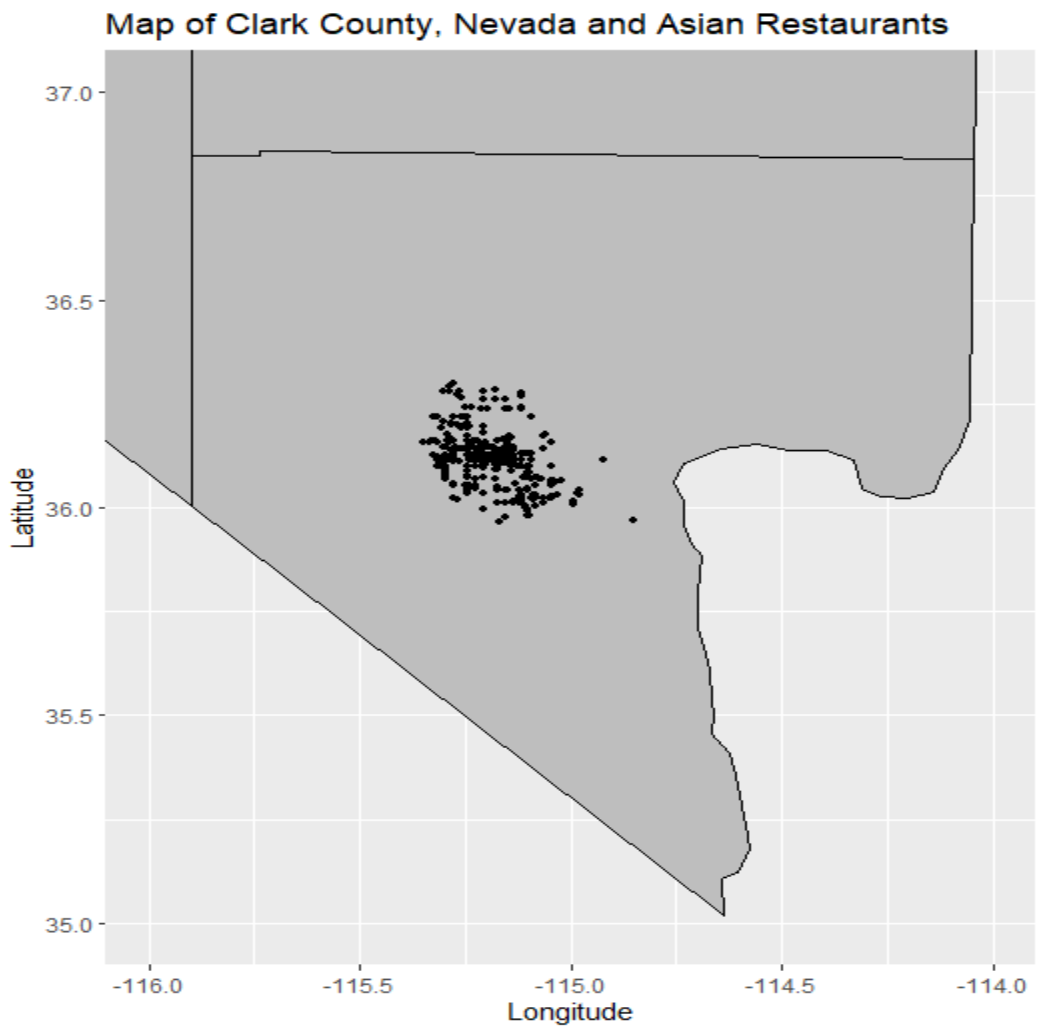
We used the clean data set with review count to indicate whether the restaurant is popular or not. By looking at how many reviews a restaurant has, we are able to distinguish whether a restaurant is high popularity (more than 800 reviews), medium popularity (between 50 and 800 reviews) or low popularity (under 50 reviews but above 20 reviews).

We also discern from our categories list what exactly an "Asian restaurant" is. We determined that these categories are considered to be "Asian" cuisine by our experience. Then, we used dplyr functions to filter our clean dataset into Asian restaurants only. Further, we split the data set by state to define only Asian restaurants in AZ, WI, NV.

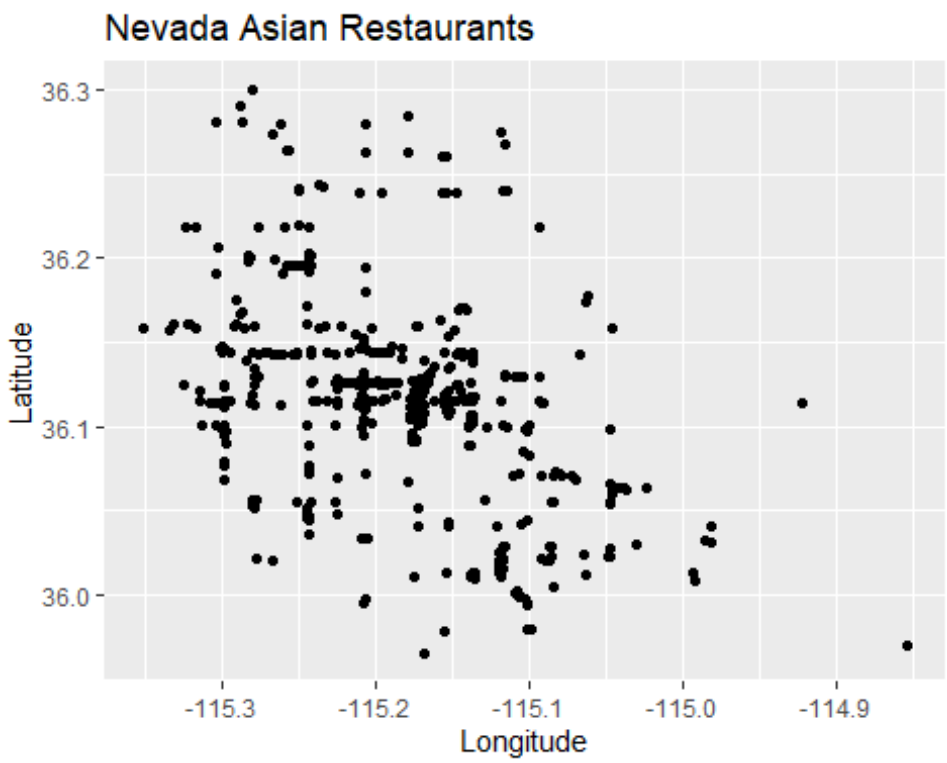
Our first question we would like to answer is how Asian restaurants differ in their clustering when compared by state and how various clustering methods work with the data. We use K-means clustering, which is a clustering algorithm that returns the natural grouping of data points, based on their similarity. It is a special case of Gaussian Mixture Models. We considered PCA as a method to study; however, given the time constraints and the fact that we are more comfortable with K-means, we carried out analysis using K-means. PCA is a mathematical procedure that converts a lot of correlated variables into a few uncorrelated variables called principal components (PCs). It reduces a large set of vars to a small set that still contains most of the info. This is definitely a procedure that could also be used for the analysis, but it could be done for future research and would be interesting to compare the results. However, when we are just looking at location clustering, we are not looking at high-dimensional data. Thus, we proceed with K-means.

Clustering Methods for Location Data

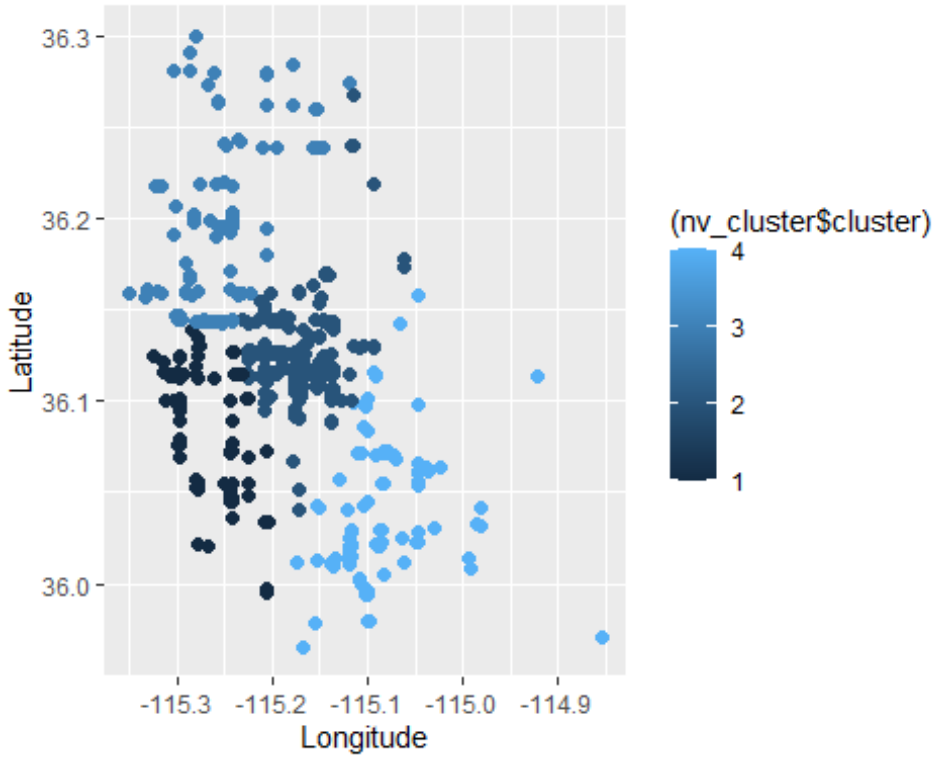
We will begin our clustering analysis with Asian restaurants in Nevada by plotting them on a map to get a sense of the distribution of the data.



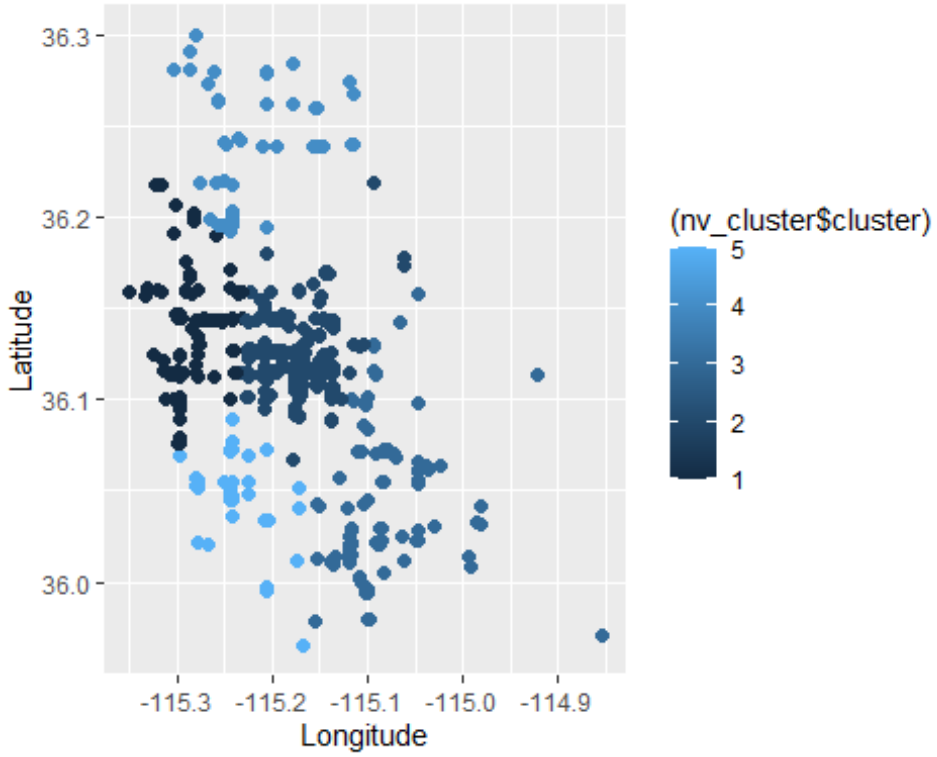
Next, we will make a data frame of only the longitudinal and latitudinal data.



We will now use K-means clustering to analyze the data, starting with 4 clusters.

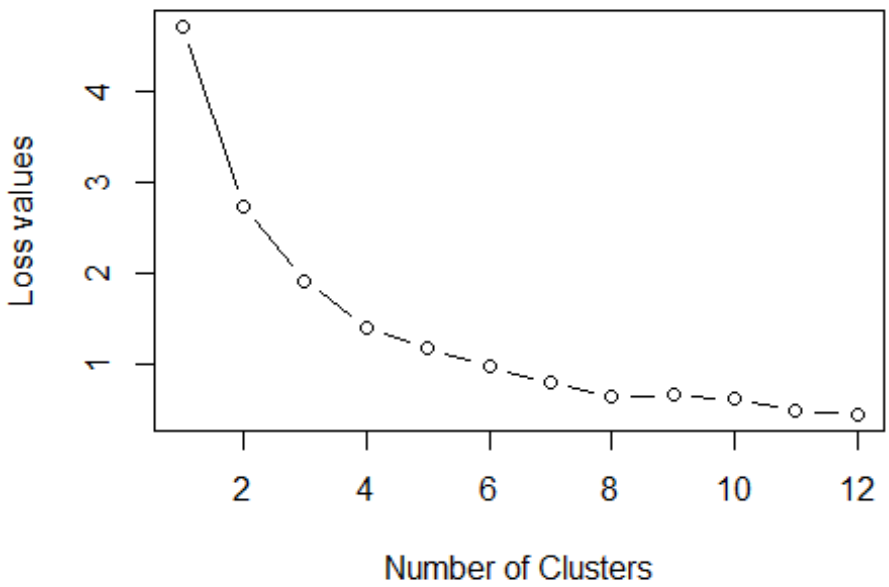


K-means with 5 clusters.



We see from the 5-cluster model some distinct clusters, with the top portion of the data being its own cluster, the middle part seems to be split into three separate clusters, and the last cluster mostly near the southeast portion.

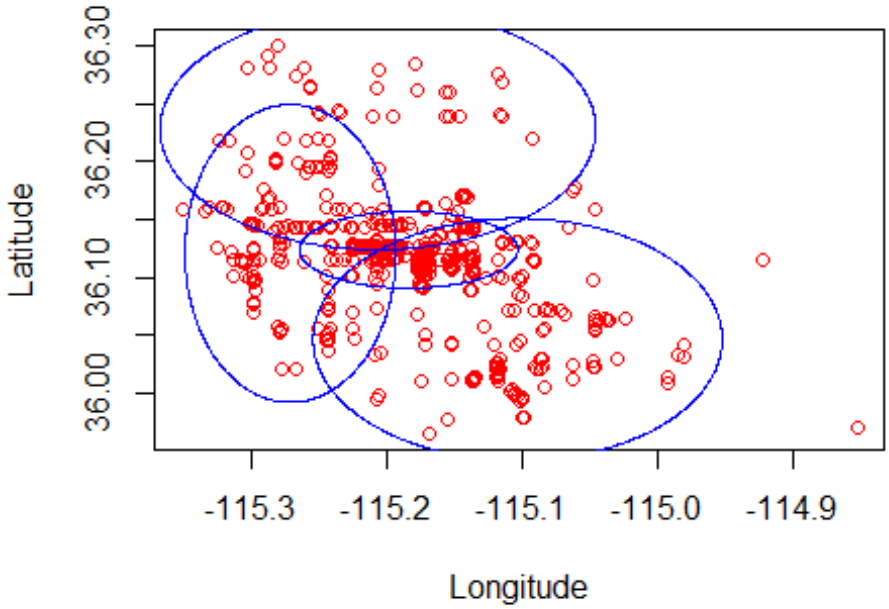
We will now use the elbow method to select an appropriate number of clusters for the data.



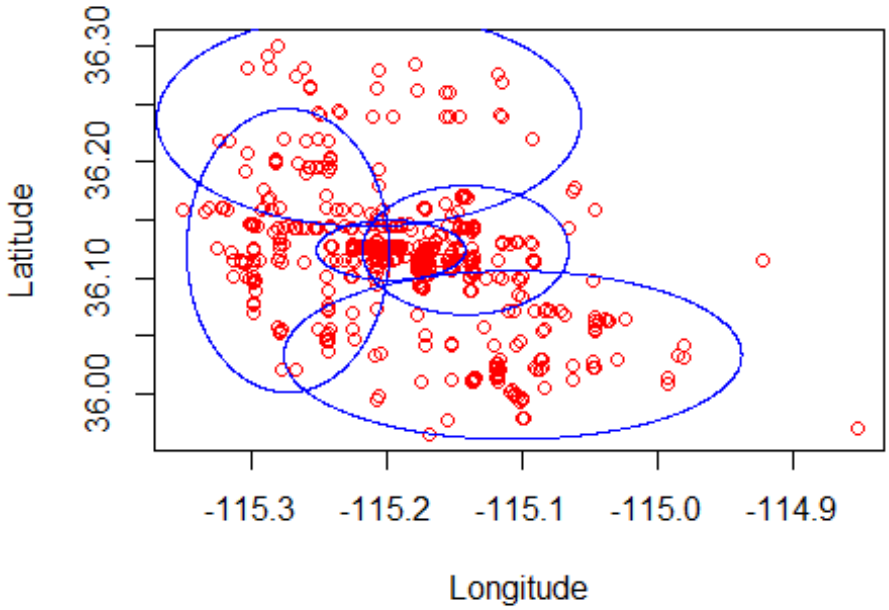
We see from these loss values that there is a relatively steep decline in the loss values until around 4 or 5 clusters. Either of these values would be a good number of clusters to balance a model's simplicity while minimizing loss.

We will now consider a Gaussian mixture model as the basis for our clustering. We will begin with clustering by the same number of clusters we used with K-means.

With 4 clusters.

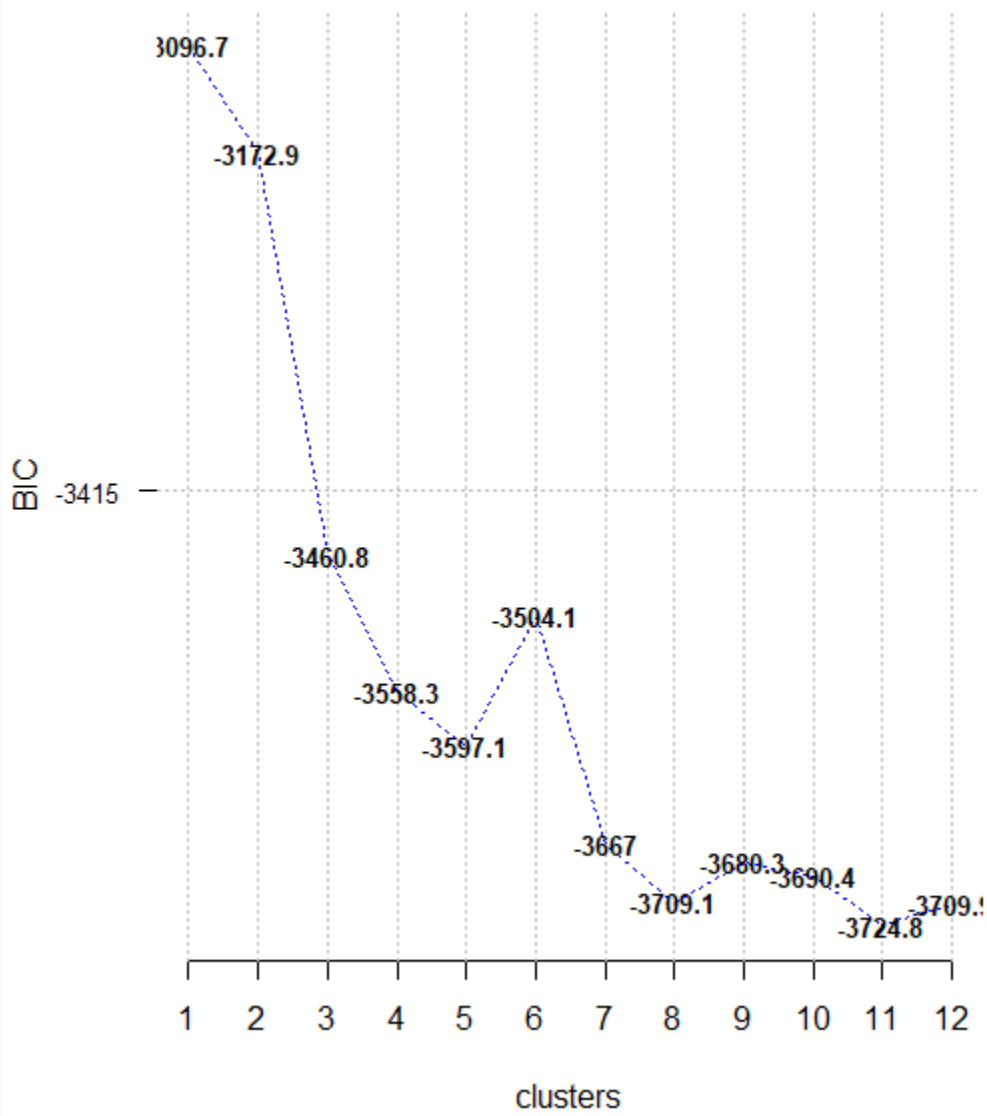


With 5 clusters.



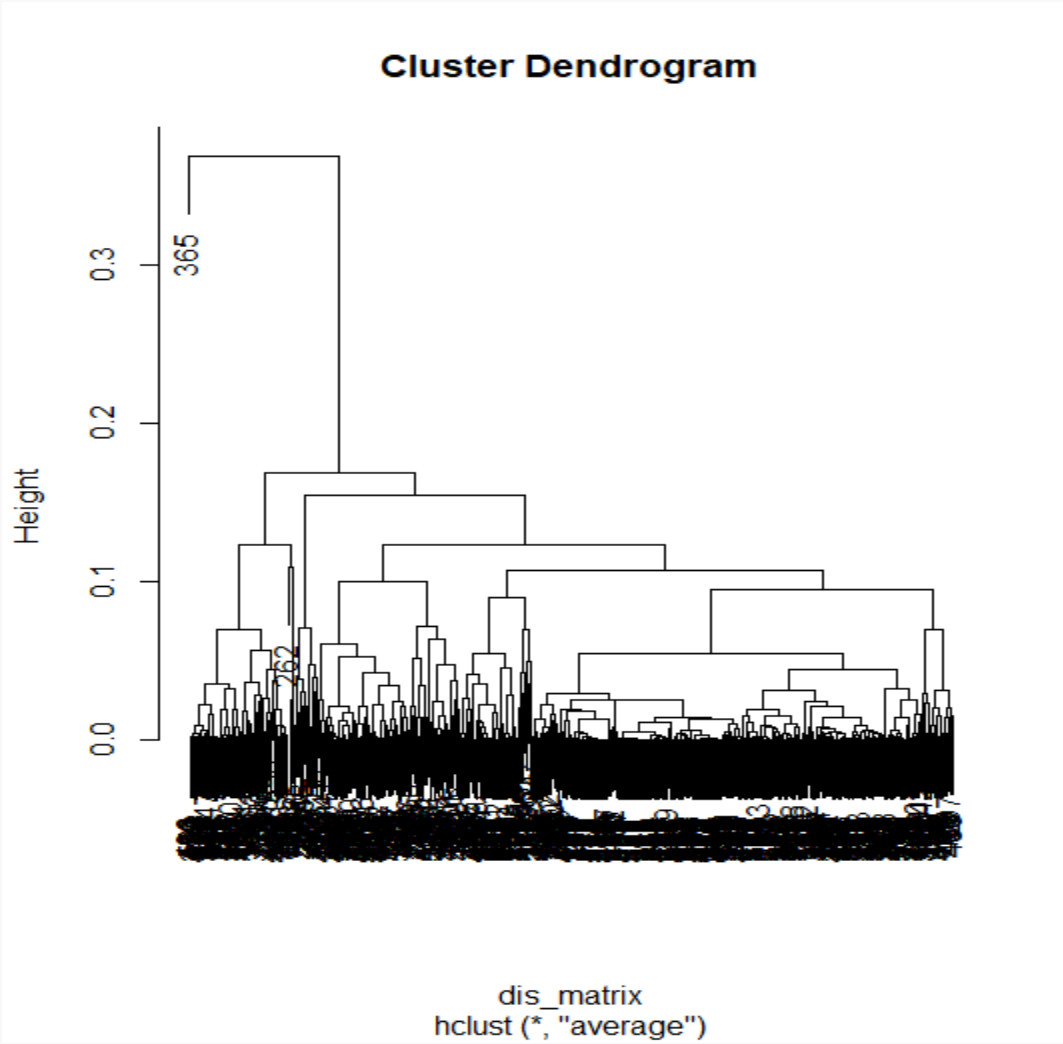
For both of these models above, there is some significant overlap between the ellipses representing the clusters of data. Furthermore, we notice both these models place a cluster near the center of the location data, which, when compared to images from Google Earth, matches approximately where The Strip in Las Vegas is located. This suggests there is a cluster of Asian restaurants near where many of the casinos are, and this is not a surprising result given the large amount of gambling that goes on there. There are lots of restaurants on The Strip, so it is also not surprising that there is a cluster of Asian restaurants there. However, since many different kinds of people pass through The Strip, it is interesting that there exists a high concentration of Asian restaurants to cater to different people.

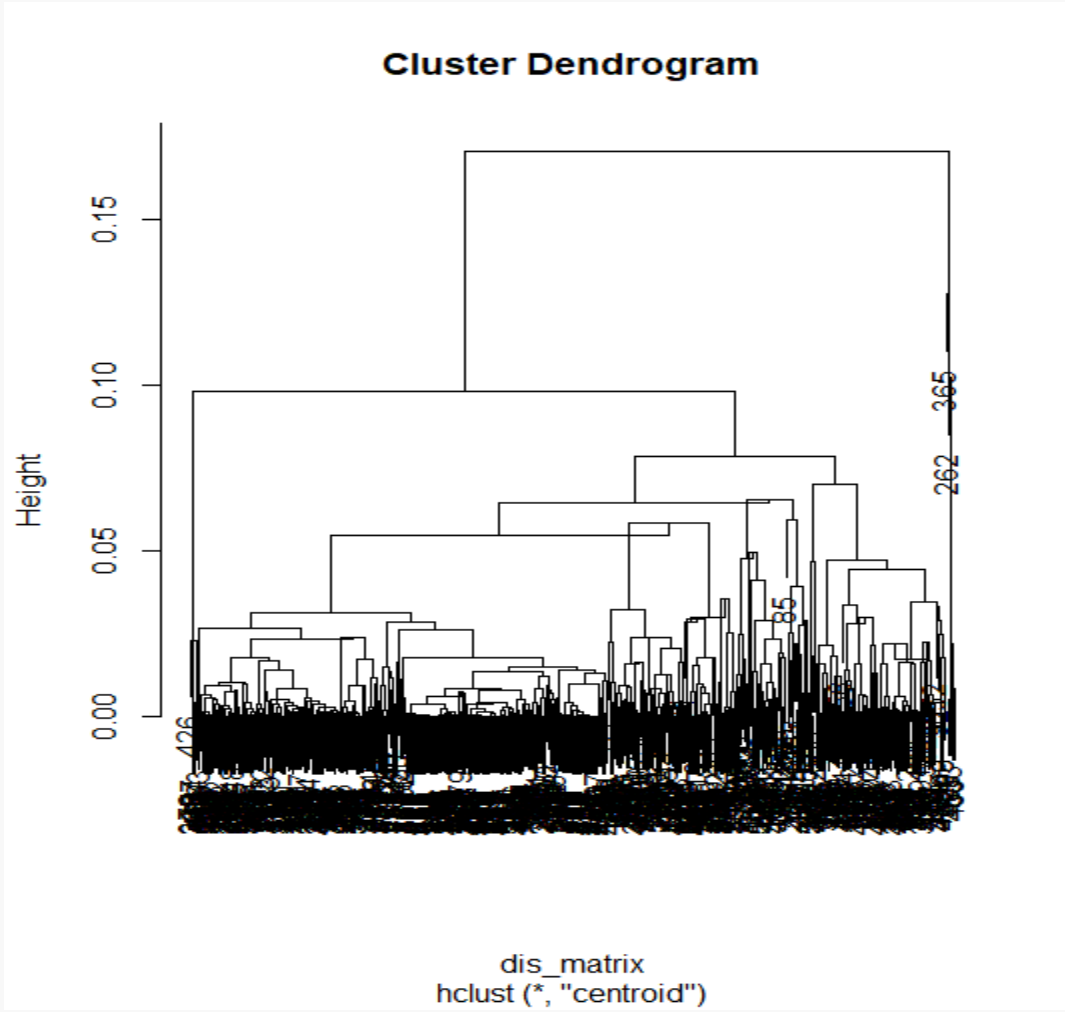
We will now determine the optimal amount of clusters using Gaussian mixture models.



To strike a balance between model simplicity and minimizing errors, we chose 4 to 5 clusters for the restaurants because those are the values where the decline in BIC becomes less steep. Even though the lowest BIC is when there are 10 clusters, we chose not to have 10 clusters because the model may be difficult to draw any conclusions from.

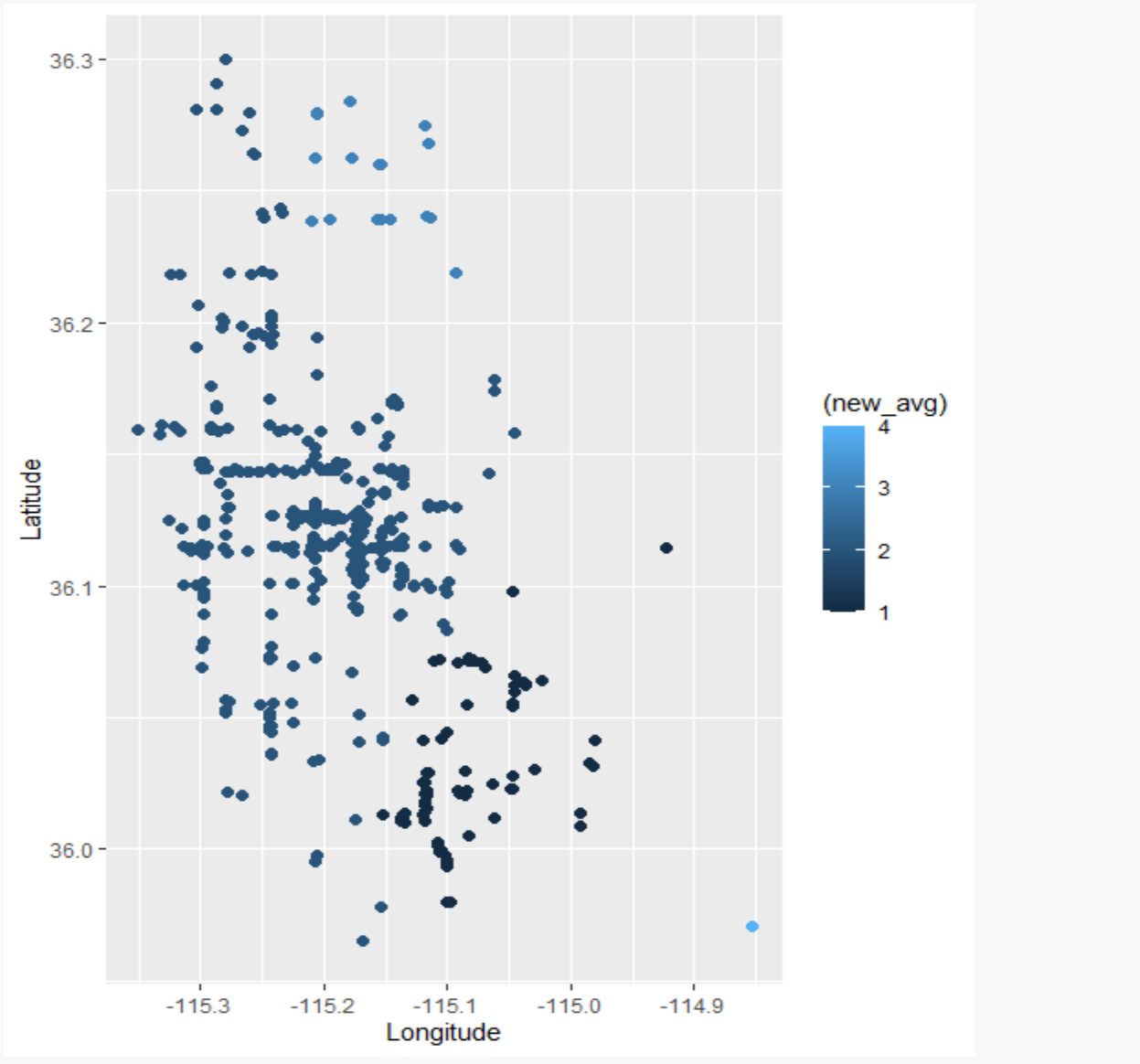
Finally, we will use hierarchical clustering with both average and centroid linkage because these methods seemed to be the most fruitful in class.

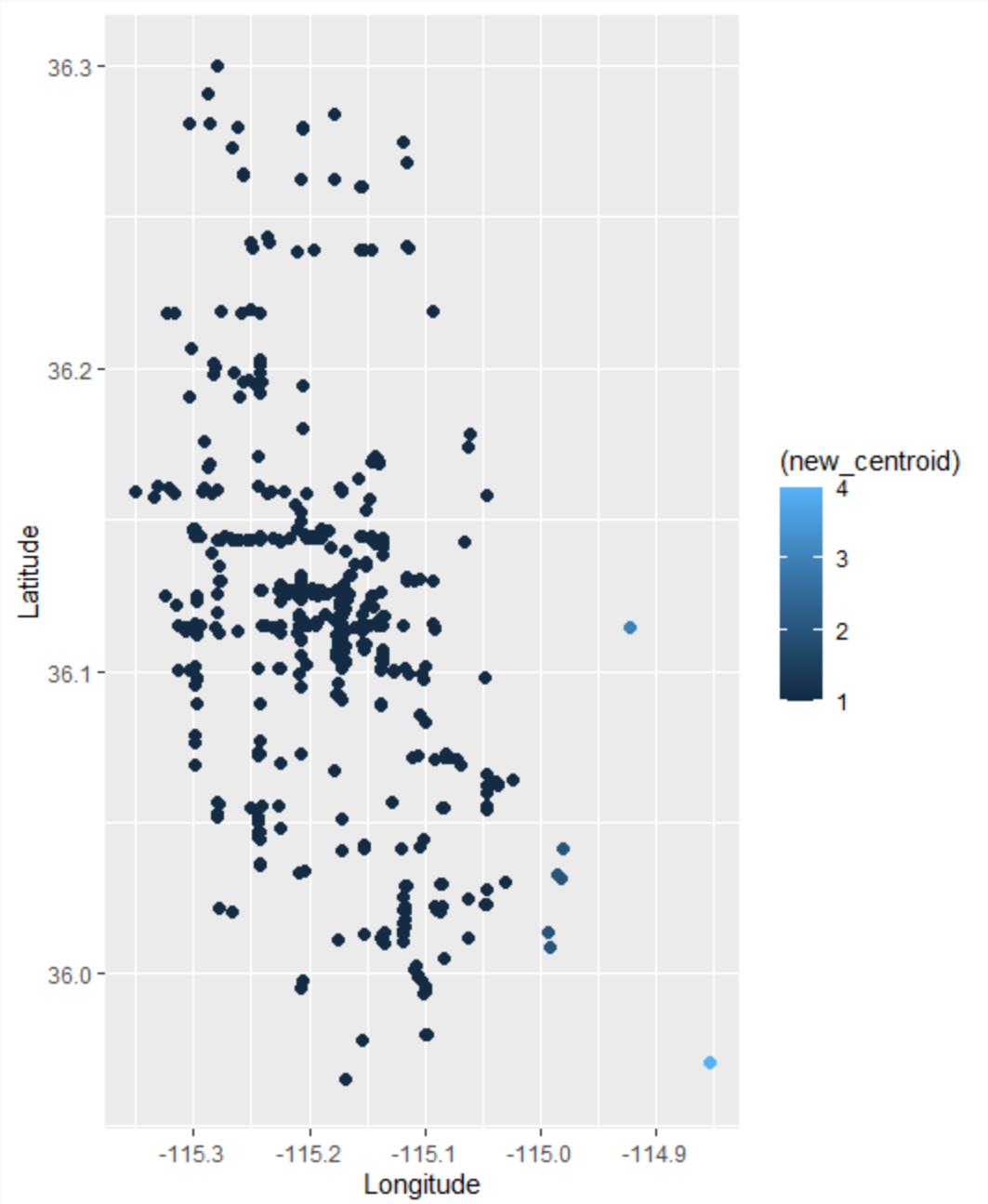




Both of these dendrograms are somewhat difficult to interpret because the dendrograms are bunched up together at the lowest level. Additionally, with our centroid dendrogram, the results are not readily interpretable given there are several inversion points where the tree goes back up into itself.

We will now determine our labels for the location points by first using our average linkage.

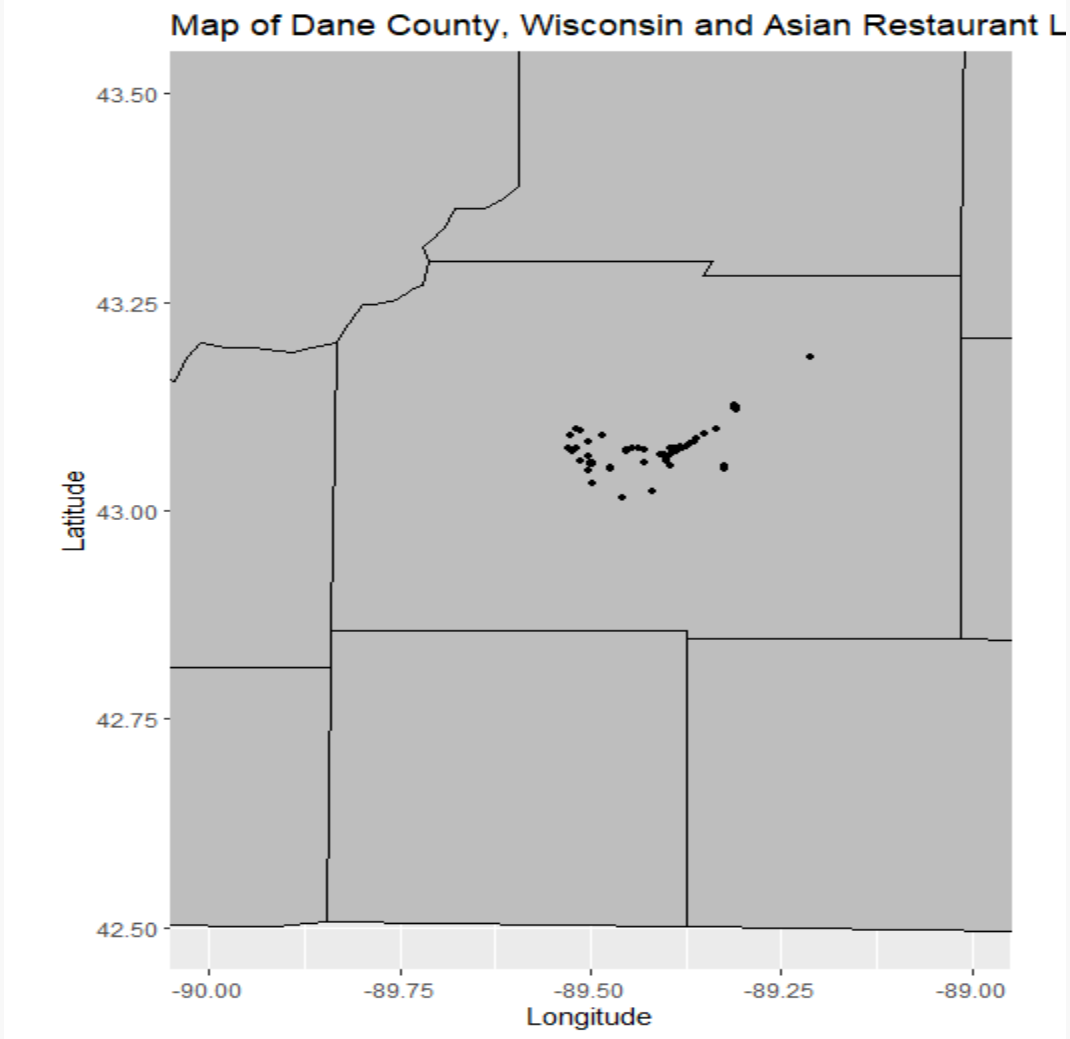

36.3
36.2
36.1
36.0
Latitude
-115.3
-115.2
-115.1
-115.0
-114.9
Longitude
(new_avg)
4
3
2
1



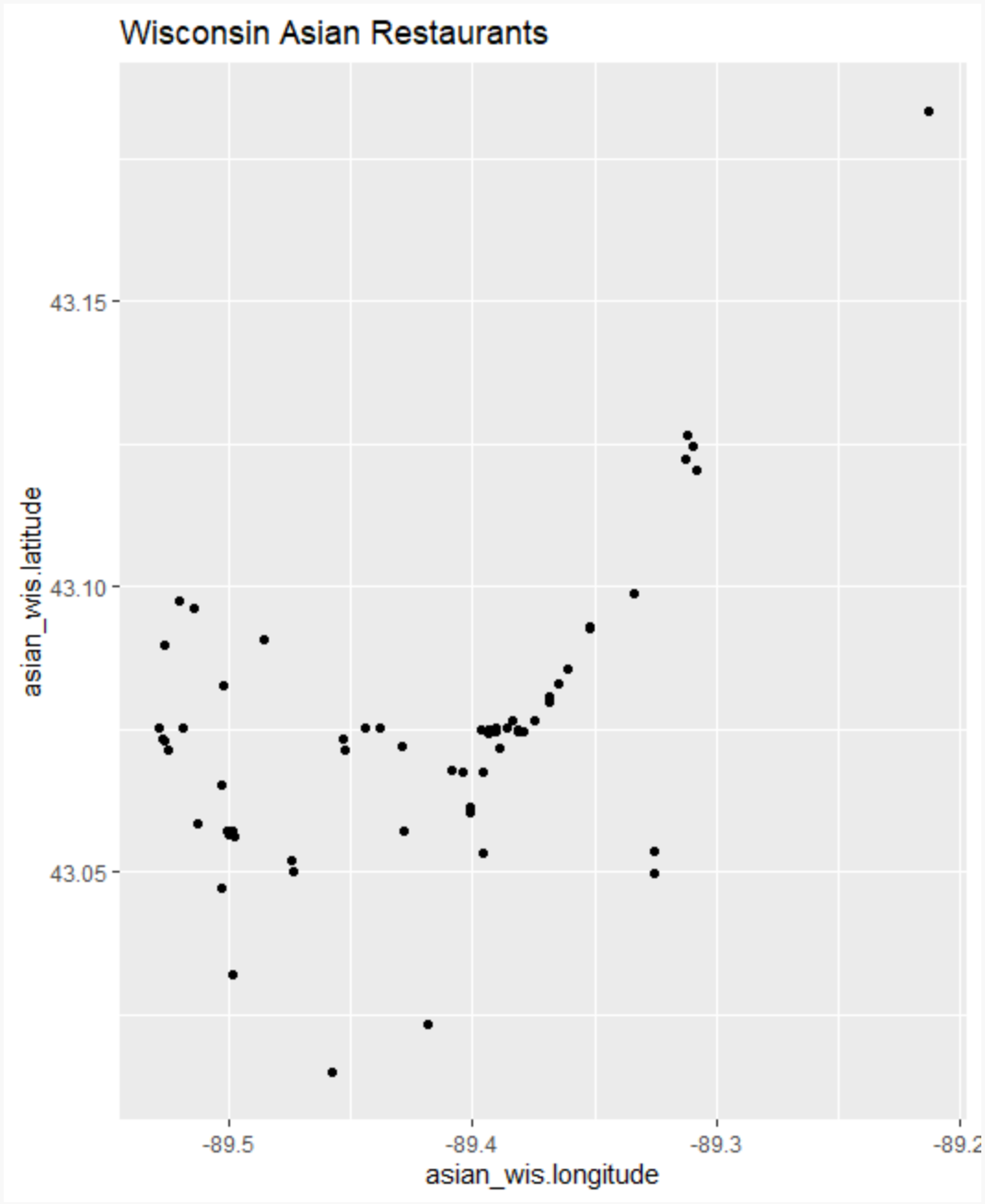
Our centroid method produces some very peculiar results that are not very useful because most of the data are being classified in a single cluster. This is the same case, though to a lesser extent, when we use average linkage, with many of the locations simply being clustered together. Furthermore, both linkage types categorize a single data point in the lower right quadrant as a single cluster, suggesting that neither method is well equipped to deal with the data.

Overall, K-means clustering and hierarchical clustering using average linkage produce results that cluster the data fairly well. This is indicated with clear clusters in various parts of Clark County that shows a cluster in those models, which suggests a high concentration near The Strip.

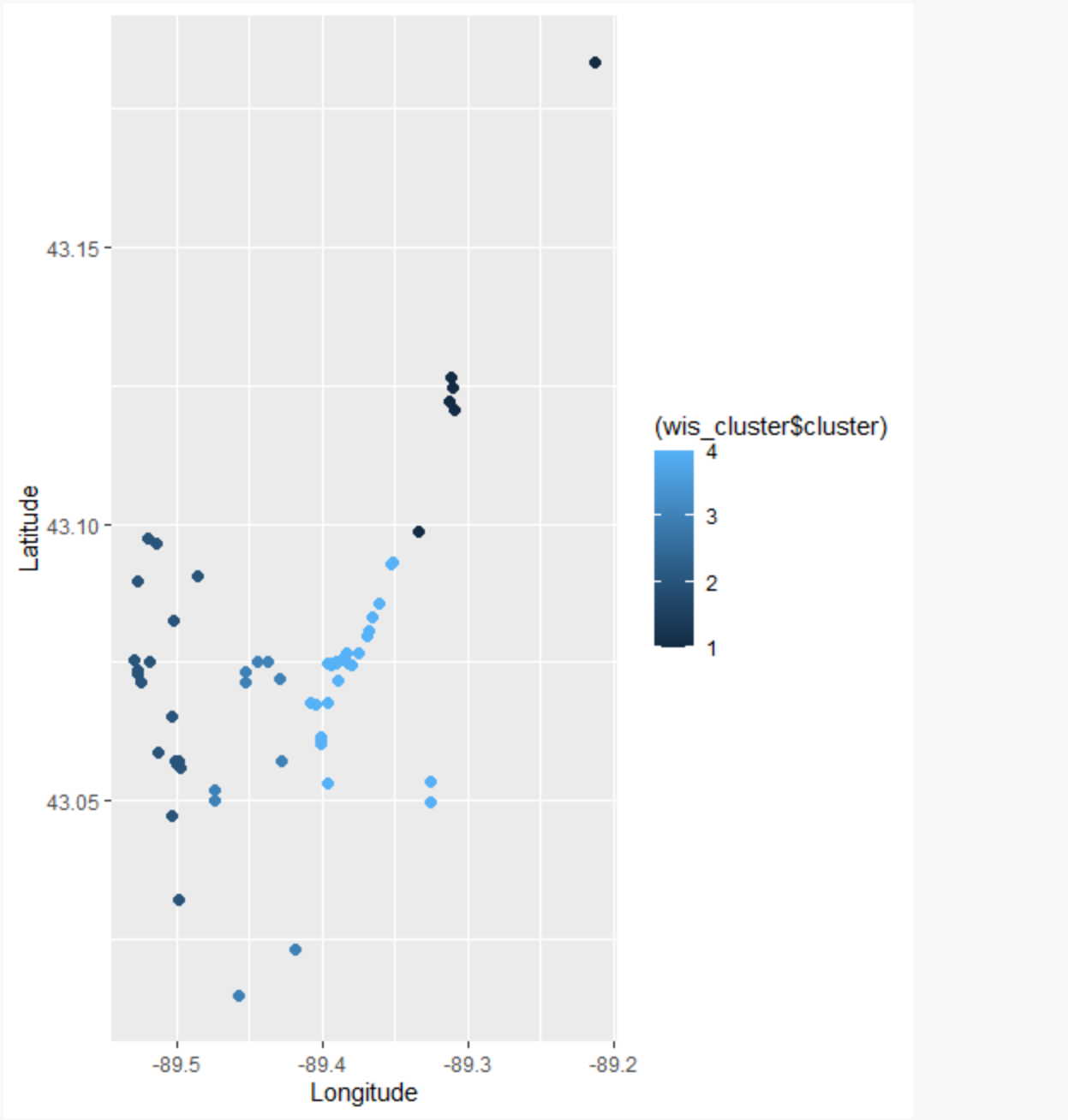
Now we will cluster Asian restaurants in Wisconsin



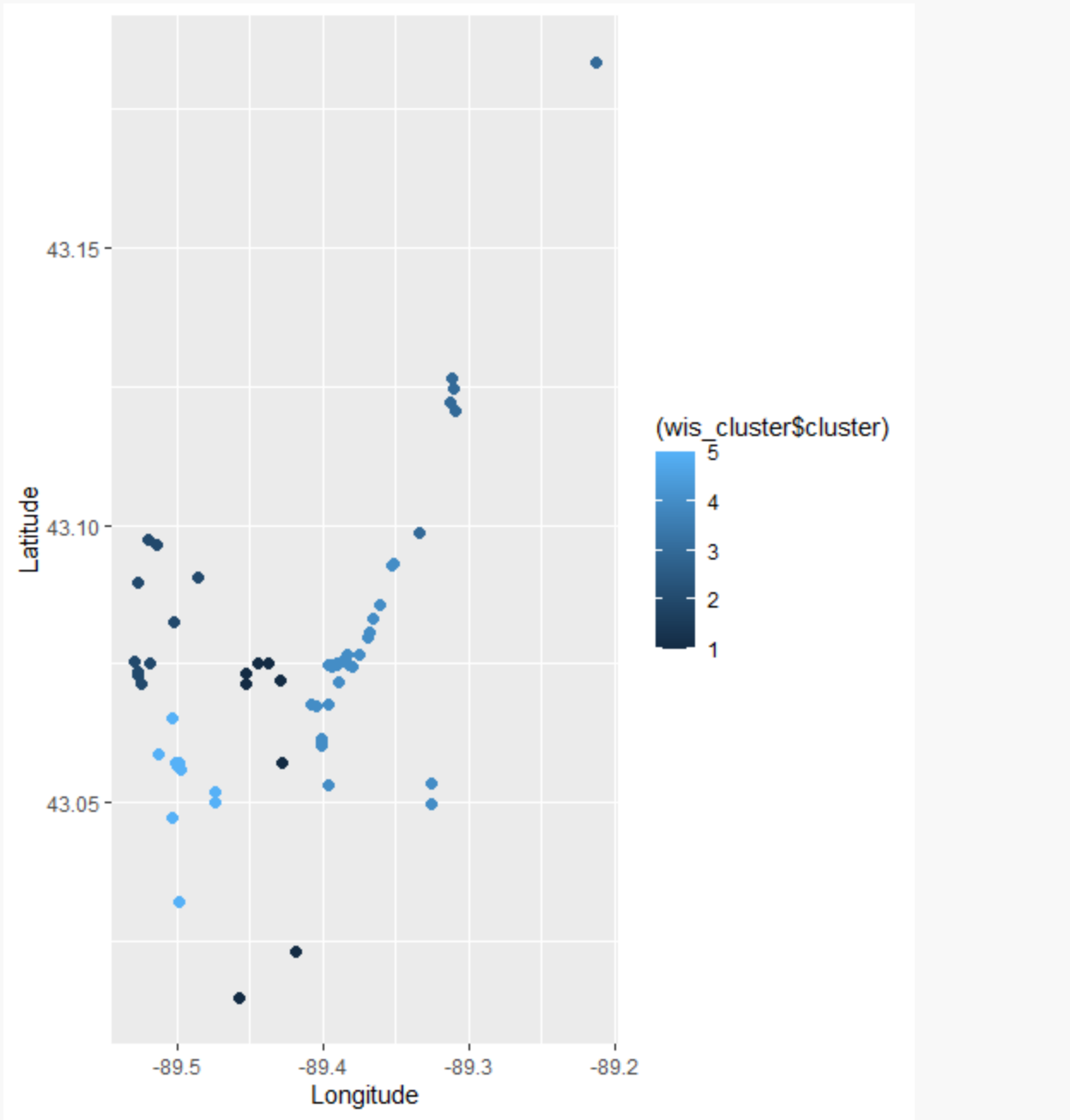
We will make a data frame for the longitudinal and latitudinal data for Asian restaurants in Wisconsin.



We will now use K-means clustering to analyze the location data, starting with 4 clusters.

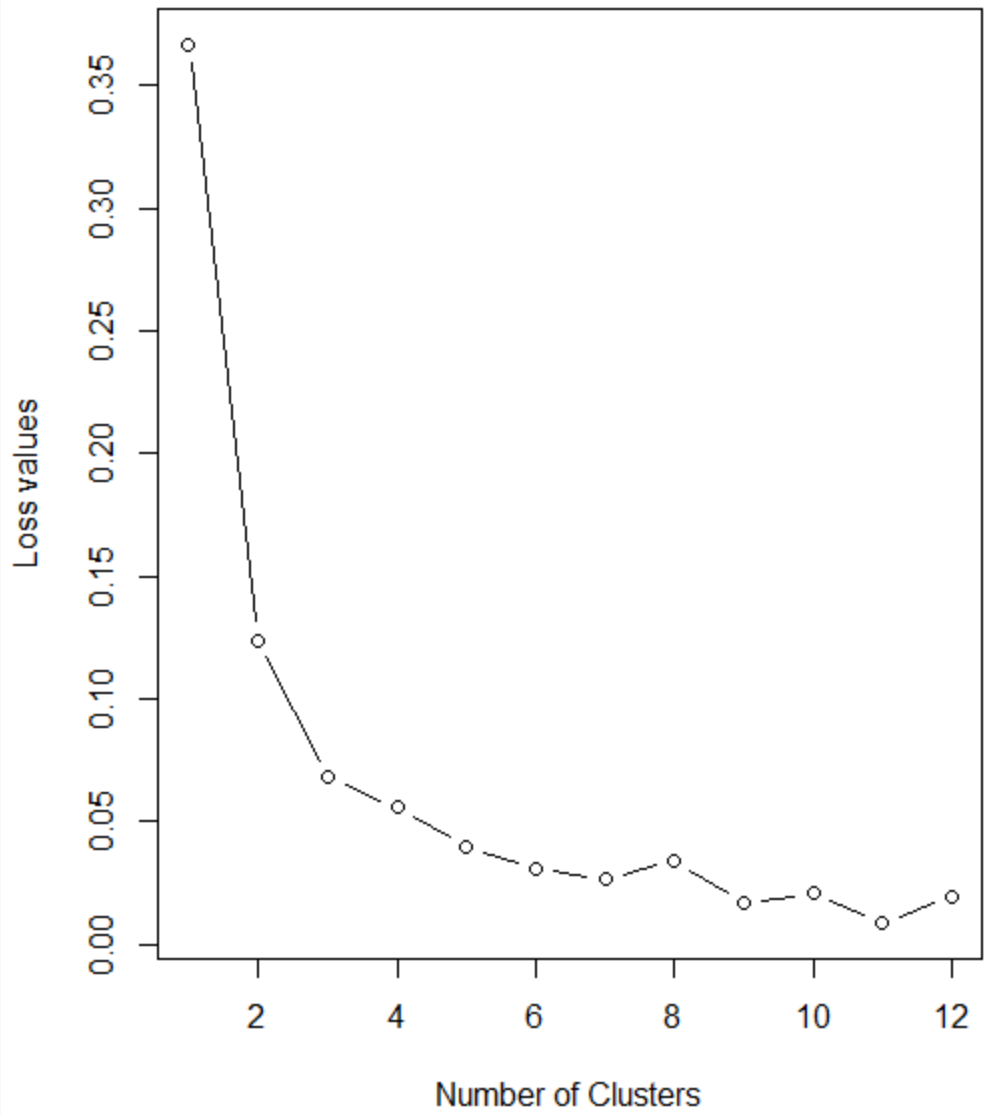


K-means with 5 clusters



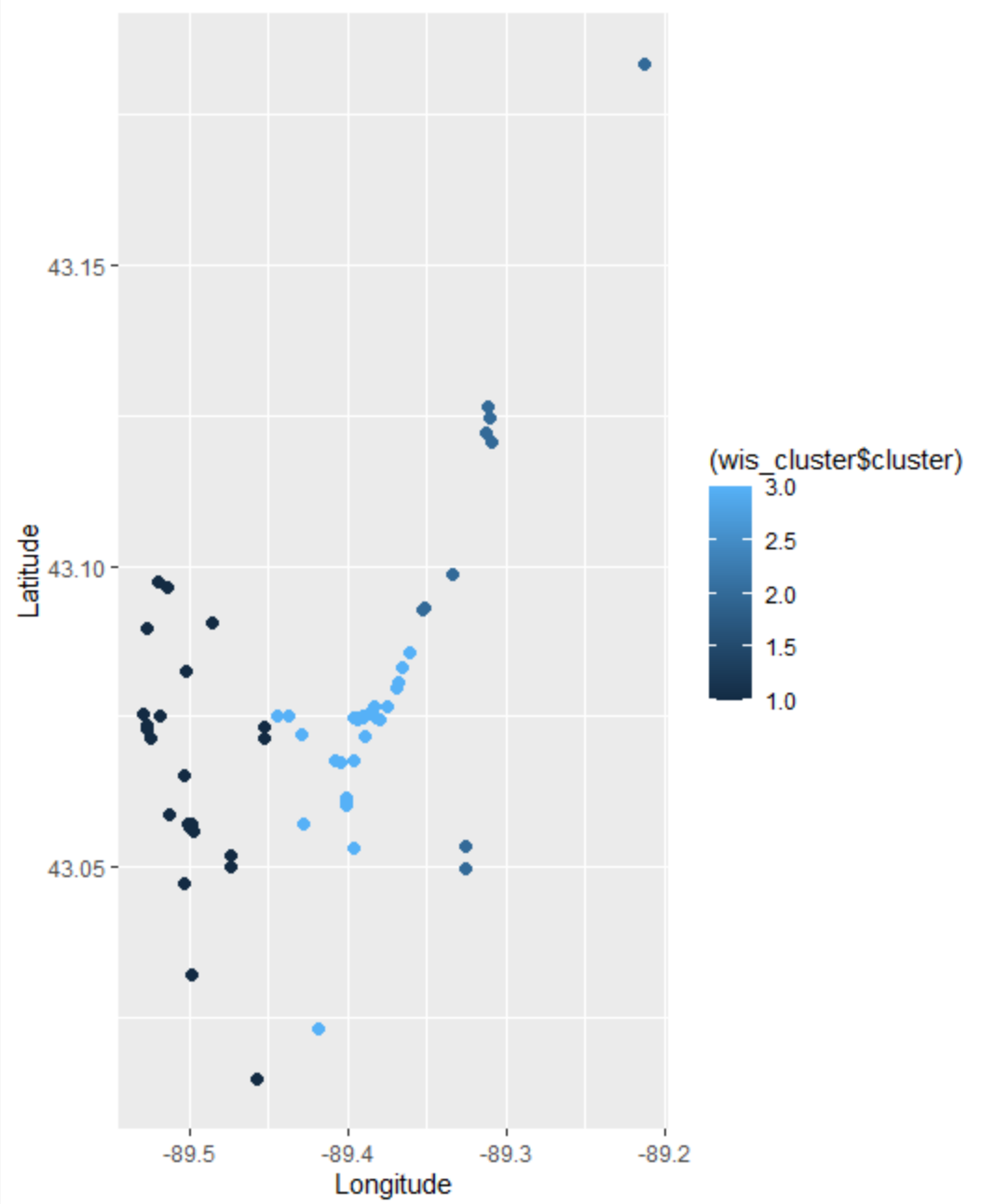
Our cluster models suggest that the few points at the higher latitudes should be clustered together, so we split up the remaining locations near the center of the plots into fairly evenly distributed groups.

The elbow method will now be employed to select the optimal number of clusters.



From our loss values, it appears that either 3 or 4 clusters are optimal, which seems to be lower than the optimal clusters we found for Nevada businesses.

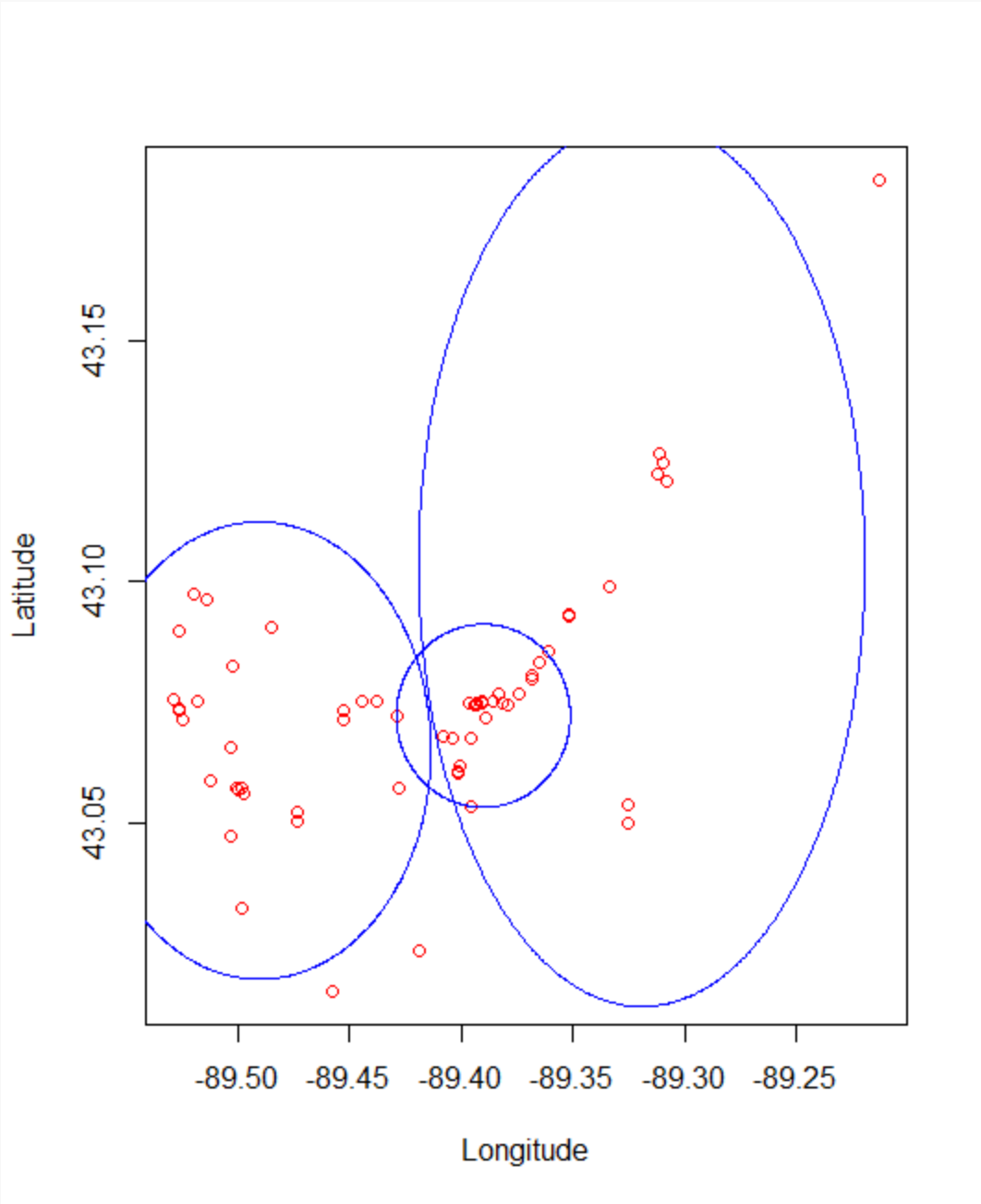
We chose K-means with 3 clusters since we already clustered with 4 clusters.



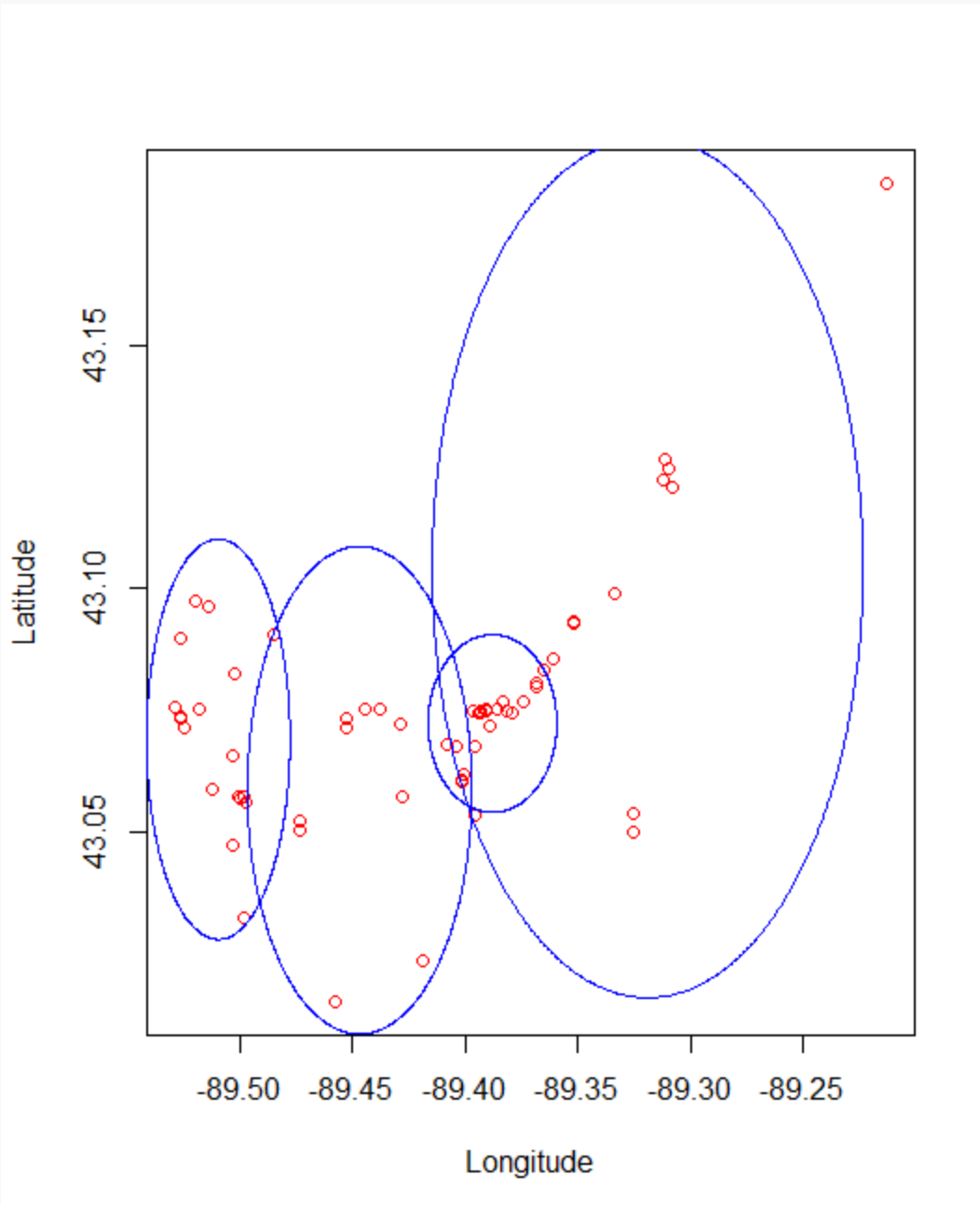
This clustering model performed reasonably well.

Now, we will use a Gaussian mixture model to cluster.

With 3 clusters:

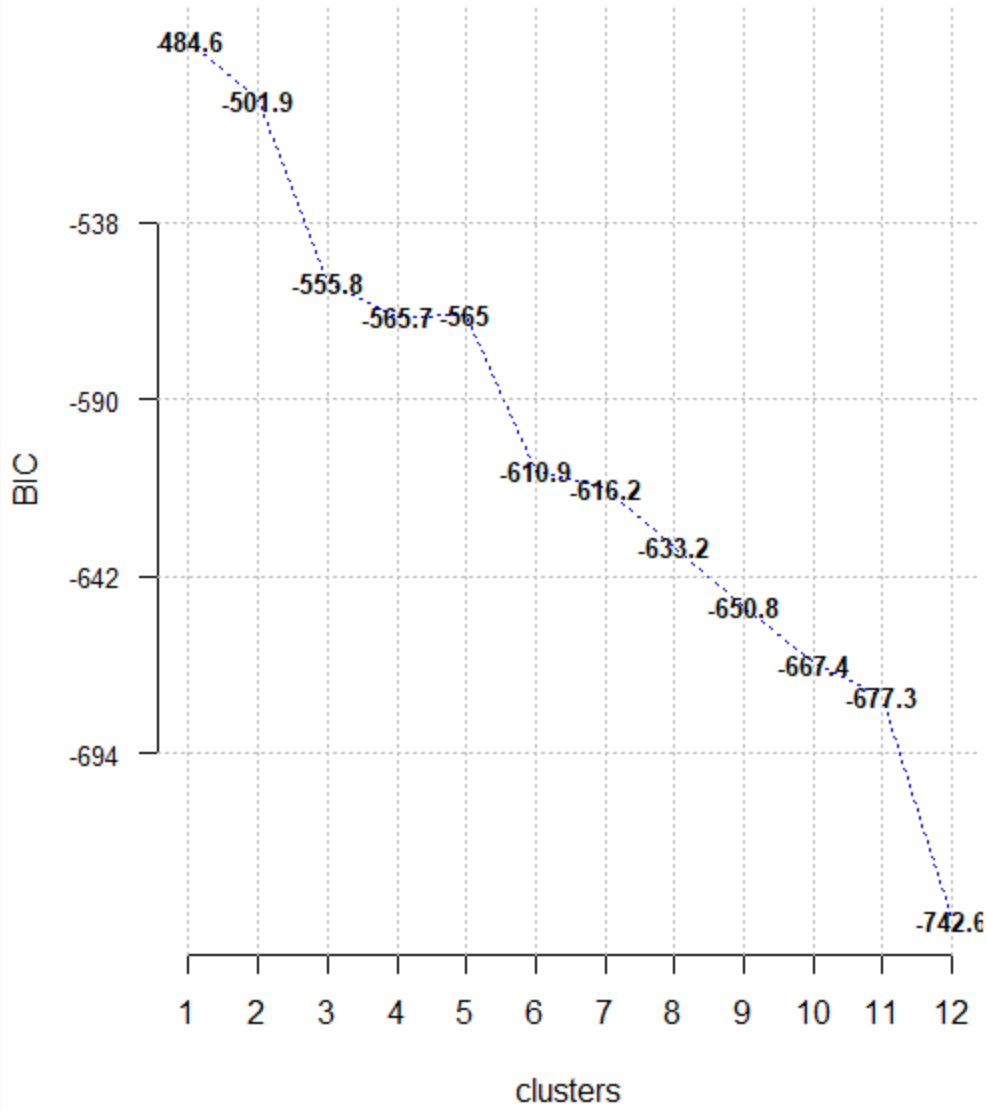


With 4 clusters:



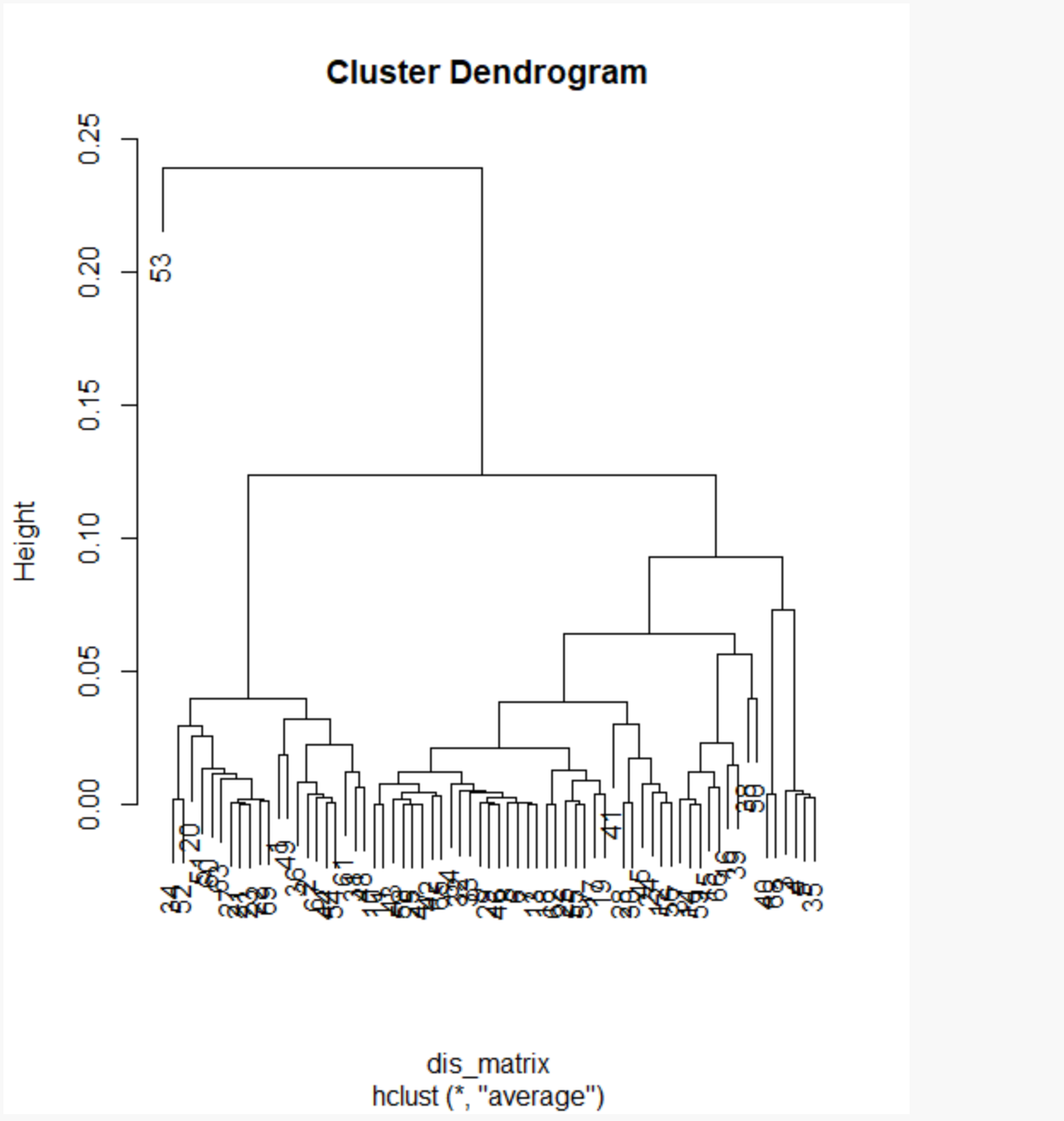
We see that there is less overlap in these clusters than in our Nevada set. However, as with both sets, the Gaussian mixture model dedicates a cluster to the center where many businesses are located.

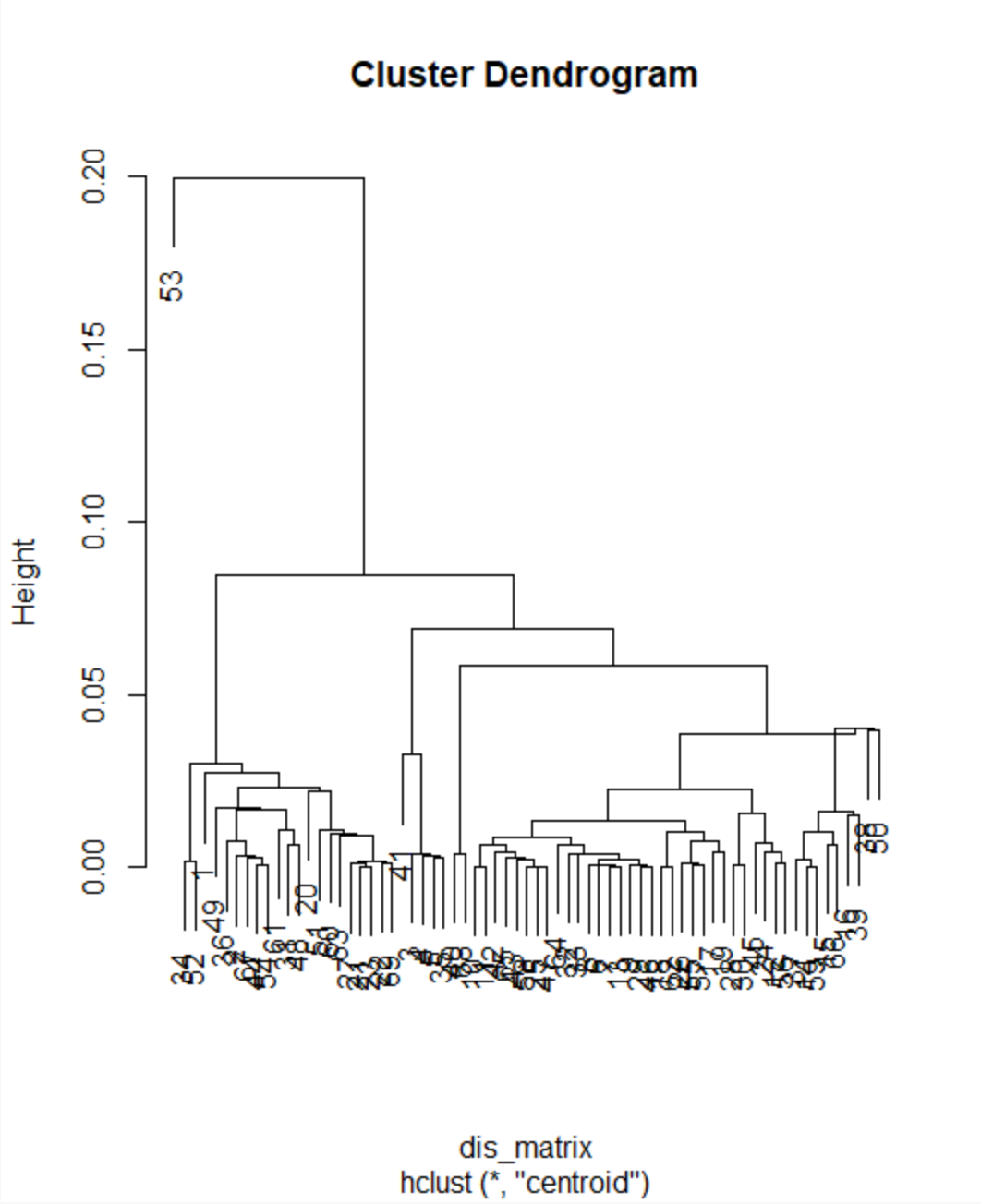
Let us now determine the optimal amount of clusters using Gaussian mixture models.



This graph indicates that the higher the number of clusters there are, the better our model fits the data. However, we also noticed that there is a fairly steep drop in terms of inaccuracies from 2 to 3 clusters and from 5 to 6 clusters. Since these results are a bit confusing, a Gaussian mixture model may not fit well with our data.

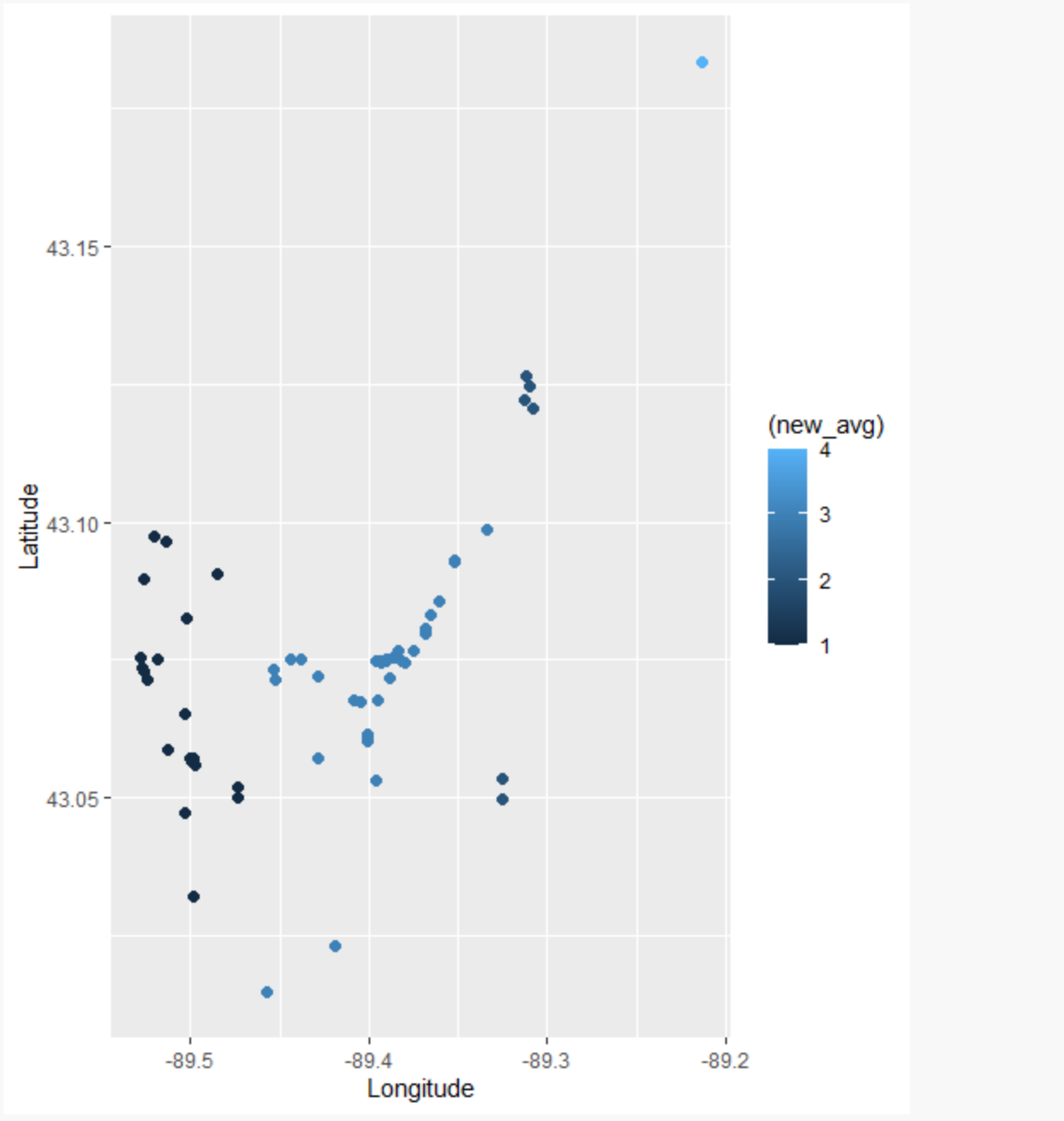
Finally, we will use hierarchical clustering methods using average and centroid linkage.

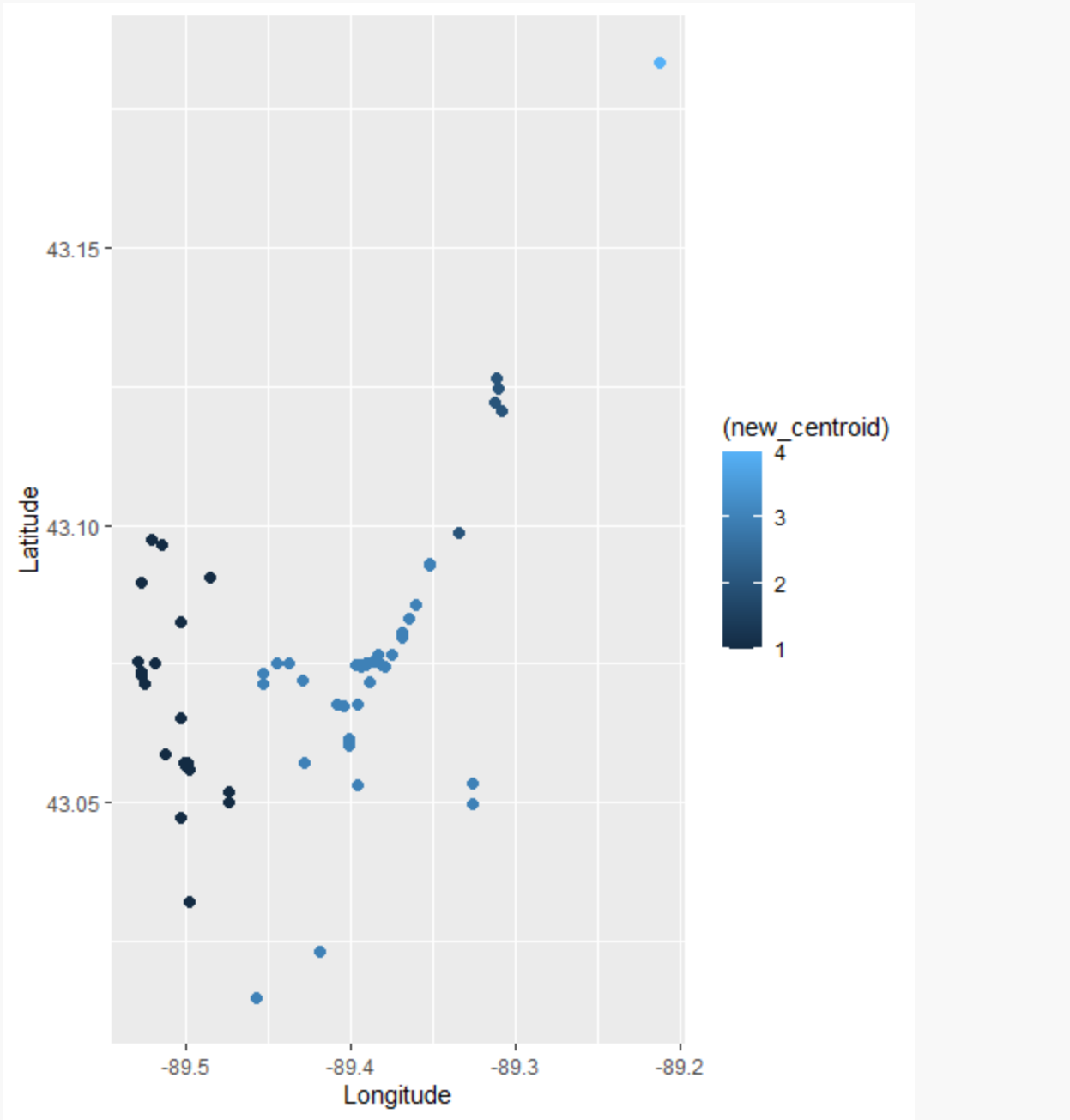

Cluster Dendrogram
Height
0.00
0.05
0.10
0.15
0.20
0.25
53
dis_matrix
hclust (*, "average")



Our dendrogram using average linkage is well-balanced while our centroid dendrogram has inversions that make it difficult to interpret.

Let us now label our location data according to each method.

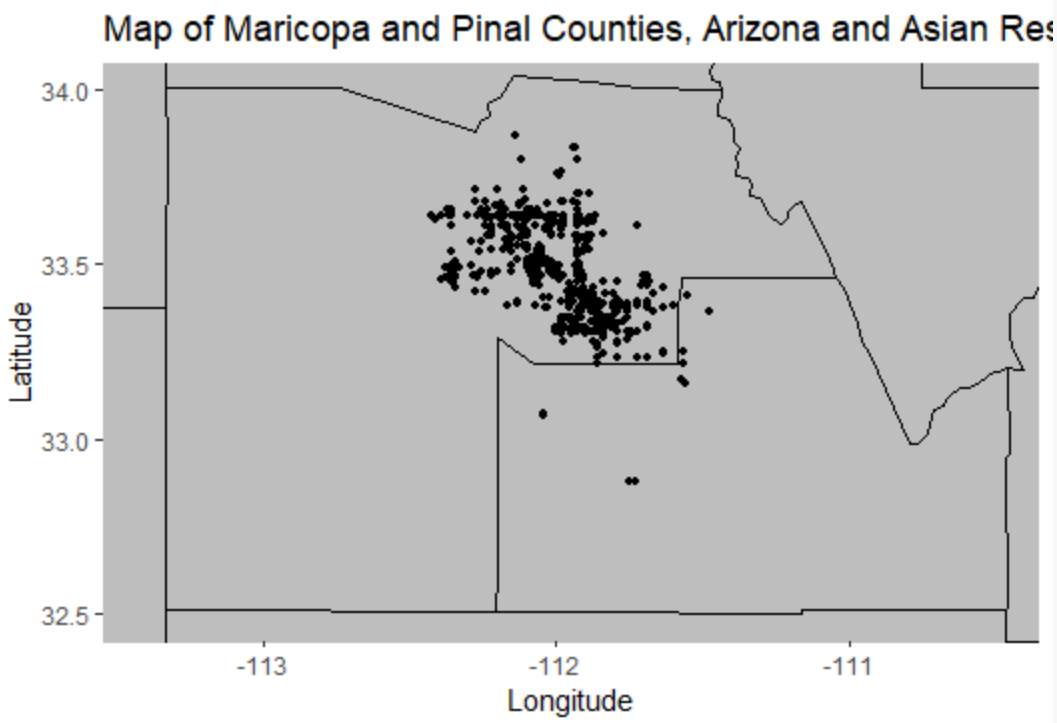

43.15
43.10
43.05
Latitude
-89.5
-89.4
-89.3
-89.2
Longitude
(new_avg)
4
3
2
1



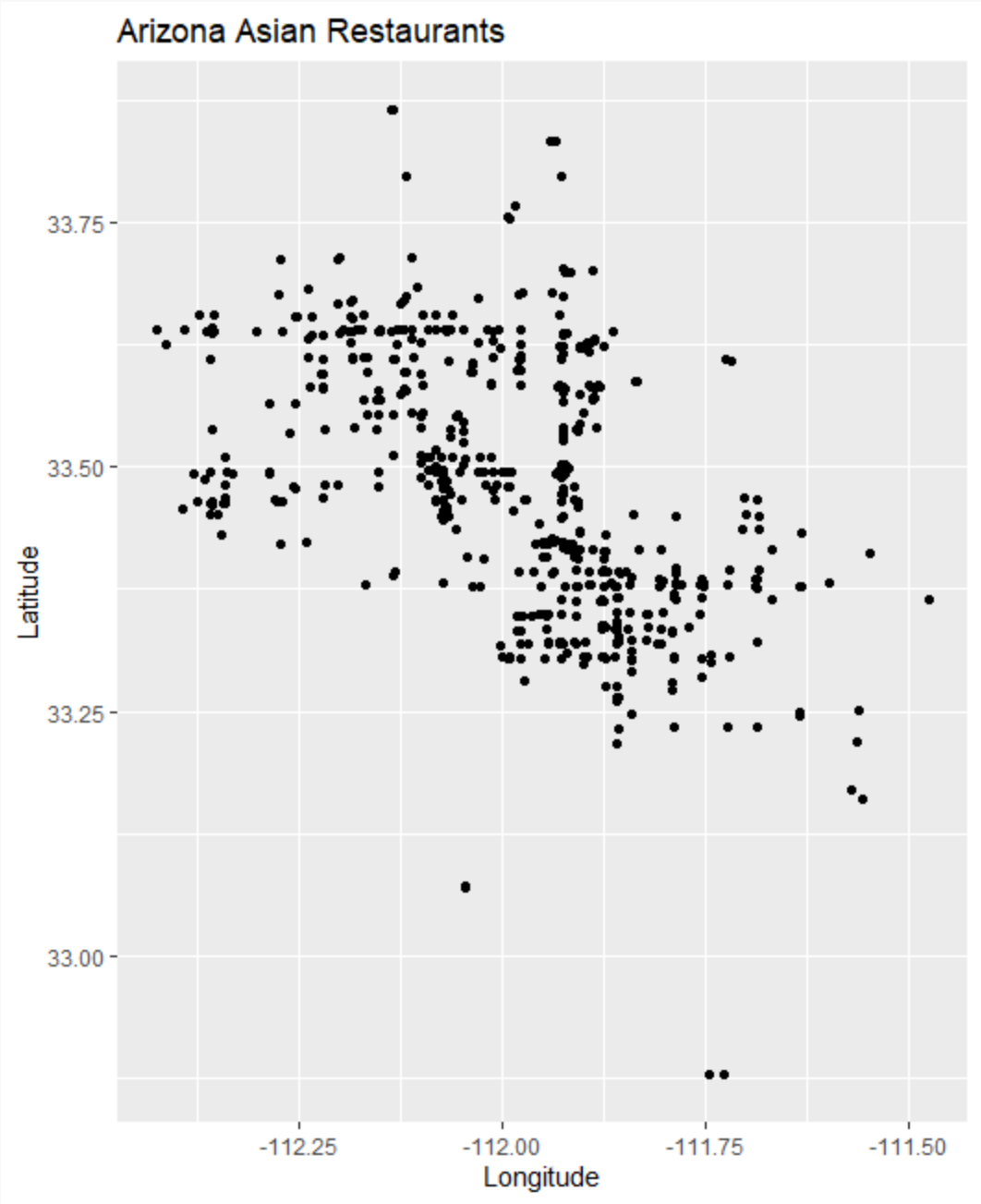
Both dendrograms, whether using average or centroid linkage, classify the uppermost restaurant location as its own cluster, suggesting that neither are well-suited for the data even though the 3 other clusters closer to the center of the plot clustered fairly well.

Overall, K-means seemed best for clustering the data because it balances potential outlier points in the uppermost latitude with the points near the center of our plots.

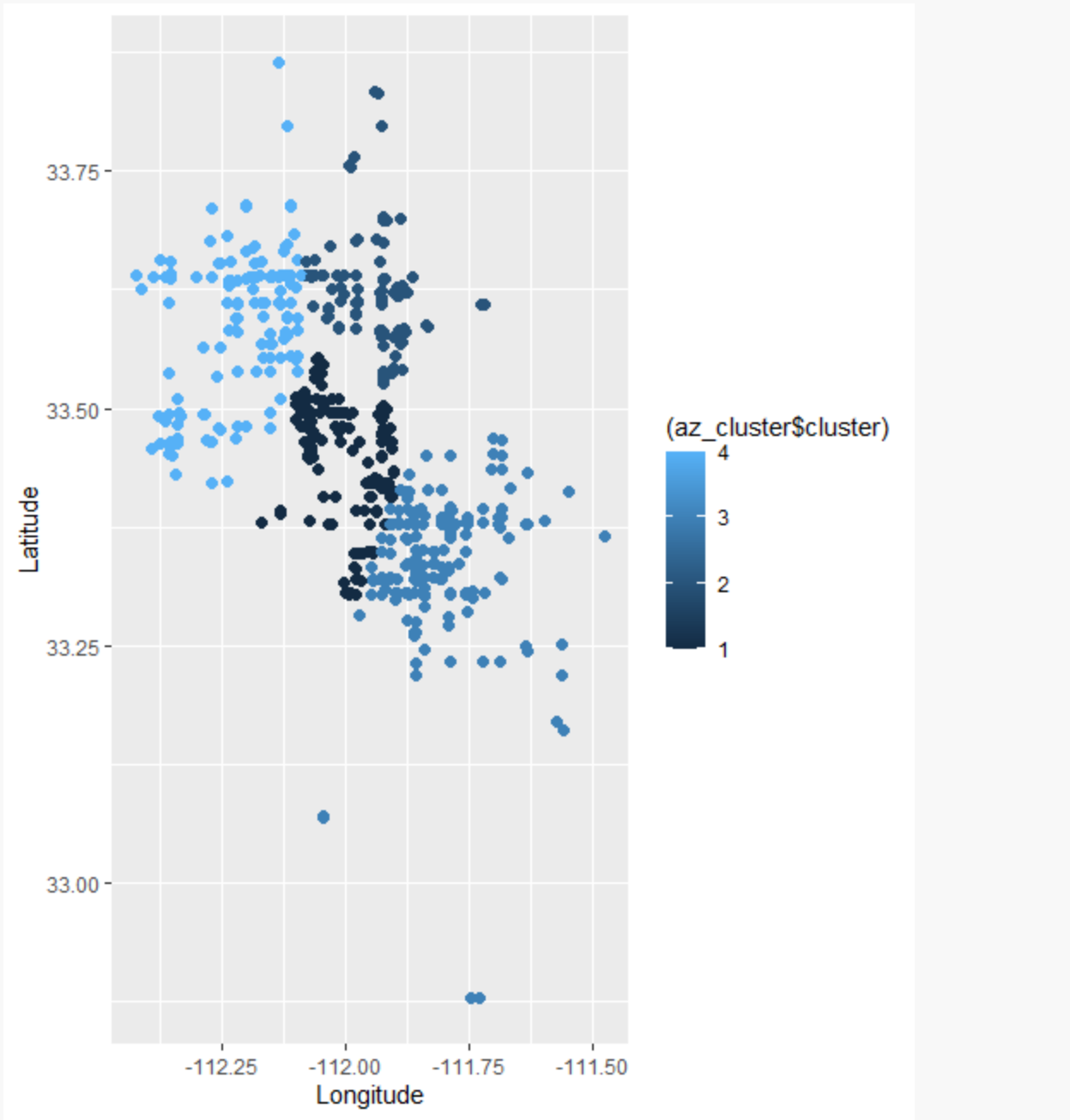
Finally, we will cluster for Asian restaurants in Arizona.



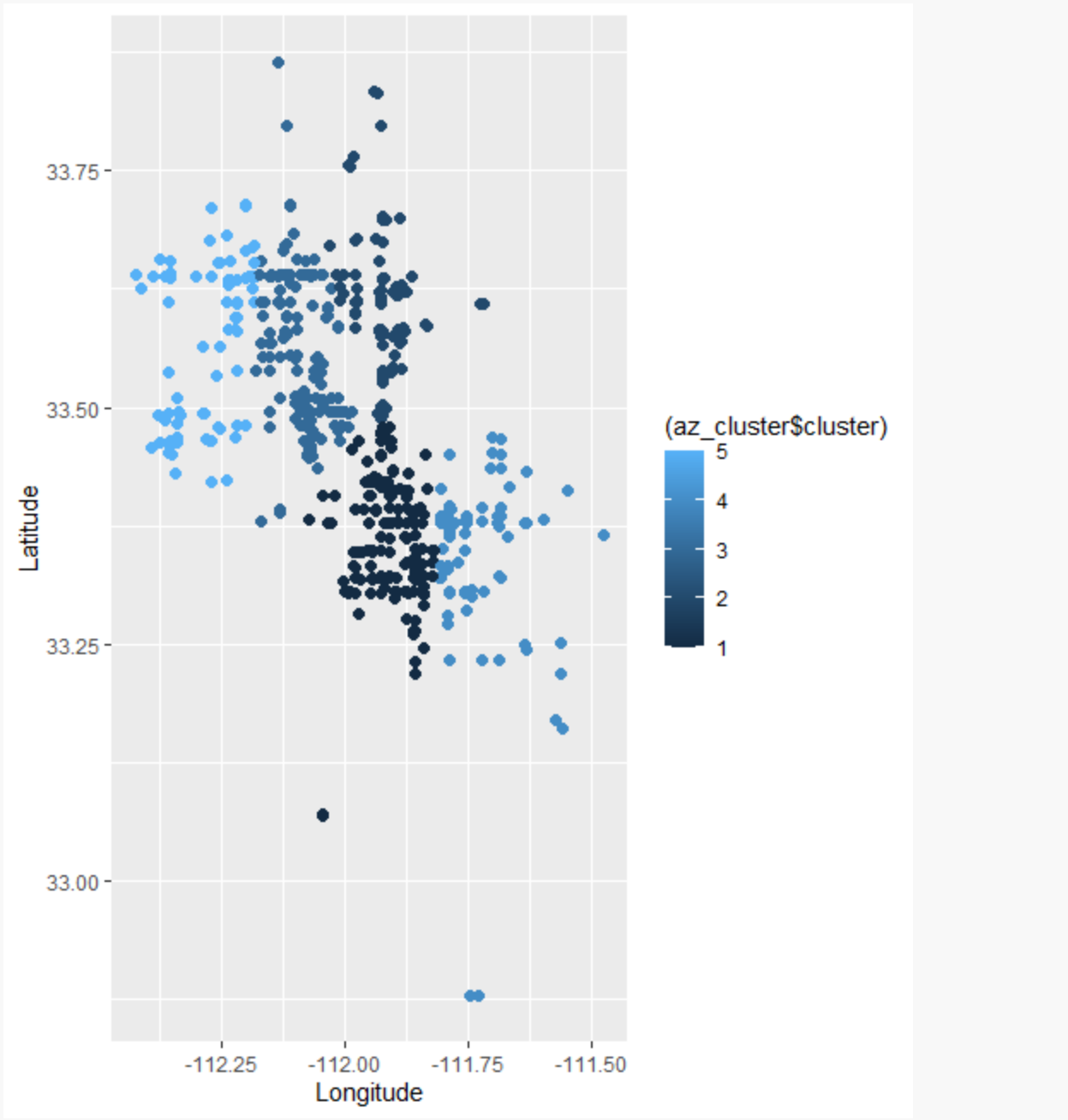
Again, we will make a data frame of only longitudinal and latitudinal data.



We will use K-means clustering to analyze the location data.
With 4 clusters:

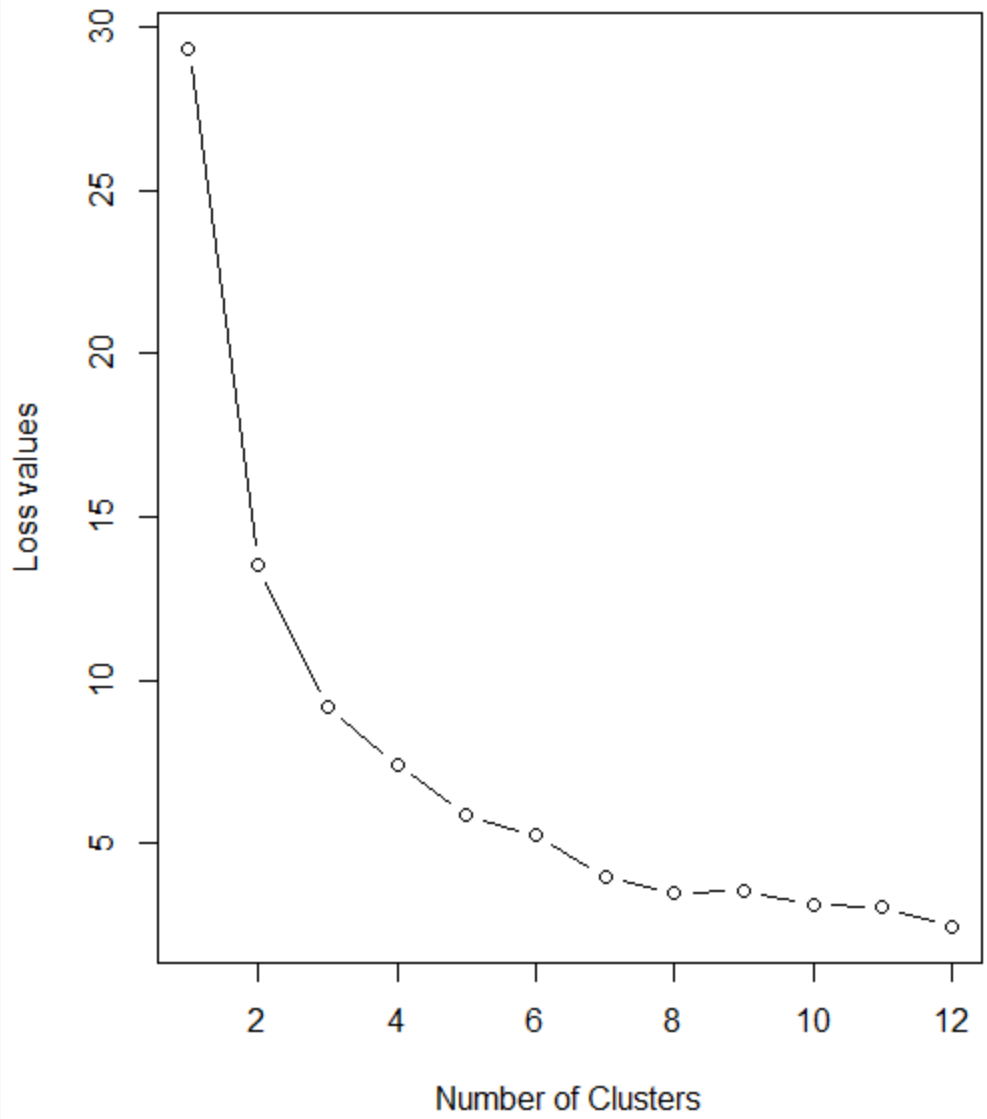


With 5 clusters:



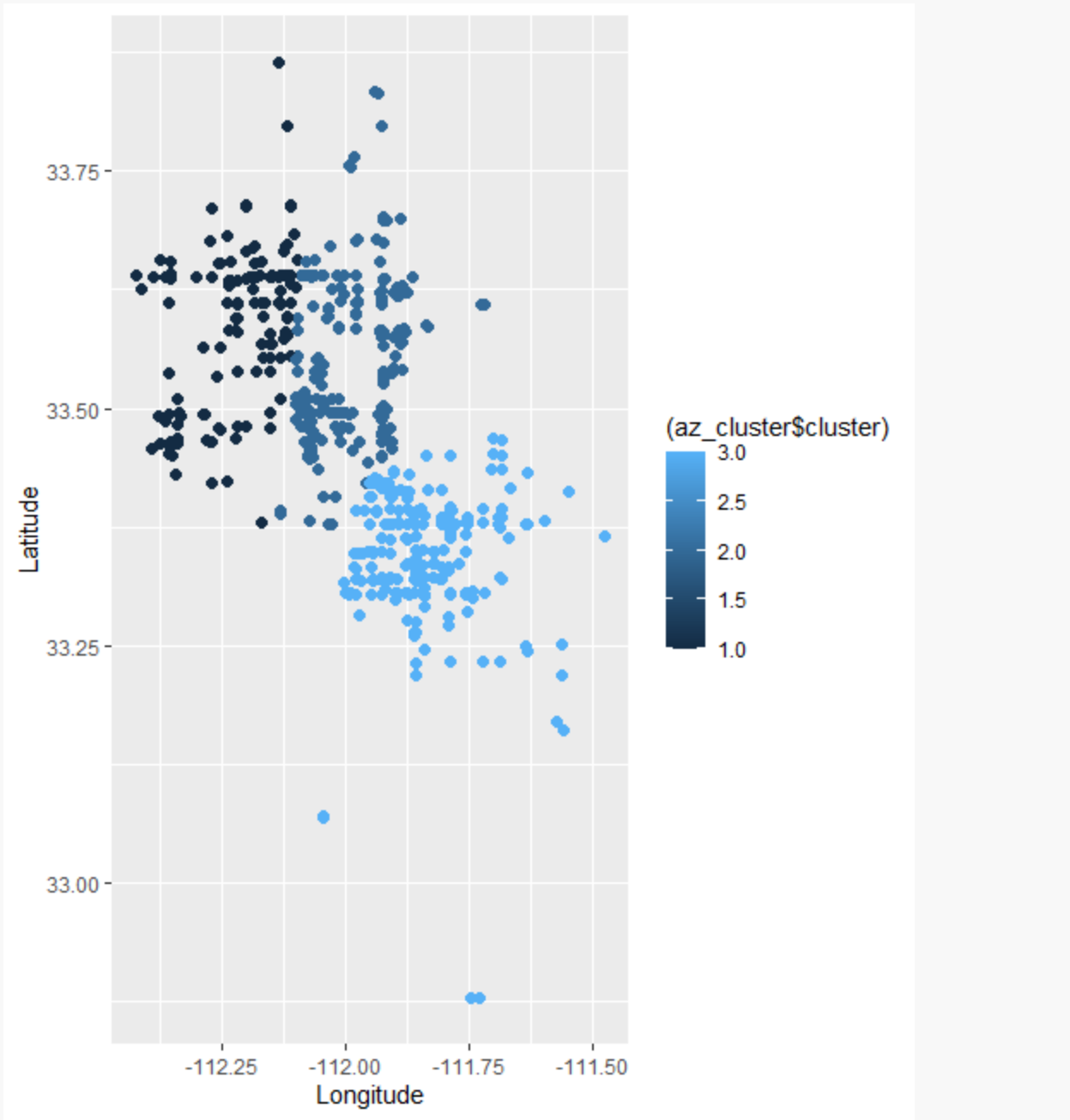
Both of these models appear to cluster the data fairly well, creating some fairly distinct clusters that appear to fit the data well.

Let us now find the optimal number of clusters.



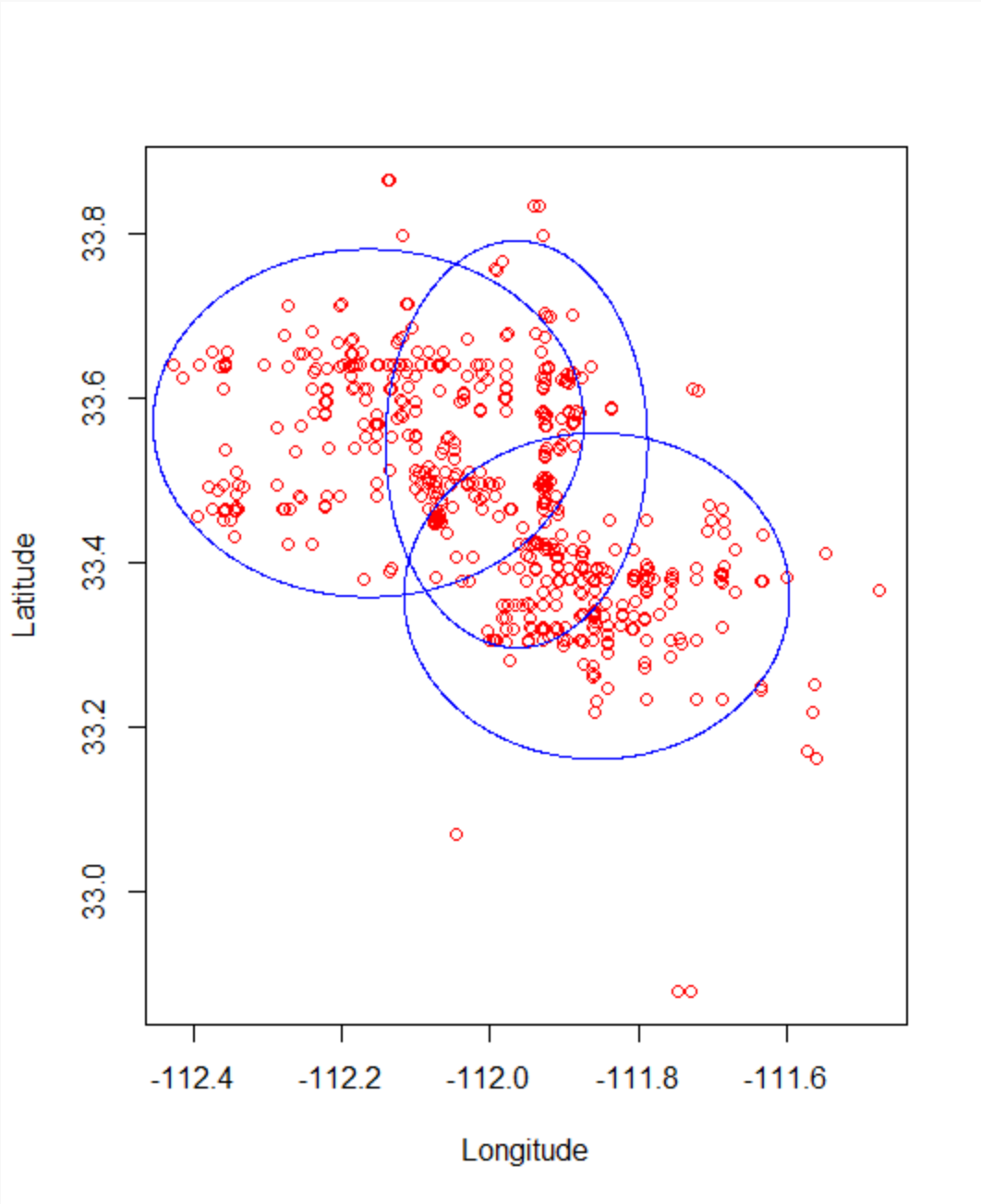
It appears that a good number of clusters are either 3 or 4 clusters, and just like our other clustering analysis, the loss value decreases as the number of clusters increases. This optimal number of clustering is roughly in line with our data from our other states.

K-means clustering with 3 clusters

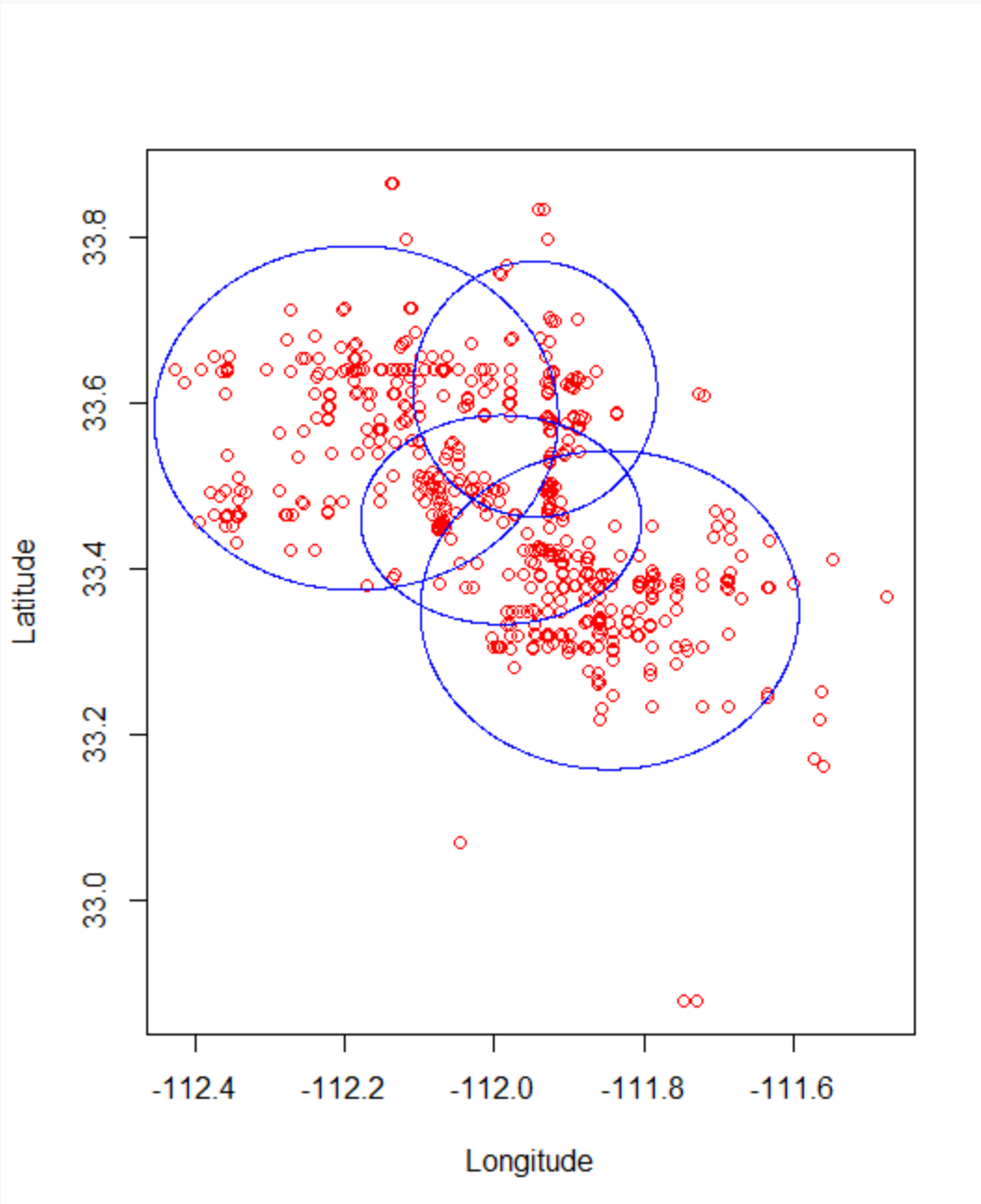


This clustering fits the data fairly well because it has distinct clusters.

Now, we will use a Gaussian mixture model to cluster the locations.
With 3 clusters.

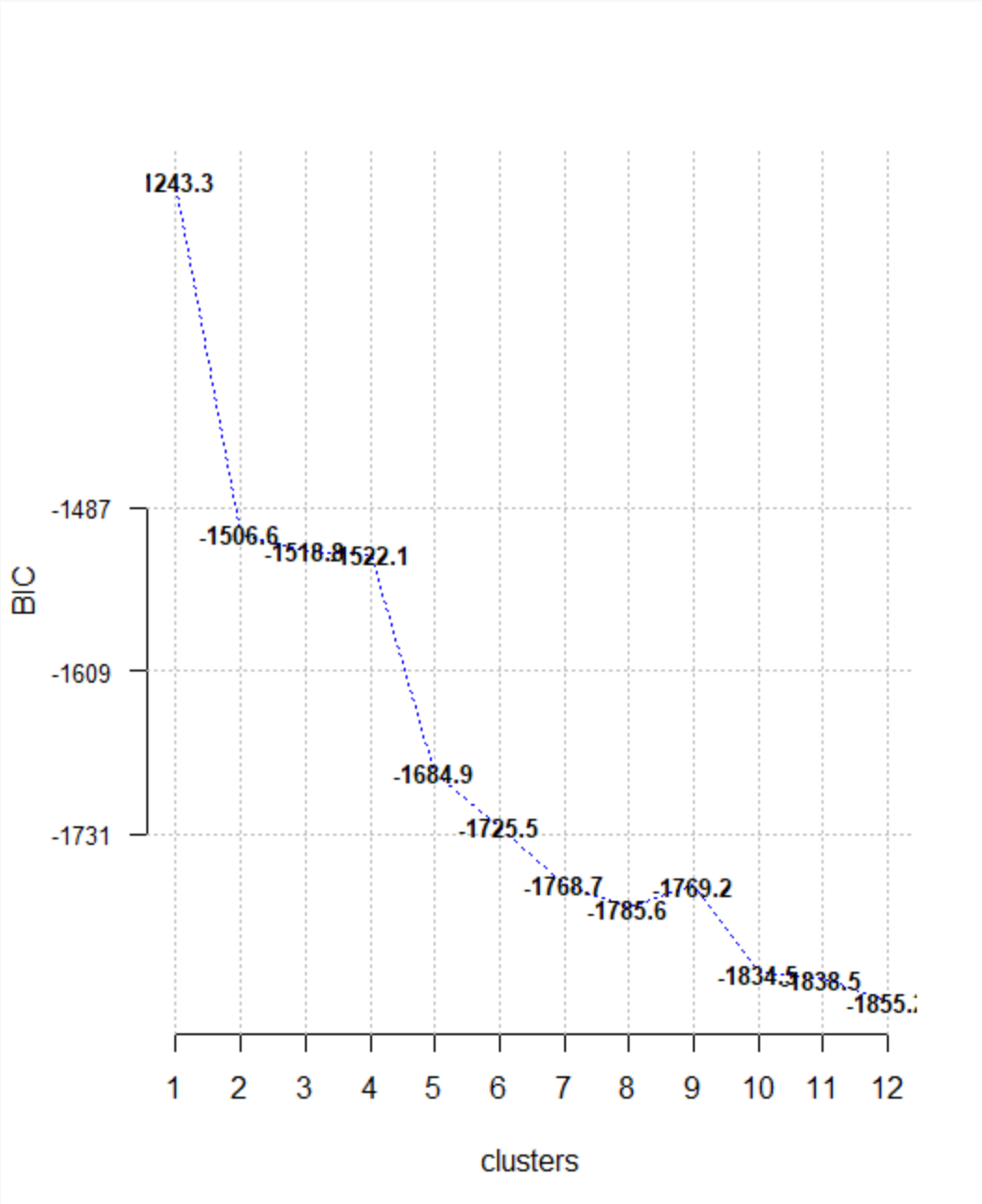


With 4 clusters.



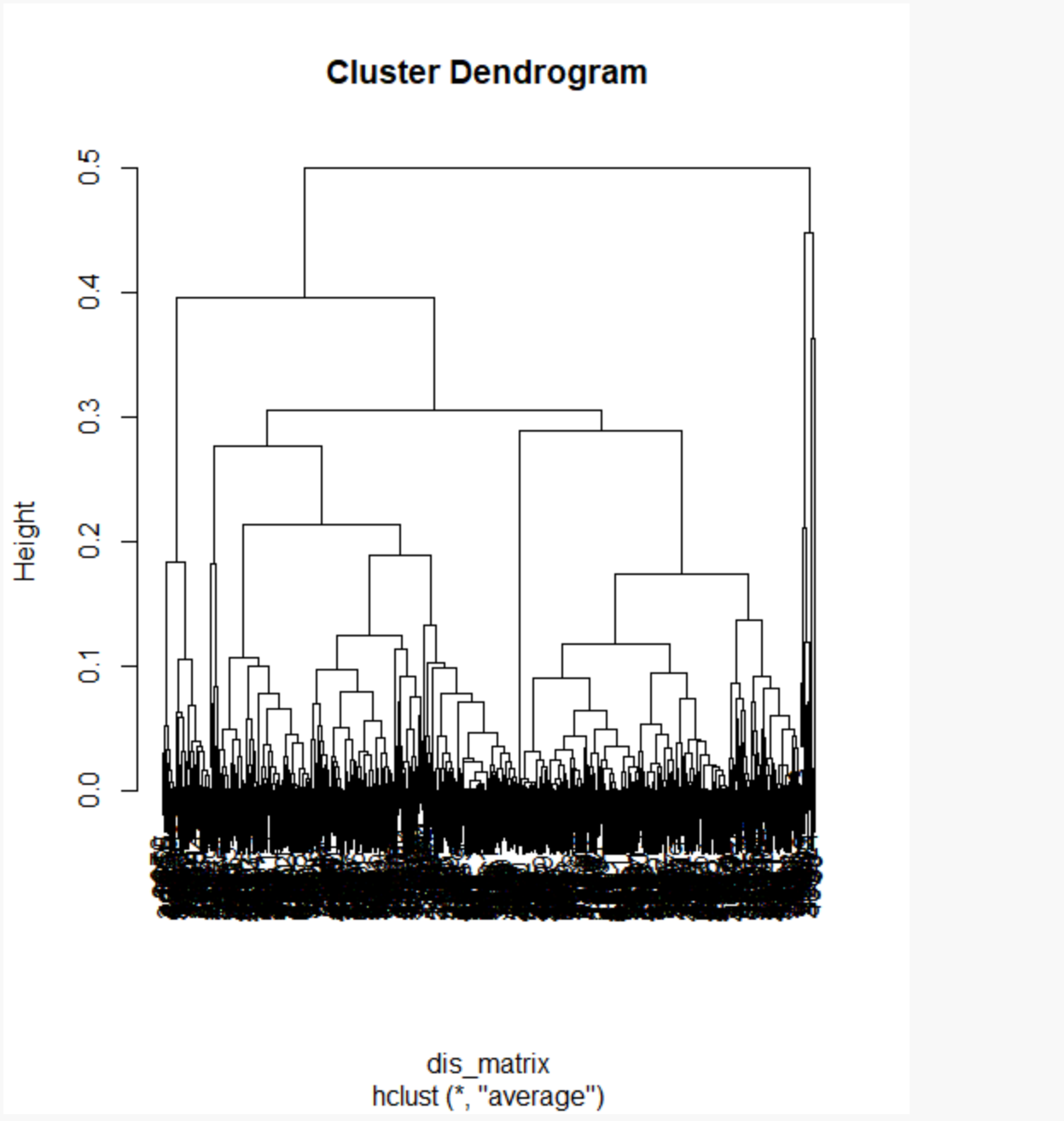
Both of these models appear to recognize a cluster of data in the center of our location plot, and they both do not include several of the outlier data points.

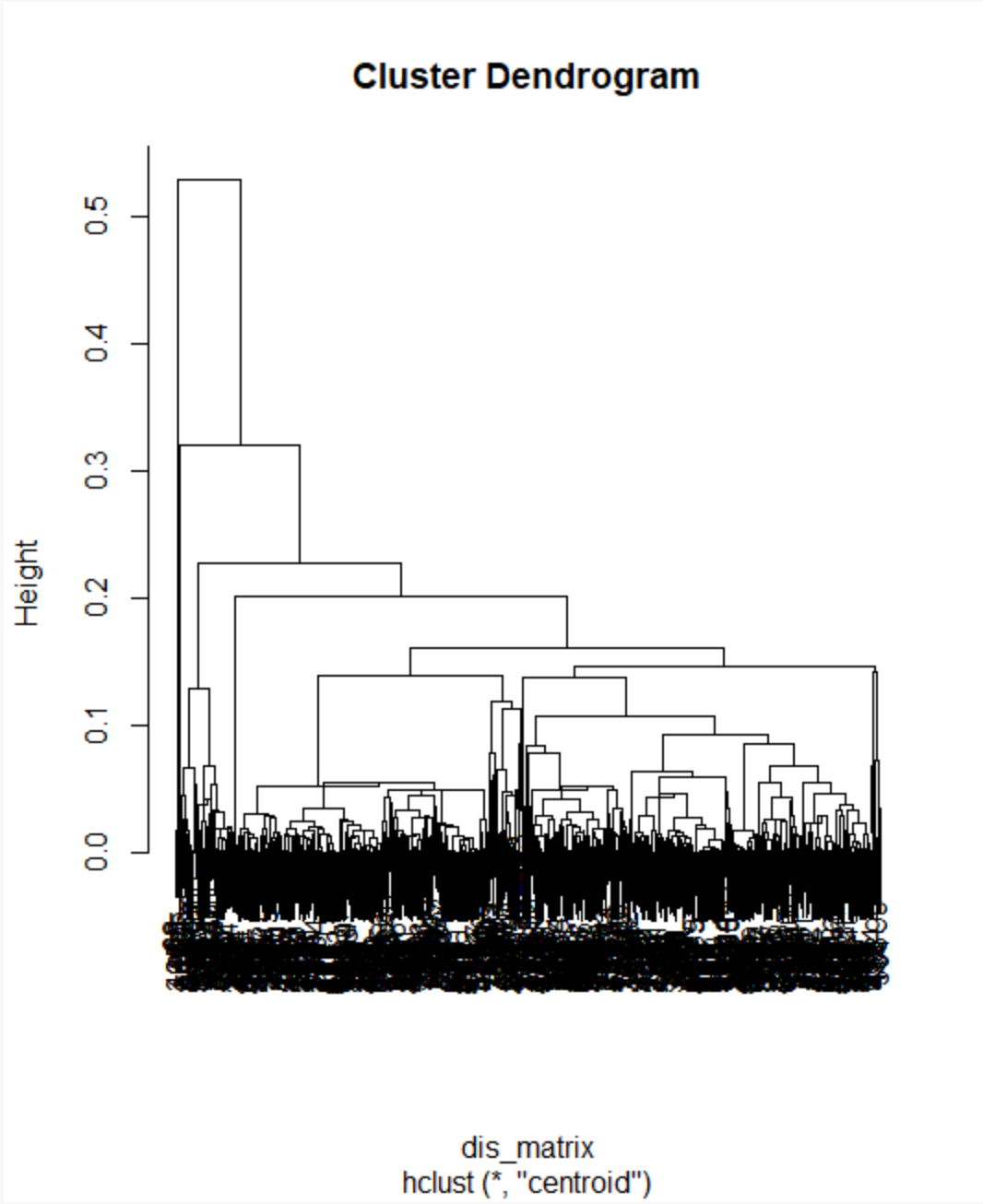
Let us now determine the optimal clusters in a Gaussian mixture model.



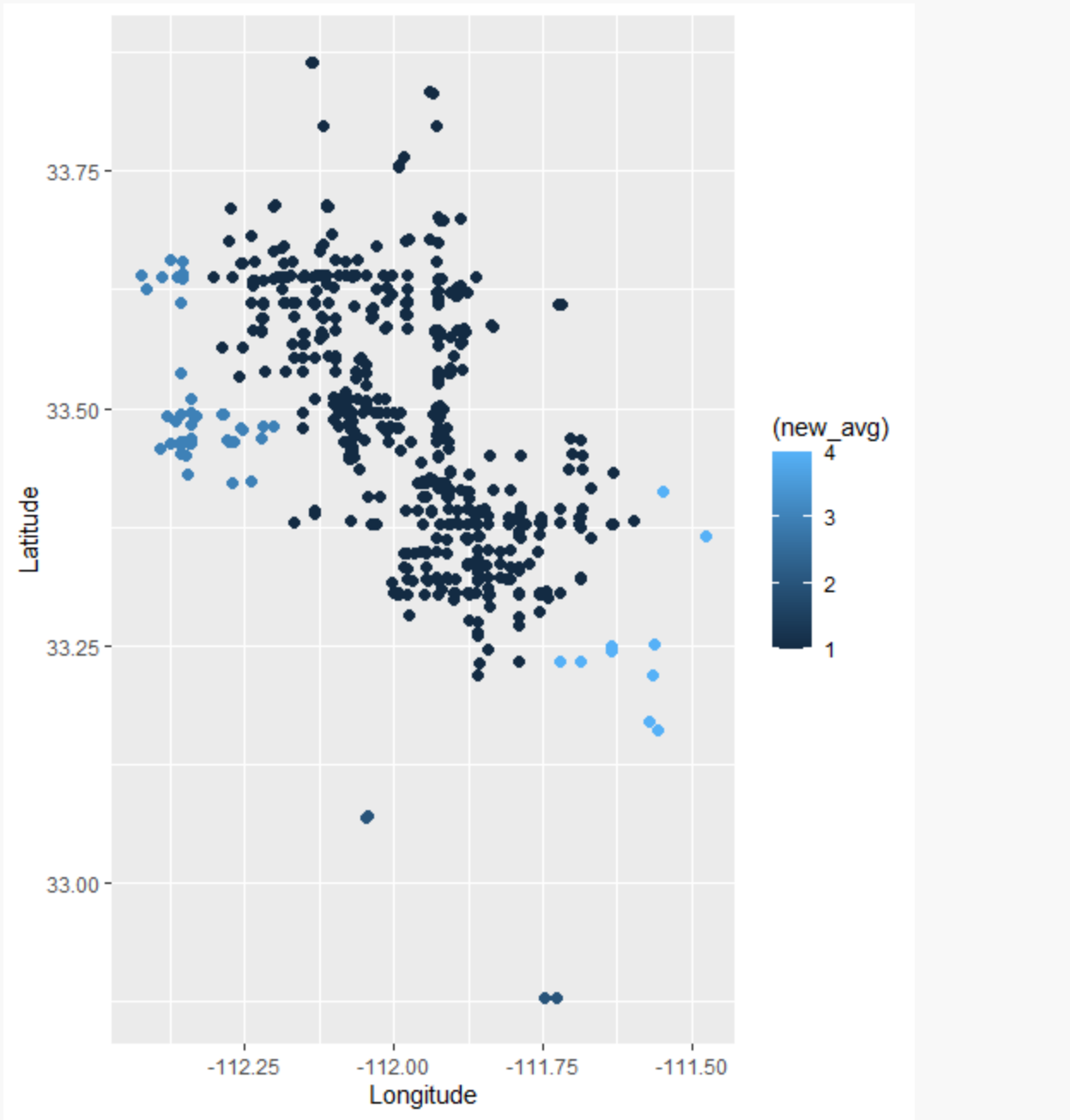
The BIC decreases as the number of clusters increases, with a large drop off from 1 to 2 clusters and from 4 to 5. Perhaps a good number of clusters would be anywhere between the values of 2 and 5.

Finally, let us cluster the data using hierarchical clustering with average and centroid linkage.

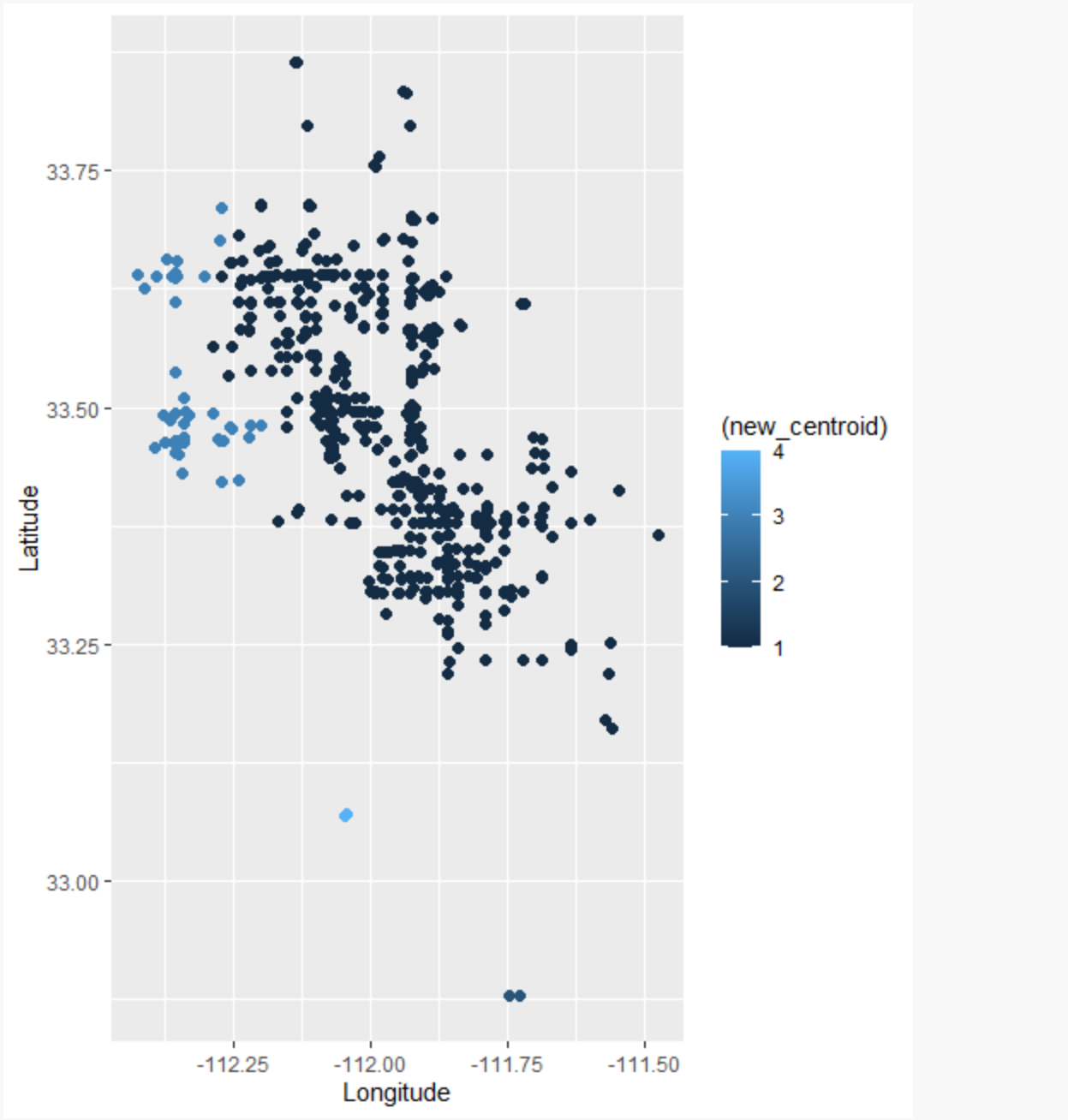

Cluster Dendrogram
Height
0.0
0.1
0.2
0.3
0.4
0.5
dis_matrix
hclust (*, "average")



These dendrograms are very messy probably due to there being more data points than our Wisconsin data set.
Let us now label the location points, first with average linkage.



Now with centroid linkage.



Both average and centroid linkage produce results that are not very helpful, with both methods clustering most of the locations into a single cluster. These results are not as easily interpretable or as useful as our K-means clustering method.

Overall, K-means clustering seems to cluster the data best, especially with three clusters. In that model, we see that there is a lower cluster and then the upper portion of the data are split into two clusters, which nicely demonstrates an East-West divide in the clusters of locations.

For all three areas of Asian restaurants, K-means clustering methods seemed to produce the most interpretable and well-fitting results. We see in the data that often there is a central cluster of

Asian restaurants in the data. This suggests that in the center of the metropolitan areas there exists a clustering of Asian restaurants. Though this may not be a surprising result, given that cities like Las Vegas, Nevada are economic hubs where a lot of commercial activity takes place, it does demonstrate that K-means can work with location data and, as a result, create the correct clustering results. This centralizing cluster is also shown in our Gaussian mixture models, where a cluster is almost always in the center of the graph, but it was often a little more difficult to interpret those clusters because it would frequently overlap each other and produce results that were difficult to differentiate. A potential future area of study for this kind of question could be comparing Asian restaurant clusterings to clusterings of non-Asian restaurants. This would allow us to see if there are any differences between those two clusers. Another potential line of future inquiry could be how the clustering of Asian restaurants changed over time in a county like Clark County, which this project could not explore as there were no time variables in our dataset.

Our second question we would like to answer is how we may create a recommender system based on the data. Note: This portion of our project was completed in Python.

Recommender System using K-means Score

Recommendation systems are widely used in different applications in industry. They have been used by companies like Amazon, Spotify, and Youtube to recommend either items, videos, or music based on the user's taste or the content of the product. In this recommender system, we will be recommending restaurants based on the user's locations and the content of the restaurants the user likes. We will use a combination of a location and content based recommendation system. Likewise, both recommender systems will be created using K-means clustering.
For the recommender system in this report, we will focus the location on Clark County, Nevada. This location can be easily adjusted by just changing the 'NV' to 'WI' if you want a recommender system in Dan County, Wisconsin or to 'AZ' if you want a recommender system in Maricopa County, Arizona.

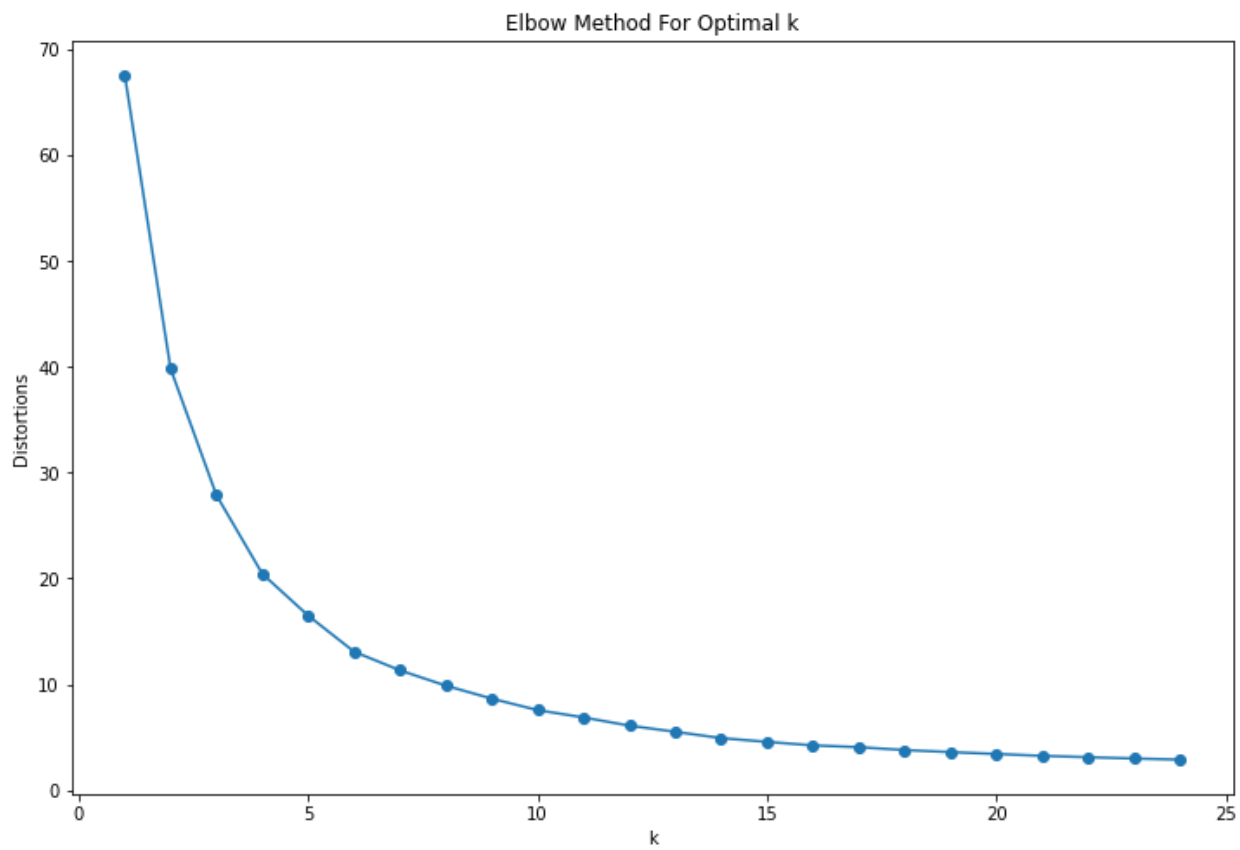
```
In [1]: import pandas as pd
        dfRst = pd.read_csv('restaurant_updated.csv')

C:\Users\kevin\anaconda3\lib\site-packages\IPython\core\interactiveshell.py:3146: DtypeWarning: Columns (3,6,7,8,9,16,19,22,28,
29,31,39,44,49,50,53,63,65,67,71,77,83,85,90,93,98,101,103) have mixed types.Specify dtype option on import or set low_memory=F
alse.
  has_raised = await self.run_ast_nodes(code_ast.body, cell_name,

In [2]: dfRstWI = dfRst[(dfRst['state']=='NV')] # change this to NV, WI, or AZ based on whether you're interested in
```

Before constructing the K-means model, an optimal number of clusters was needed for the location. Therefore, the elbow method will be used to collect the optimal number of K-clusters.

```
In [3]: from sklearn.cluster import KMeans
        # include location based recommendation
        # first we will use elbow method to get the appropriate number of clusters

        # Elbow method to determine the number of K in Kmeans Clustering
        coords = dfRstWI[['longitude','latitude']]

        distortions = []
        K = range(1,25)
        for k in K:
            kmeansModel = KMeans(n_clusters=k)
            kmeansModel = kmeansModel.fit(coords)
            distortions.append(kmeansModel.inertia_)
```

```
In [4]: # plot the results
        import matplotlib.pyplot as plt
        fig, ax = plt.subplots(figsize=(12, 8))
        plt.plot(K, distortions, marker='o')
        plt.xlabel('k')
        plt.ylabel('Distortions')
        plt.title('Elbow Method For Optimal k')
        plt.savefig('elbow.png')
        plt.show()
```

The graph of the results are shown below.



As can be seen from the graph, the most optimal number of k-clusters is around 5 because after a k-cluster of 5, the distortion, or sum square of error, does not decrease very much. This result

also reinforces the number of clusters we used to analyze restaurant locations in Nevada from question 1 of the final report.

Subsequently, the location based recommendation system is constructed using the code below. The K-means cluster will cluster 5 areas based on the longitude and latitude values provided in the Yelp data set. We then stored the cluster results on a new column in the dataframe.

```
In [5]: # so optimal number of cluster is 5 for the location system
        # we will now filter out and recommend restaurants in NV based on where you are located
        kmeans1 = KMeans(n_clusters=5, init='k-means++')
        kmeans1.fit(coords)
        y1 = kmeans1.labels_
        dfRstWI['cluster'] = kmeans1.predict(dfRstWI[['longitude','latitude']])
```

```
In [6]: # now there is a new column called cluster that will identify which part cluster you are based on location
        dfRstWI.head()
```

Out[6]:

es.Open.24.Hours	attributes.Ambience.romantic	attributes.Hair.Types.Specialized.In.perms	attributes.Music.jukebox	attributes.Ambience.upscale	population	cluster
NaN	NaN	NaN	NaN	NaN	2267000.0	1
NaN	False	NaN	NaN	False	2267000.0	3
NaN	NaN	NaN	NaN	NaN	2267000.0	3
NaN	NaN	NaN	NaN	NaN	2267000.0	3
NaN	False	NaN	NaN	False	2267000.0	3

Now it is time to make the content based recommendation system. For the content based recommendation system, we will be using solely categories as our descriptor. Initially, we wanted to use attributes as one of our features; however, we realized there were too many missing data for those attributes. If we had imputed those missing values randomly or dropped those missing values, the model's accuracy would be hampered. Therefore, we chose to just use categories as the description. Before doing any form of descriptive based clustering, we needed to preprocess and clean the 'categories' column and 'name' column. For the categories, the list of categories tended to slice the categories with a semicolon. Therefore, we needed to remove the semicolons in between those unique categorical string values. We also removed any empty spaces at the end or beginning of the string values for both the restaurant name and the categories to avoid any repeats of the restaurant names or the categories. Additionally, any rows with missing categories were filled with empty string values, and any duplicate restaurant names were dropped.

```
In [7]: # now make content based recommendation
        # Maricopa County, AZ, Dan County, WI, or Clark County, NV.
        dfRstWI['categories'] = dfRstWI['categories'].fillna('')
        dfRstWI = dfRstWI.drop_duplicates(subset='name')
```

```
<ipython-input-7-1fe8c3d57a2c>:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/us
rsus-a-copy
  dfRstWI['categories'] = dfRstWI['categories'].fillna('')
```

```
In [8]: dfRstWI = dfRstWI.reset_index(drop=True)
```

```
In [9]: # preprocessing step
        dfRstWI['categories'] = dfRstWI['categories'].replace({";": " "}, regex=True)
        dfRstWI.categories = dfRstWI.categories.str.strip()
        dfRstWI.name = dfRstWI.name.str.strip()
        comb_frame = dfRstWI['categories']
```

We then created vectors for each word by calculating the weight of each text. Therefore, the TFIDF vectorizer was used because it signifies the importance of a term in the document. Essentially, it is a statistical measure of each of the words where it converts the measurement of the importance and frequency of a word into a vector matrix.

```
In [10]: # Create word vectors from combined frames
         # Make sure to make necessary imports
         from sklearn import metrics
         from sklearn.feature_extraction.text import TfidfVectorizer
         vectorizer = TfidfVectorizer(stop_words='english')
         X = vectorizer.fit_transform(comb_frame)
```

After this, the matrix can be directly fed into a K-means clustering algorithm. The number of k-clusters was chosen arbitrarily as 31 based on computing speed. Generally, as the number of clusters goes up, the more variety of recommendations there will be assuming the number of clusters is not unreasonably high.

```
In [11]: # true_k, we have 31 clusters
         # the more clusters you have, the more different categories you will have
         # due to computing, we will just stick to 45 clusters for now.
         true_k = 31
         # Running model with 15 different centroid
         # initializations & maximum iterations are 500
         model = KMeans(n_clusters=true_k, init='k-means++', max_iter=500, n_init=15)
         model.fit(X)
```

Now the clusters are placed into groups and the names of the features are received from the vectorization matrix.

```
In [12]: # check if the clusters are appropriate based on the categories
         print("Top terms per cluster:")
         order_centroids = model.cluster_centers_.argsort()[:, ::-1]
         terms = vectorizer.get_feature_names()
         for i in range(true_k):
             print("Cluster %d:" % i),
             for ind in order_centroids[i, :15]:
                 print(' %s' % terms[ind]),
             print
```

Some of the results of the clusters can be seen below.

Top terms per cluster:

Cluster 0:

traditional
american
restaurants
nightlife
bars
burgers
diners
sandwiches
cafes
seafood
steakhouses
breweries
hawaiian
cheesesteaks
chinese

Cluster 1:

italian
restaurants
pizza
delis
sandwiches
chicken
wings
french
desserts
food
mediterranean
court
gluten

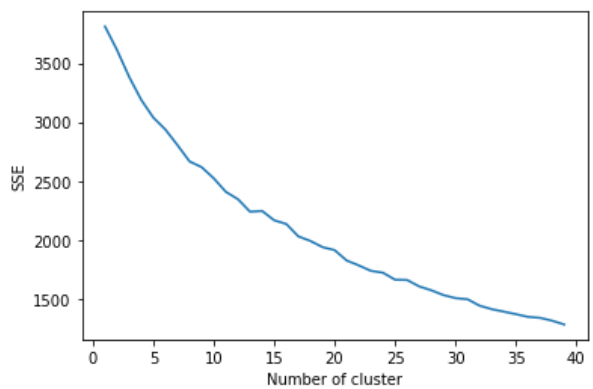
free
nightlife

We then looked at the sum square of errors for our cluster results, and we noticed that as the number of clusters increases, the error decreases. This is fine especially since these are text based clustering.

```
In [13]: # store sum of square errors
         sse = {}
         # Looping over multiple values of k from 1 to 40
         for k in range(1, 40):
             kmeans2 = KMeans(n_clusters=k, init='k-means++', max_iter=100).fit(X)
             comb_frame["clusters"] = kmeans2.labels_
             sse[k] = kmeans2.inertia_
```

The resulting graph can be seen below.

```
In [14]: # notice that square sum of errors decreases drastically as cluster increases.
         # this means that there is no clear optimal number of clusters based on the SSE
         # In general, the more clusters we have, the better because there will be more variety in recommendation
         import matplotlib.pyplot as plt
         # Plotting the curve with 'k'-value vs SSE
         plt.plot(list(sse.keys()), list(sse.values()))
         plt.xlabel("Number of cluster")
         plt.ylabel("SSE")
```



Then we created a function to predict any given cluster based on the description you put into it.

```
In [16]: def cluster_predict(str_input):
             Y = vectorizer.transform(list(str_input))
             prediction = model.predict(Y)
             return prediction
```

After that, we created a new column in the dataframe and filled those columns with all the results of the clusters based on the categorical text of each restaurant.

```
In [17]: # Create new column for storing predicted categories from our trained model.
         dfRstWI['ClusterPrediction'] = ""
```

```
In [18]: # Cluster category
         dfRstWI['ClusterPrediction']=dfRstWI.apply(lambda x: cluster_predict(dfRstWI['categories']), axis=0)
```

Finally, we combined all of that into a restaurant recommendation system. In this function, the location based recommendation system and the content based recommendation system were combined into one recommendation system. The algorithm will first grab the categories of a specific restaurant, and it will then return which content cluster those sets of categories belong to. Then it will predict which location cluster the user belongs to based on where the user is located at. Finally, based on which location cluster the user is at, it will randomly output 5 restaurants from the content cluster group.

In [19]:
```
# now combine longitude and latitude coordinates with restaurant recommendation
import numpy as np
def rst_rec_sys(str_input, longitude, latitude):

    # match on the basis restaurant name and form whole 'categories' entry out of it.
    temp_df = dfRstWI.loc[dfRstWI['name'] == str_input]

    str_input = list(temp_df['categories'])

    # Predict category of input string category
    prediction_inp = cluster_predict(str_input)
    prediction_inp = int(prediction_inp)
    # Predict the cluster for longitude and latitude provided
    cluster = kmeans1.predict(np.array([longitude,latitude]).reshape(1,-1))[0]
    dfRstWI2 = dfRstWI[dfRstWI['cluster']==cluster]

    # Based on the above prediction pick 10 restaurants based on the location and the restaurant content
    # Recommendation Logic is kept super-simple for current implementation.
    # it will also randomly pick 5 samples
    temp_df = dfRstWI2.loc[dfRstWI2['ClusterPrediction'] == prediction_inp]
    temp_df = temp_df.sample(5)
    return temp_df[temp_df['cluster']==cluster].iloc[0:5][['name', 'latitude','longitude']]
```

Below are the results of the recommendation system. I also included a few Asian restaurants to recommend in the Clark County, Nevada area.

In [20]:
```
rst_rec_sys('Boba City Cafe',-115.1891691,  36.1017316)
```

Out[20]:

	name	latitude	longitude
3453	Jullie's Caffe Soleil	36.087978	-115.190216
1179	Teahouse	36.118169	-115.172435
3203	3pm Cafe	36.104474	-115.136561
2525	The Beat Coffeehouse	36.169116	-115.139599
3248	Le Cafe Coffee Express Bar	36.112492	-115.170998

In [26]:
```
rst_rec_sys('Burger King', -115.2,  36.2)
```

Out[26]:

	name	latitude	longitude
2354	Popeyes	36.239388	-115.251079
104	Kfc	36.239789	-115.159990
2374	Robertos Taco Shop	36.257077	-115.225171
383	Roberto's Taco Shop #22	36.246667	-115.209286
3275	Ascar Cafe	36.218227	-115.208687

In [22]:
```
rst_rec_sys('Great Wall Chinese Restaurant', -115.2398, 38)
```

Out[22]:

	name	latitude	longitude
829	Grand China III	36.300025	-115.279864
2967	Bangkok 9	36.217763	-115.242392
2368	Dragon Bowl	36.246758	-115.209185
1831	Dragon Wok	36.221024	-115.328673
4026	Super No1 Chinese Restaurant	36.263013	-115.180561

```
In [23]: rst_rec_sys('China Wok', -115.2398, 38)
```

Out[23]:

	name	latitude	longitude
1133	Golden Phoenix Chinese Cuisine	36.274785	-115.118304
2368	Dragon Bowl	36.246758	-115.209185
105	Chai Tip's Thai and Chinese	36.230863	-115.162130
1831	Dragon Wok	36.221024	-115.328673
2432	China Wok	36.237905	-115.181957

```
In [24]: rst_rec_sys('Pho Sing Sing',-115.2891691,  36.0017316)
```

Out[24]:

	name	latitude	longitude
4082	WTPho	36.020663	-115.267231
3138	Rainbow Pho Express	36.036156	-115.243138
3296	DaKao Quan	36.126508	-115.218285
3729	Pho Sing Sing	36.126577	-115.225926
1213	Bamboo Bistro	36.051627	-115.243870

```
In [25]: rst_rec_sys('Thai Food To Go',-115.1891691,  36.2017316)
```

Out[25]:

	name	latitude	longitude
313	Pin Kaow Thai Restaurant	36.195322	-115.241468
3181	Mix Zone Cafe 2	36.196861	-115.262203
1037	Thai Basil	36.262339	-115.178170
3502	Sun's Thai Food & Jerky	36.238685	-115.209788
4084	Siam Thai Kitchen	36.179324	-115.241967

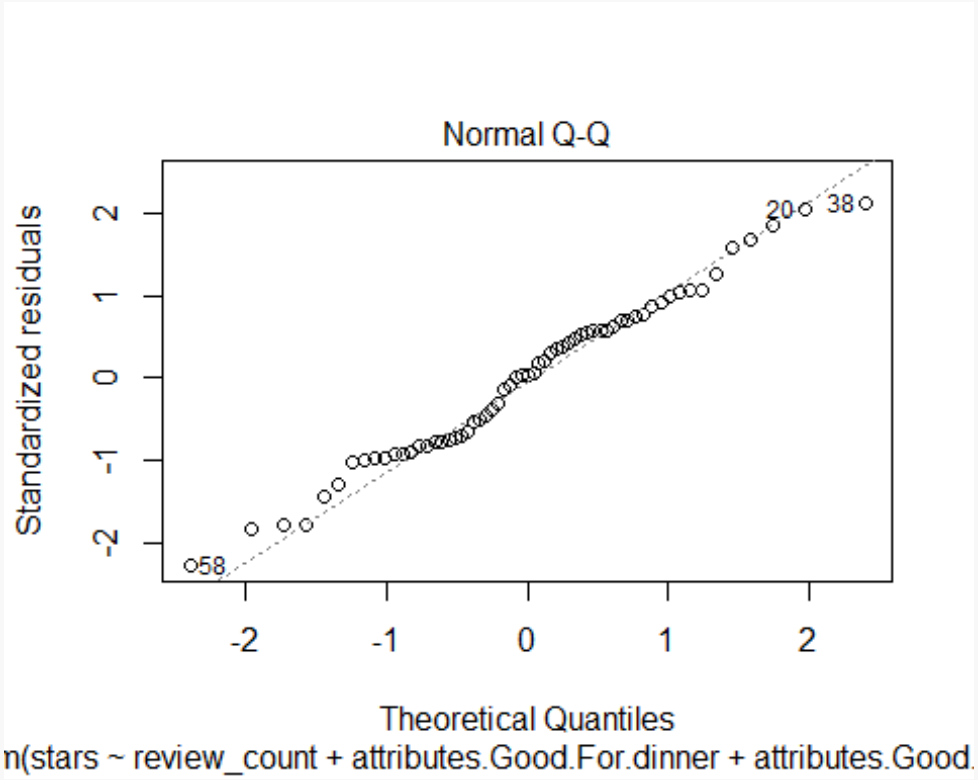
Based on our findings, we were able to create a location and content-based recommendation system for restaurants using K-means clustering. By clustering different descriptions and locations together, a location and content-based recommendation system can be created. Ideally, this system would grab a user's latitude and longitude location and give a restaurant recommendation based on previous restaurant that the user liked in that area. Some potential improvements to the recommendation system is being able to filter out repeats that have the same restaurant name but with a different way of titling the restaurant name. For example, Starbucks is sometimes titled as Starbucks Co. and other times it is titled as Starbucks Coffee. Another potential improvement would be to use an alternative to K-means clustering for the text-based recommendation because, for example, it may be better to use cosine similarity rather than K-means clustering to determine whether a restaurant should be grouped with each other. Additionally, another potential improvement would be to implement a top scored based recommendation instead of the randomly generated recommendations made by the K-means clustering algorithm.

Our third question is if we may use regression analysis and tree methods to find the best predictors of stars of Asian restaurants.

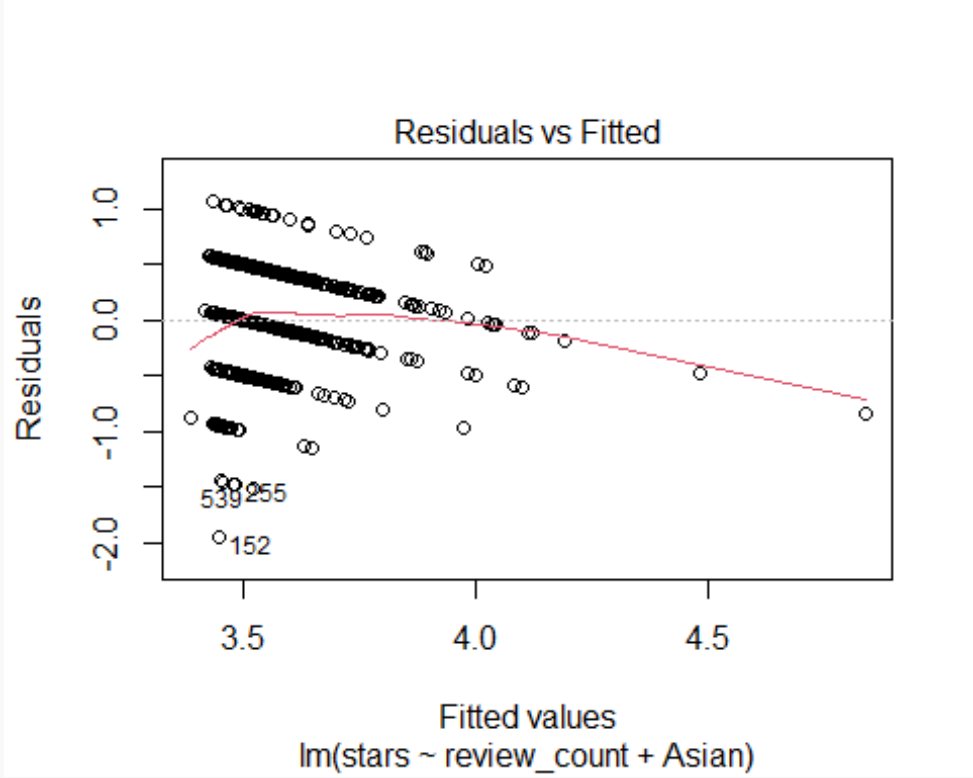
Regression and Decision Trees

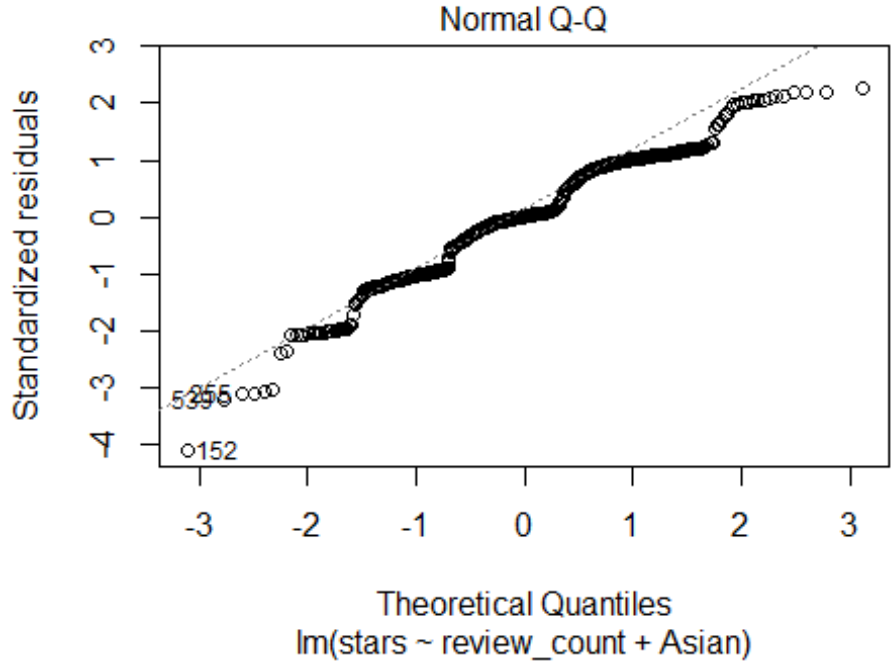
We want to use methods such as linear regression and decision trees to determine which attributes contribute the most to a restaurant's rating (on the Yelp star rating scale of 1-5, in increments of 0.5). The restaurant dataset was combined with the US Census population dataset (2015), and the tract ID in the census dataset was paired with the city of each restaurant. After merging the columns for total population and the local ethnic populations, we removed the attributes with a low number of entries (columns with mostly N/As), then split the data into three separate datasets for each state.

Residuals vs. fitted and Q-Q plots for Wisconsin:

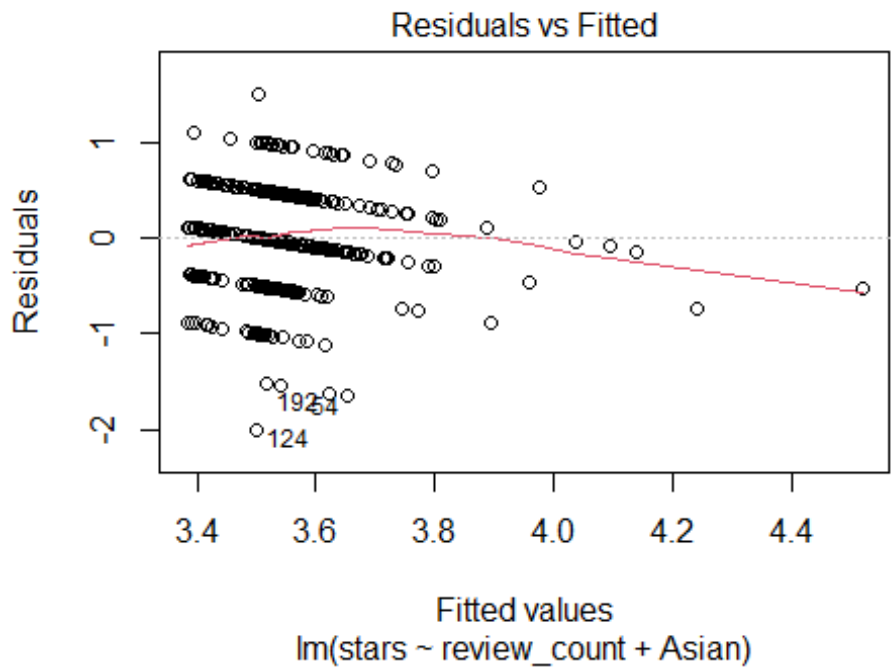


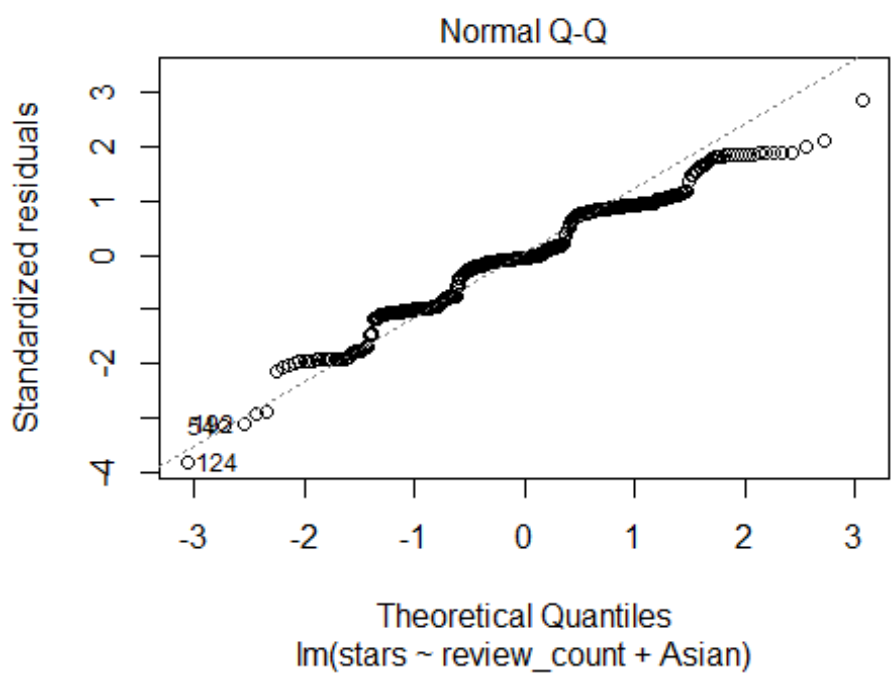
Residuals vs. fitted and Q-Q plots for Arizona:



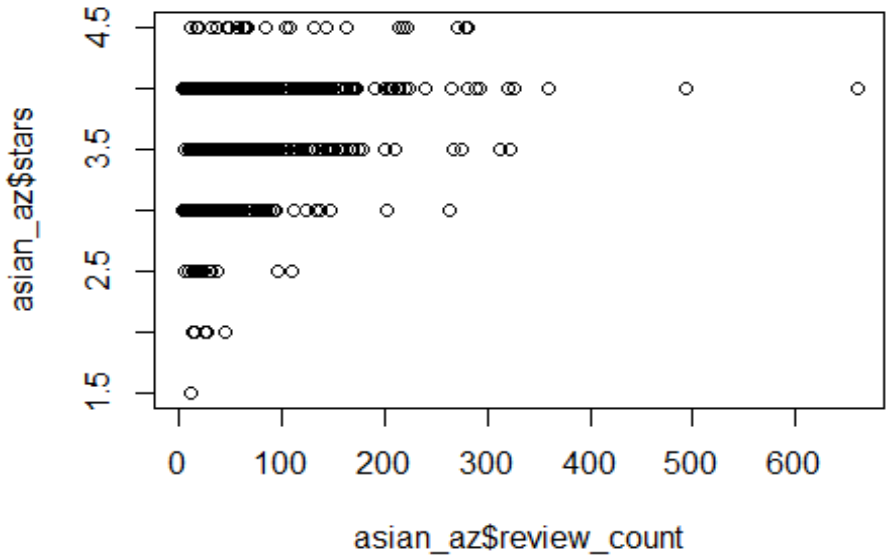


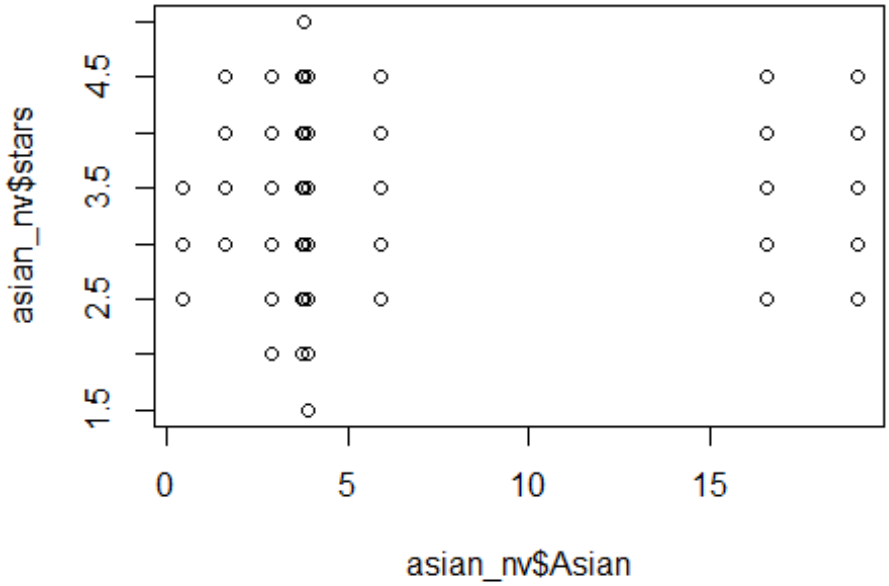
Residuals vs. fitted and Q-Q plots for Nevada:





Preliminary graphs to look for possible correlation:



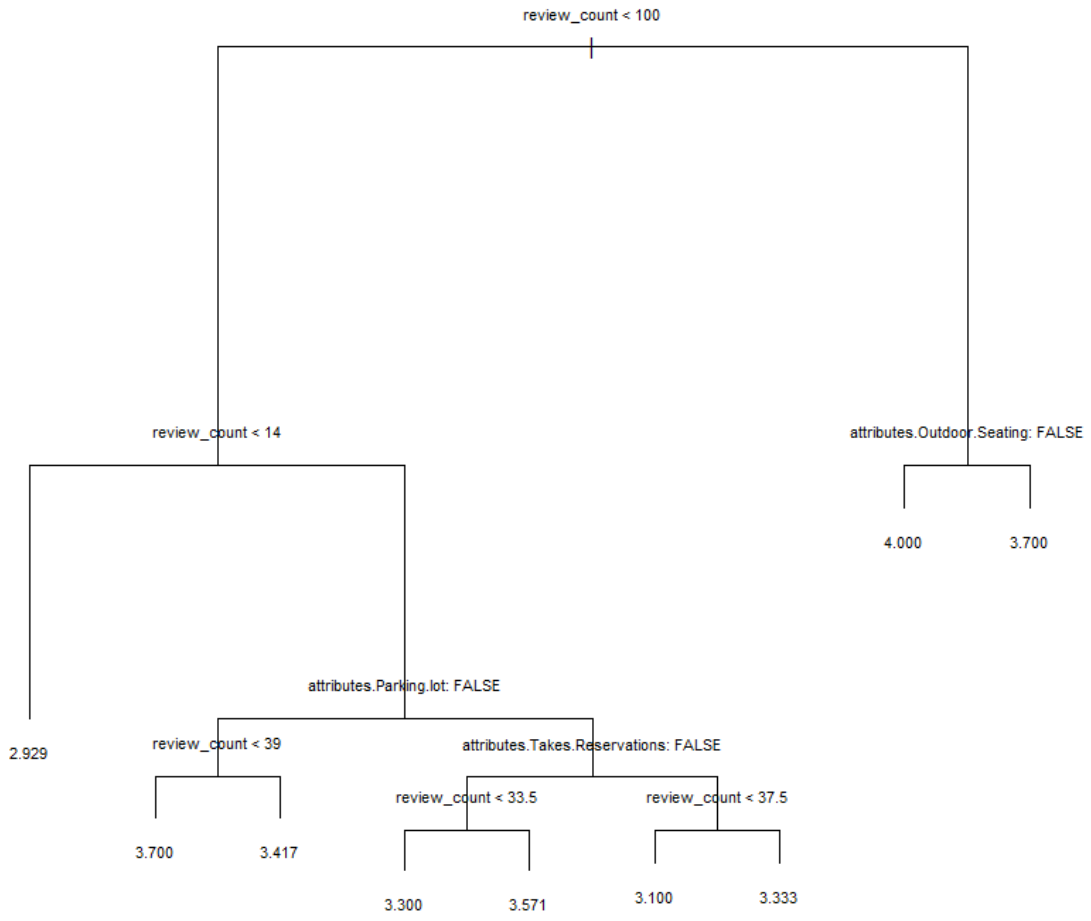


Looking at the regression models for each state, the only predictor with a statistically significant p-value is the number of review counts per restaurant. We plotted some factors such as Asian population and review count against the star ratings and could not notice a visible trend in the data. Looking at the diagnostic plots, the residual vs. fitted values plot gave a very non-uniform, non-random distribution of residual points. In Nevada and Arizona, there seems to be very few highly rated restaurants, so the residuals are concentrated on the lower end of the graph. The Normal Q-Q plots for Arizona and Nevada show a wave pattern of error terms, which might suggest that the error terms are not normally distributed. We can conclude from this that we should not use linear regression.

Using the decision tree, we ask the question of what attributes can improve a restaurant's ratings greatly?

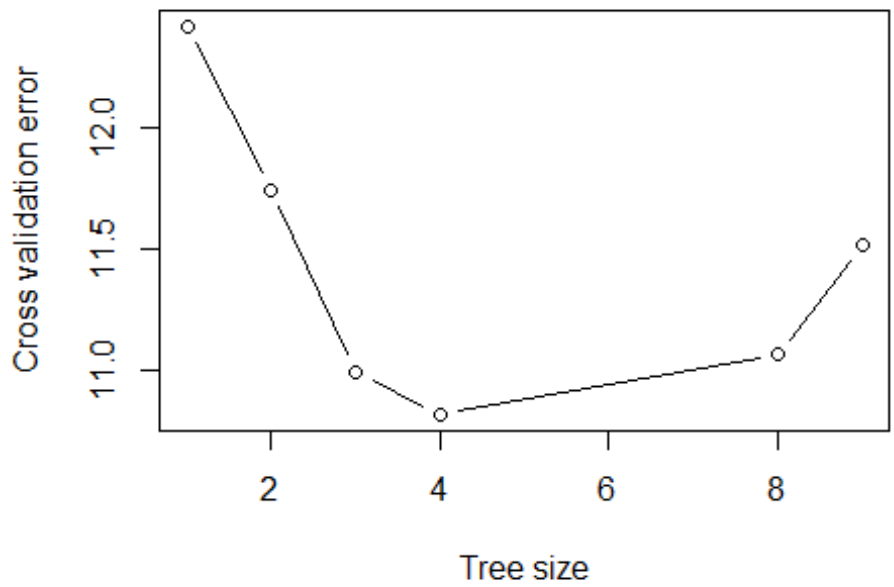
Our next option was to use decision trees to predict which attributes can affect a restaurant's Yelp rating. Similarly, a regression tree was applied to each set of data for the three US states. Each dataset was split into 90% training data and 10% testing data. Decision trees were plotted with and without cross-validation to determine the best number of nodes, and bagging and random forest methods were used to see how we could improve test error and see which attributes contribute the most to restaurant ratings. For these trees methods, the attributes 'city' and 'categories' were not used, since we believe city should not be included as a contributing factor of restaurant rating (established businesses cannot help where they are located) and for Arizona and Nevada the 'categories' column had too many combinations of restaurant labels for a tree to split on. The rest of the attributes (28 out of 122 columns in the original data set) were used to create the models.

Asian restaurants in Wisconsin

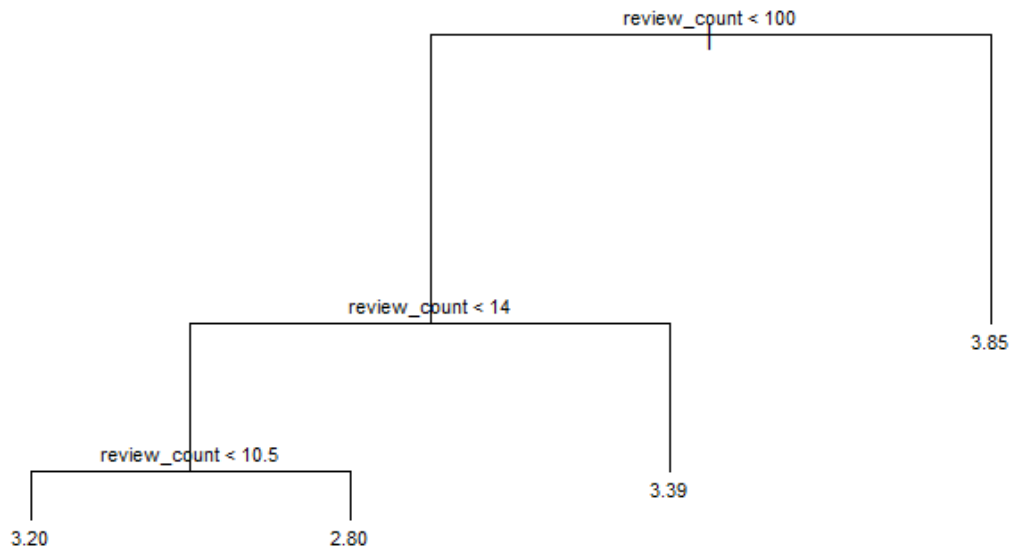


Before doing cross-validation, we fit a regression tree to the training test for Wisconsin Asian restaurants. Looking at the tree above, we first look at the restaurant's review counts. If the number of reviews is lower than 100, we need to consider if it is lower than 14 reviews as well. The tree shows that if there are fewer than 14 reviews, then based on this data the restaurant will most likely have an average rating of 2.929 stars. If the restaurant has more than 14 but less than 100 reviews, has a parking lot, takes reservations, and also has a review count of greater than 37.5, then the average rating of about 3.333 stars. On the other hand, if there are more than 100 reviews and there is no outdoor seating, the average rating is predicted to be 4 stars.

The MSE calculated below is about 0.329.



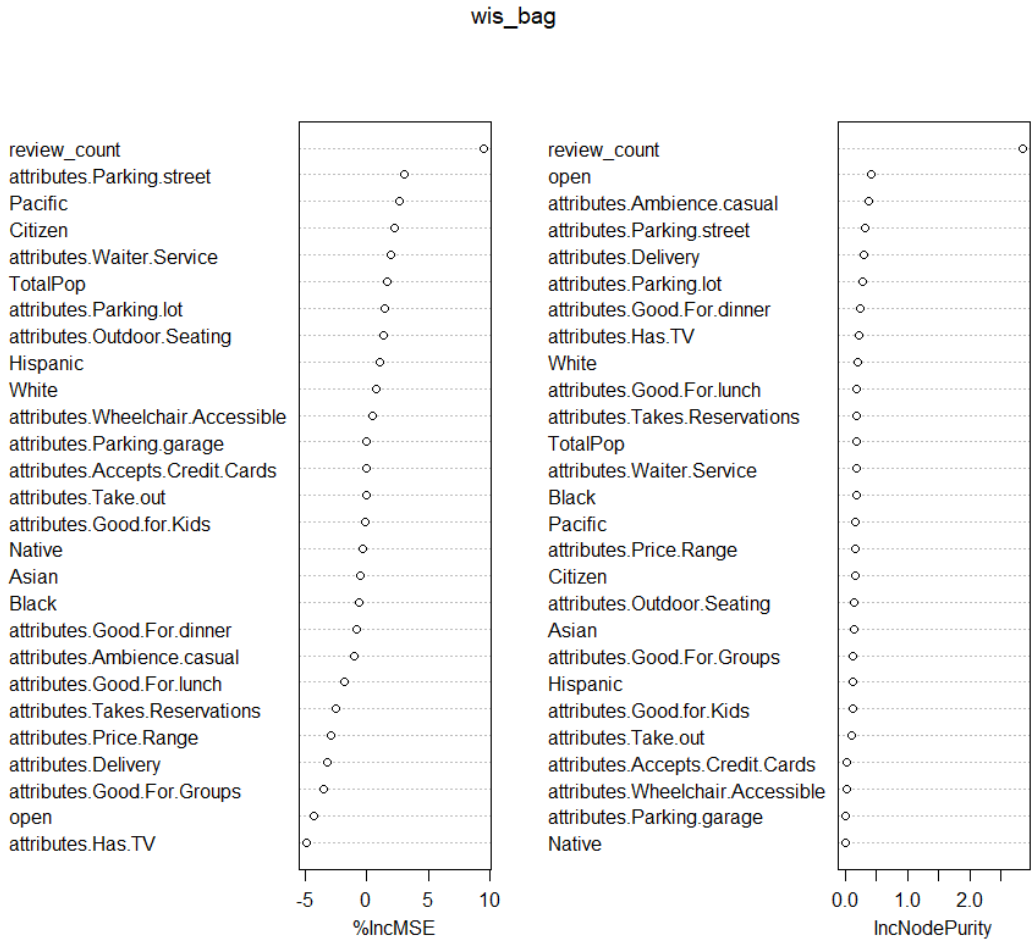

```
# CV errors
## [1] 11.51030 11.06108 10.81446 10.98763 11.74154 12.41404
## test error: 0.07045333
```



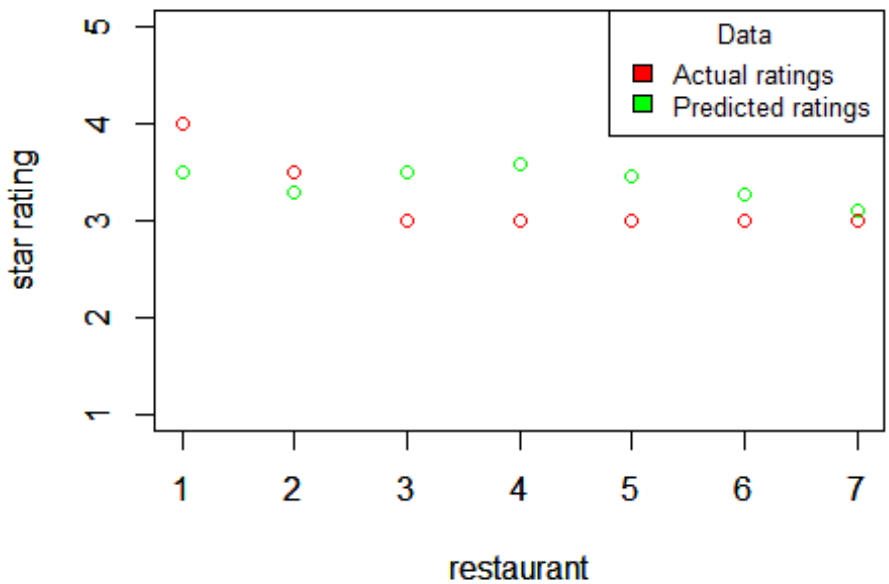
After applying 10-fold cross-validation, our plot shows that the lowest cross-validation error rate is at a tree size of four, so the pruned tree is reduced down to four nodes. Our test error from this pruned tree is about 0.07, which is a significant improvement from 0.33 of the previous tree. We can see that this pruned tree only considers the review count of the restaurant when predicting it's rating. If there are more than 100 reviews, the restaurant will have an average predicted rating of 3.85 stars. If there are between 14 and 100 reviews, the restaurant will have an average predicted rating of about 3.39 stars. If there are between 10.5 and 14 reviews, the ratings will be on average 2.8 stars. Finally if there are less than 10.5 reviews for a restaurant, the average predicted star rating is about 3.2.

```
#bagging
## test error: [1] 0.1659563

## important values

##                                %IncMSE IncNodePurity
## categories                   10.1190740  3.4229724563
## attributes.Outdoor.Seating     -0.4576187   0.136844513
## attributes.Parking.lot          1.6772024   0.243693980
## attributes.Waiter.Service       1.2716504   0.133441998
## attributes.Parking.street       2.7137993   0.321698445
## attributes.Price.Range         -0.9517066   0.174918753
## attributes.Parking.garage       0.0000000   0.004688335
## review_count                   11.6096761   2.401172359
```

```
## attributes.Accepts.Credit.Cards   0.0000000   0.037770433
## attributes.Good.For.lunch        -2.3142327   0.182649713
## attributes.Take.out               0.0000000   0.086220299
## attributes.Takes.Reservations    -2.0237016   0.191723538
## attributes.Delivery              -2.0427637   0.292317184
## open                             -0.6153985   0.381546993
## TotalPop                          2.0520650   0.161419256
## Hispanic                          2.5158890   0.121229055
## White                             2.6590964   0.201653805
## Black                             0.5285804   0.095455771
## Native                            1.2813443   0.002595375
## Asian                            -0.7455191   0.104822367
## Pacific                           1.1883974   0.140355602
## Citizen                           1.4926396   0.145702960
## attributes.Wheelchair.Accessible -2.2277234   0.020590210
## attributes.Good.For.dinner       -0.8891439   0.212501879
## attributes.Good.for.Kids         -0.4703248   0.091612911
## attributes.Has.TV                -1.7968254   0.202304305
## attributes.Ambience.casual        2.1425427   0.387887380
## attributes.Good.For.Groups       -2.7629477   0.118402544

#plot importance of predictors
```



Using the importance and varImpPlot functions, we can see that the categories and review_count variables are the most important predictors the random forest used to predict star rating. They have %IncMSE of 10.1190740 and 11.6096761 respectively. 'Categories' of restaurant type was not seen as a variable used in the previous decision tree and pruned tree, so we are not sure if there is some miscalculation error and do not have a clear explanation for why it does not appear.



For the test set of Wisconsin Asian restaurants, the actual ratings and predicted ratings were plotted together for comparison. We can see that the second and seventh restaurant have very high accuracy of prediction, and overall the tree's predictions are not very far from the actual ratings. However, we have to consider that there might be some amount of overfitting since there is very little data to work with particularly for the state of Wisconsin. This is likely due to the smaller population compared to the other two US states, so there are not many businesses or restaurants in the area.

```
# errors with varying tree size
my_forest(2)

## [1] 0.1707833

my_forest(3)

## [1] 0.1661654

my_forest(4)

## [1] 0.1596649

my_forest(5)

## [1] 0.1593786

my_forest(6)
```

```
## [1] 0.1632074
```

```r
my_forest(7)
```

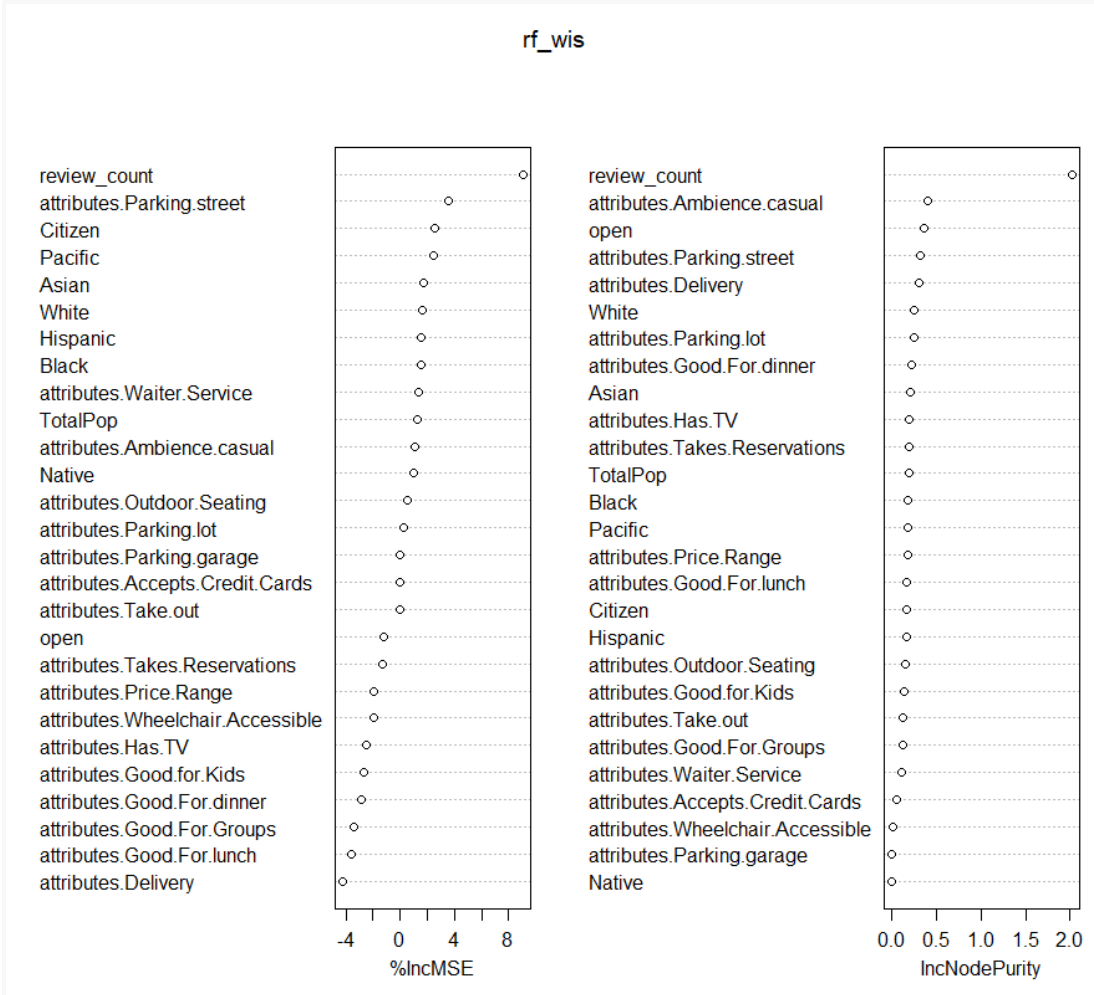
```
## [1] 0.1684663
```

```r
my_forest(8)
```

```
## [1] 0.1708814
```

```r
# importance and variance plots
```

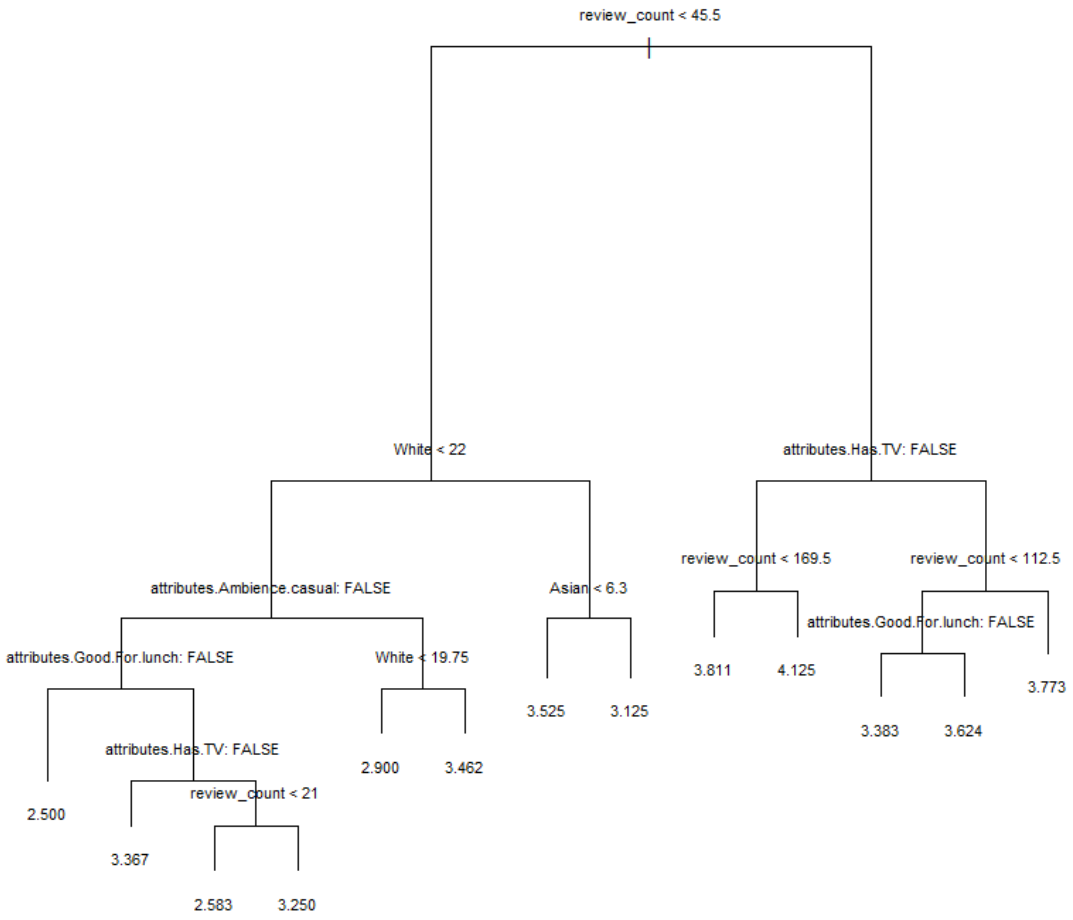
```
%IncMSE                                    IncNodePurity
## attributes.Outdoor.Seating        2.0674096   0.120247838
## attributes.Parking.lot            0.3540349   0.176051221
## attributes.Waiter.Service        -1.3342812   0.082039401
## attributes.Parking.street         2.7916334   0.209779602
## attributes.Price.Range           -0.0617948   0.111372538
## attributes.Parking.garage         0.0000000   0.002690194
## review_count                      6.2997493   1.068511365
## attributes.Accepts.Credit.Cards   0.0000000   0.051043748
## attributes.Good.For.lunch         0.1176912   0.189215193
## attributes.Take.out               0.0000000   0.060994471
## attributes.Takes.Reservations    -0.6643849   0.129202544
## attributes.Delivery              -0.4086927   0.196804554
## open                             -1.0184915   0.215529215
## TotalPop                          2.3805784   0.210549418
## Hispanic                          2.5126617   0.192769161
## White                             3.7789017   0.135542650
## Black                             3.6858879   0.158405833
## Native                            0.3939104   0.003780779
## Asian                             1.7901525   0.205888669
## Pacific                           3.1128523   0.151677397
## Citizen                           3.8656444   0.202317832
## attributes.Wheelchair.Accessible -0.1082545   0.022257298
## attributes.Good.For.dinner        0.2973724   0.131314541
## attributes.Good.for.Kids         -2.1193562   0.093773605
## attributes.Has.TV                -3.0701362   0.131159553
## attributes.Ambience.casual        2.3256041   0.292235979
## attributes.Good.For.Groups       -1.1411719   0.125988688
```



We can see from this random forest error calculation function that the MSE is the lowest at m = 5 nodes. From the previous importance function we can see that there are two important variables in the matrix, so the best number of nodes should bc 2 * sqrt(27) = 10, but because we are aware that 'categories' might not actually be an important predictor of star rating, is it reasonable that the MSE is lowest at sqrt(27) = 5 nodes. When we construct the random forest with mtry = 5, we can see that the review count has the highest %IncMSE of about 6.2997493.

To conclude, we can see from performing tree methods on the Wisconsin Asian restaurant data that review count seems to be the most effective predictor of the restaurant star ratings. When it comes to evaluating with the test error, there may be a possibility of overfitting occurring since we do not have a very large amount of data to work with in Wisconsin.

Asian restaurants in Arizona

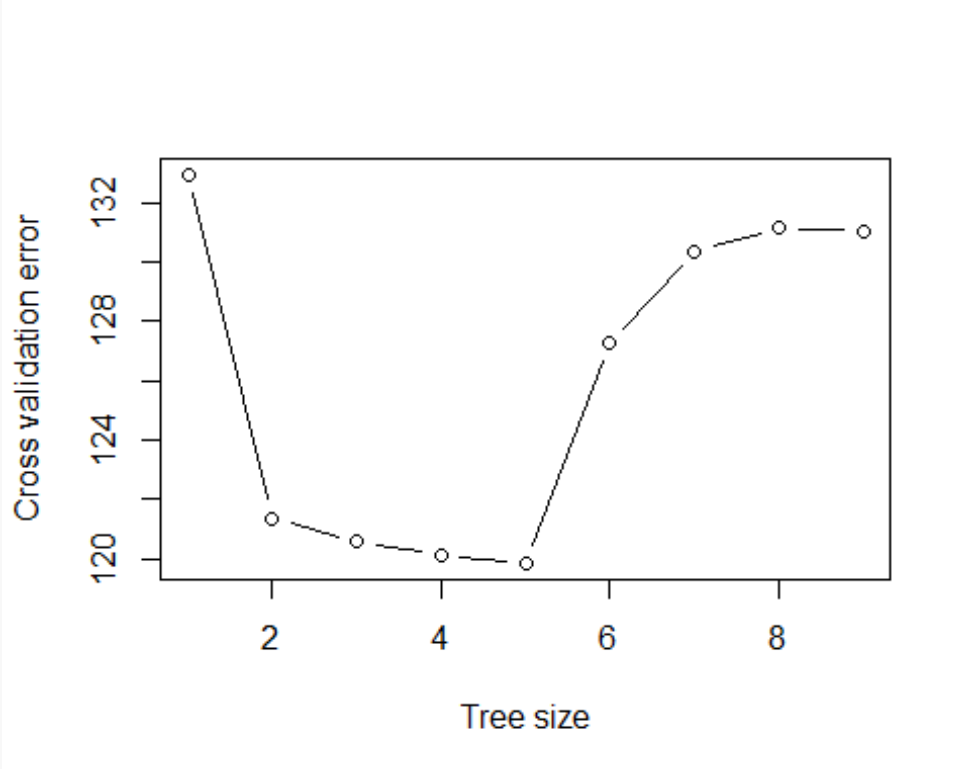



```
## test error: [1] 0.2568173
```

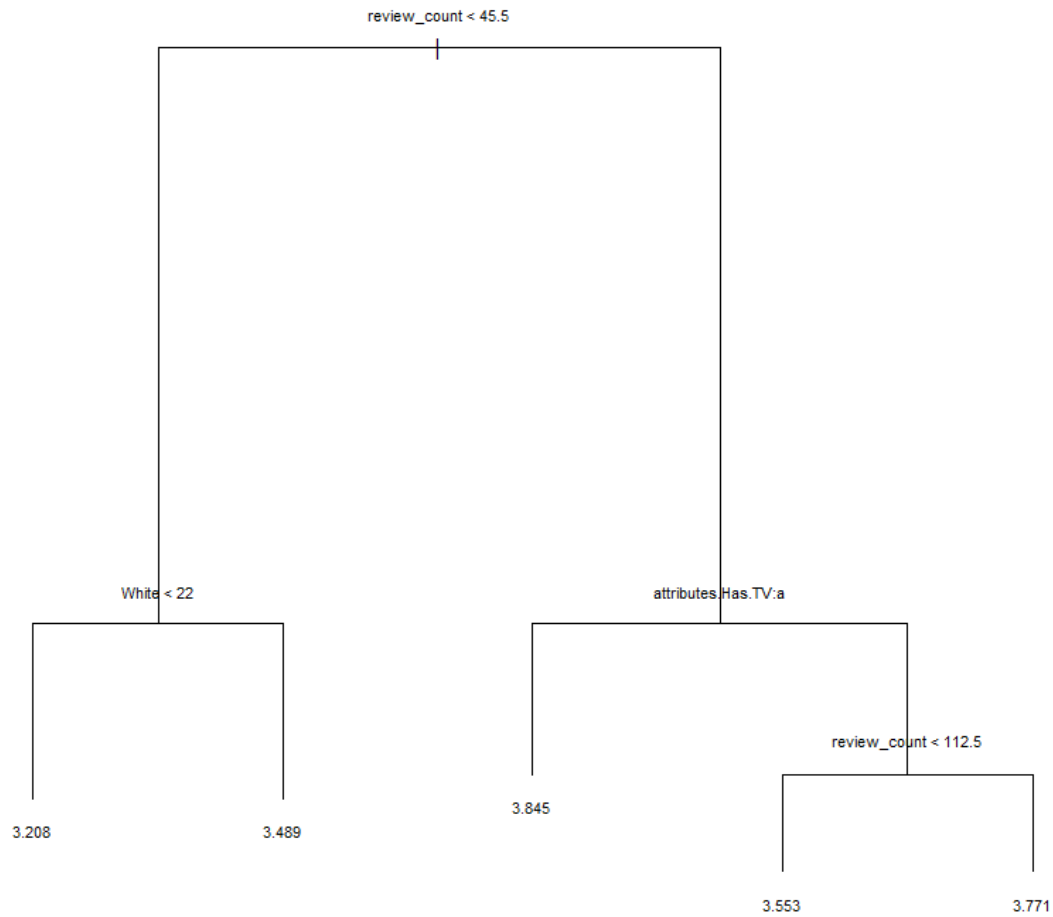
Our single regression tree for the training dataset of Arizona Asian restaurants yielded a MSE of about 0.26. Looking at the nodes, there seem to be multiple variables the tree splits on. Starting from the top if the review count is greater than 45.5 and there is no TV, the review counts are also lower than 69.5 and it is not good for lunch, the predicted average rating is about 3.811. However if the restaurant with no TV containing between 45.5 and 169.5 reviews is good for lunch, the predicted average rating goes up to 4.125. On the other hand if the restaurant has less than 45.5 reviews, a local white population greater than 22,000, and a local Asian population (based on tract ID) less than 6,300, the predicted average rating is about 3.525 stars. Next, we

will see if the complexity of the tree can be reduced by pruning. Pruning may help reduce the test error and make the tree more easily interpretable.

```
# CV to find best number nodes
```



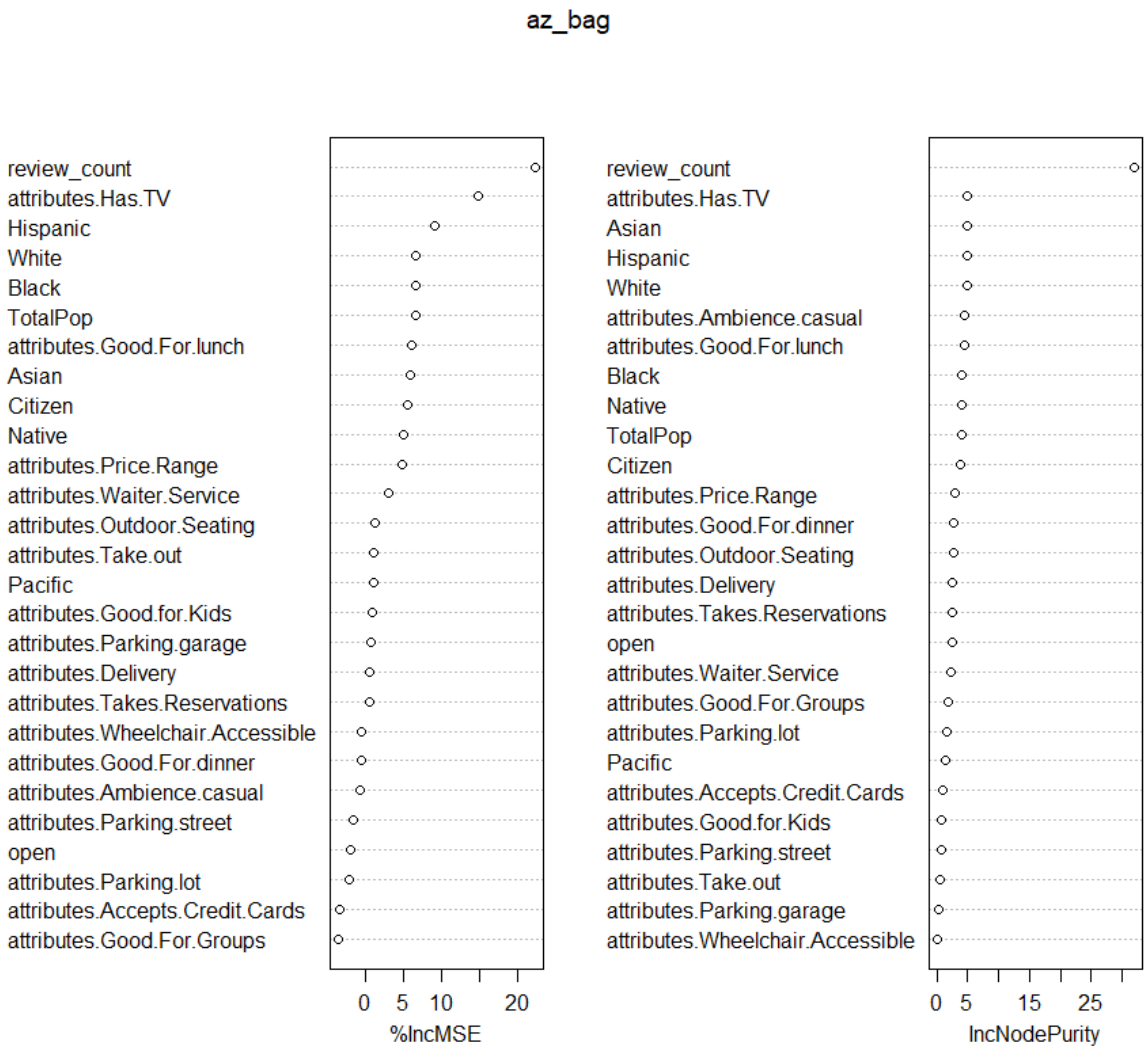

```
# CV errors

## [1] 131.0470 131.1496 130.3705 127.2888 119.8566 120.1318 120.5805 121.3711
## [9] 132.9453

## test error: [1] 0.1943167
```



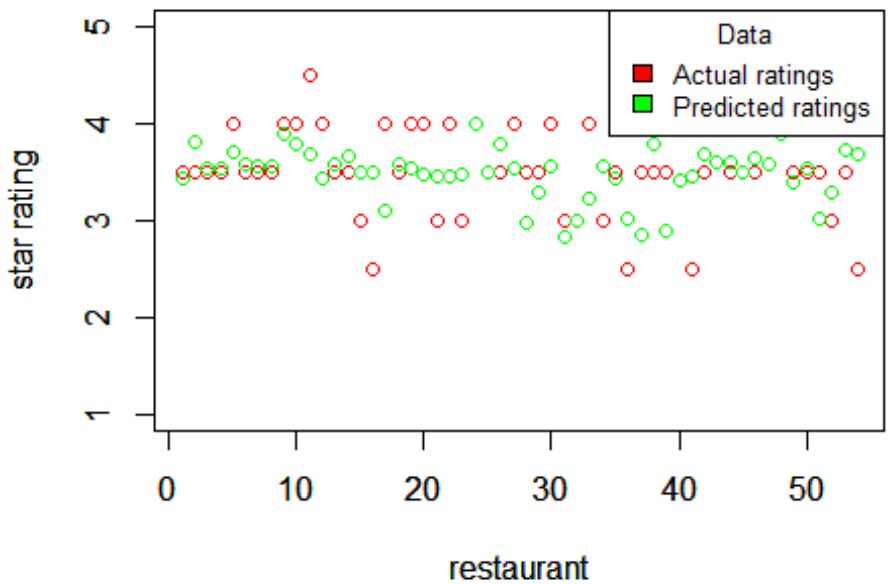
After a 10-fold cross-validation procedure, we determined the lowest cross-validation error occurred at a tree size of 5 nodes. The pruned tree is much easier to read than the previous tree, and we can see that it first splits on review count, similar to Wisconsin's pruned tree. An Asian restaurant with a review count fewer than 45.5 and in a tract ID area with less than 22,000 white population will have a predicted average rating of 3.208. If the restaurant has more than 45.5 reviews and has a TV, the predicted average rating will be slightly higher at 3.845. If the review count is greater than 45.5, does not have a TV, and the review count is also less than 112.5, then the predicted average rating is about 3.553. From this pruned tree we obtain an MSE of about 0.19, which is a significant improvement from the unpruned tree's 0.26.

```
# bagging

## test error: [1] 0.2044775

## importance value
```

```
##                                  %IncMSE IncNodePurity
## attributes.Outdoor.Seating       -0.07320465     2.7002453
## attributes.Parking.lot           -2.82324588     1.7248971
## attributes.Waiter.Service         1.07087269     2.6468058
## attributes.Parking.street        -0.37295927     0.6755062
## attributes.Price.Range            4.26958681     3.0719136
## attributes.Parking.garage         0.85259389     0.2862375
## review_count                     26.53330106    31.9326421
## attributes.Accepts.Credit.Cards  -4.15594472     1.0558310
## attributes.Good.For.lunch         7.43619677     4.1139827
## attributes.Take.out               2.27493837     0.7395792
## attributes.Takes.Reservations    -1.51417850     2.5596823
## attributes.Delivery               0.14351780     2.6625818
## open                             -2.83558857     2.3624157
## TotalPop                          3.78294746     3.7082094
## Hispanic                          9.47943140     5.1788173
## White                             6.04910865     4.9304717
## Black                             7.09593146     4.0571764
## Native                            6.16103512     4.2137526
## Asian                             4.52325953     4.7707271
## Pacific                          -0.43667247     1.4667563
## Citizen                           4.75544794     3.9784548
## attributes.Wheelchair.Accessible -0.42432014     0.1014035
## attributes.Good.For.dinner       -3.44258017     2.7317660
## attributes.Good.for.Kids          3.43280383     0.8441235
## attributes.Has.TV                11.74959025     5.0248383
## attributes.Ambience.casual       -1.54806632     4.4200815
## attributes.Good.For.Groups       -3.43352330     1.9045369

#plot importance of predictors
```



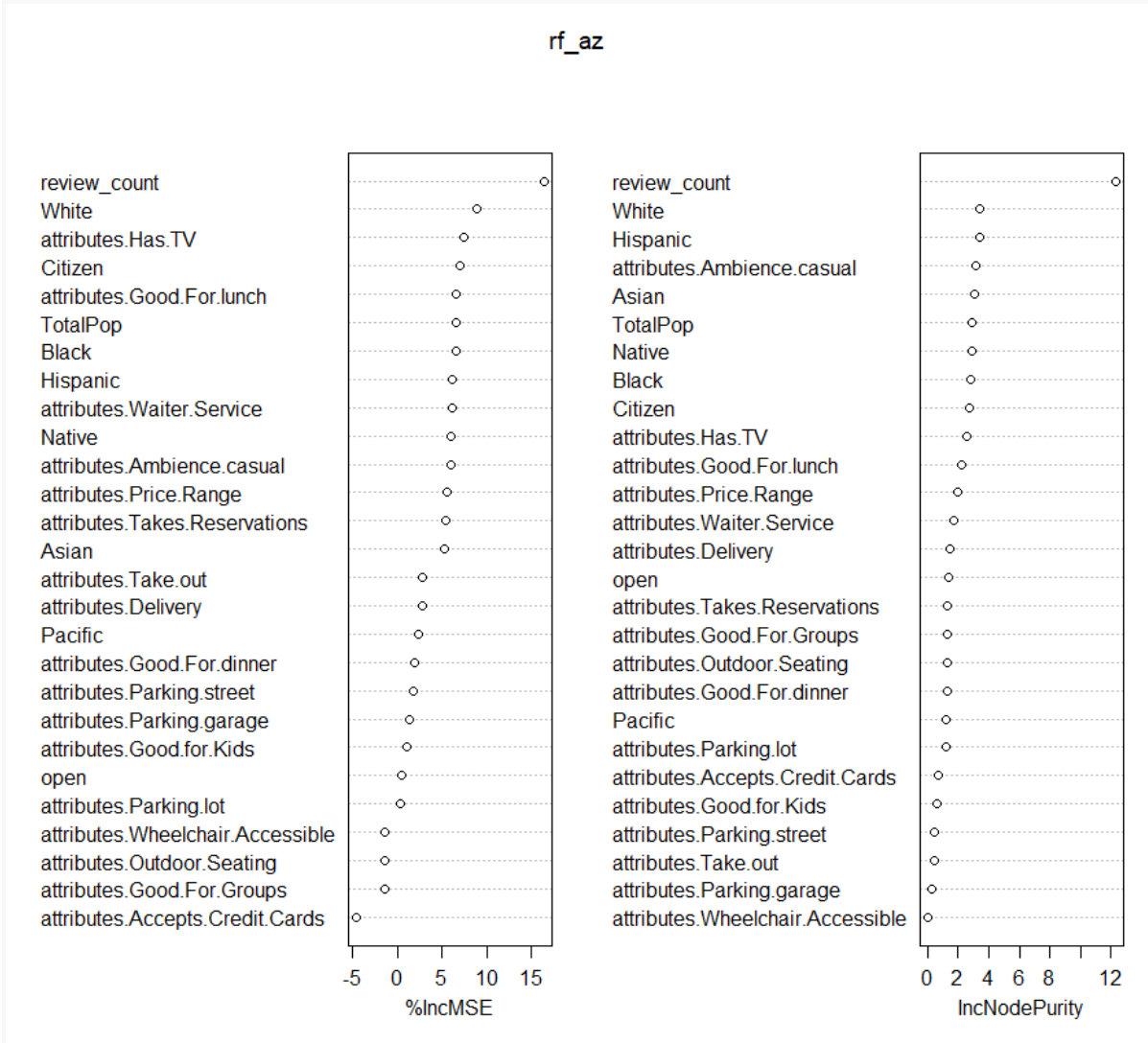
After bagging with the Arizona training set, we can see from the importance function that review count and the attribute.Has.TV are the more important predictors for this data, with %IncMSE 26.53330106 and 11.74959025 respectively. The review count variable is by far the more important predictor when compared to the other variables. We obtain an MSE of about 0.2044775 from bagging, which seems to be slightly worse than that of the pruned tree obtained from cross-validation.



Looking at the plot of actual and predicted ratings obtained from bagging, we can see that this time there is a smaller chance of overfitting (since there is a larger training and testing set), but the prediction accuracy is not doing terribly. Most points appear to accumulate at 3.5 stars, and the predicted ratings lie closely to this line y = 3.5 while the actual ratings vary a little more.

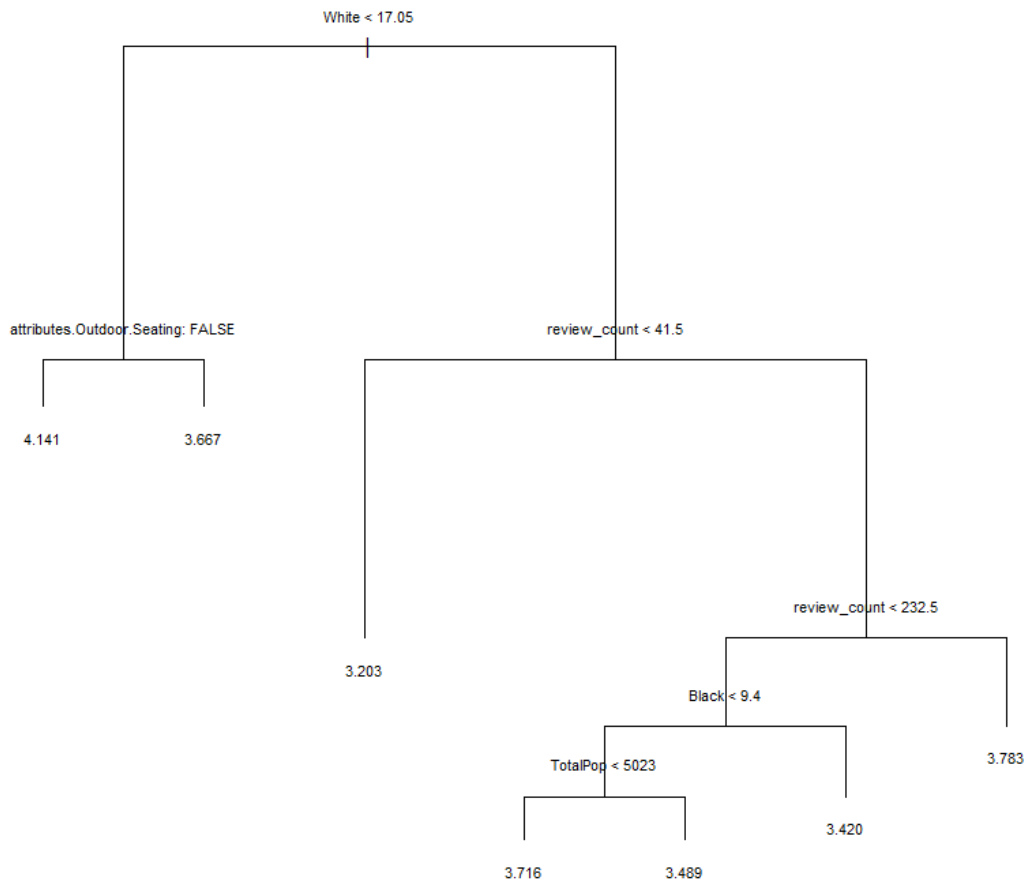
```
# random forest test errors
my_forest(2)

## [1] 0.1863437

my_forest(3)

## [1] 0.1879131

my_forest(4)

## [1] 0.1892993

my_forest(5)

## [1] 0.1939374

my_forest(6)

## [1] 0.1974426

my_forest(7)
```

```
## [1] 0.1955827

my_forest(8)

## [1] 0.1959064

## importance values and plot

## attributes.Outdoor.Seating        2.2042005    1.23077075
## attributes.Parking.lot           -0.3877164    1.26158253
## attributes.Waiter.Service         7.4871257    1.72252523
## attributes.Parking.street         0.3890498    0.49768000
## attributes.Price.Range            4.6666530    1.70446141
## attributes.Parking.garage         1.4504767    0.31639524
## review_count                     17.8344708   11.51081255
## attributes.Accepts.Credit.Cards  -5.1564724    0.59106907
## attributes.Good.For.lunch         5.3652967    2.36374433
## attributes.Take.out               2.6350857    0.52977873
## attributes.Takes.Reservations     3.4842992    1.36468802
## attributes.Delivery               2.7462857    1.41192966
## open                              1.9953314    1.31712224
## TotalPop                          7.5806456    2.96556927
## Hispanic                          8.5799016    3.39762653
## White                             5.9134730    3.29472298
## Black                             7.3334671    2.66793317
## Native                            7.0692533    2.83081219
## Asian                             8.1490761    3.18272208
## Pacific                           2.6988005    1.20127214
## Citizen                           6.7511270    2.77174121
## attributes.Wheelchair.Accessible -2.9748535    0.06523954
## attributes.Good.For.dinner        1.1155976    1.44402108
## attributes.Good.for.Kids          1.9128620    0.58395558
## attributes.Has.TV                 9.2023067    2.34487392
## attributes.Ambience.casual        5.6852352    3.24190204
## attributes.Good.For.Groups        0.3466326    1.20331533
```



From the output of the written test error function, it is not very clear the reason as to why the lowest error comes from the random forest of mtry = 2, since with at least one important variable the best model should at least be built with mtry = 5 (approximately square root of 28). When we do look at a random forest of mtry = 2, the review count has a significantly high %incMSE of 17.83447 which indicates it is a very effective predictor of star count. Without the review counts, the restaurant rating prediction likely decreases greatly.

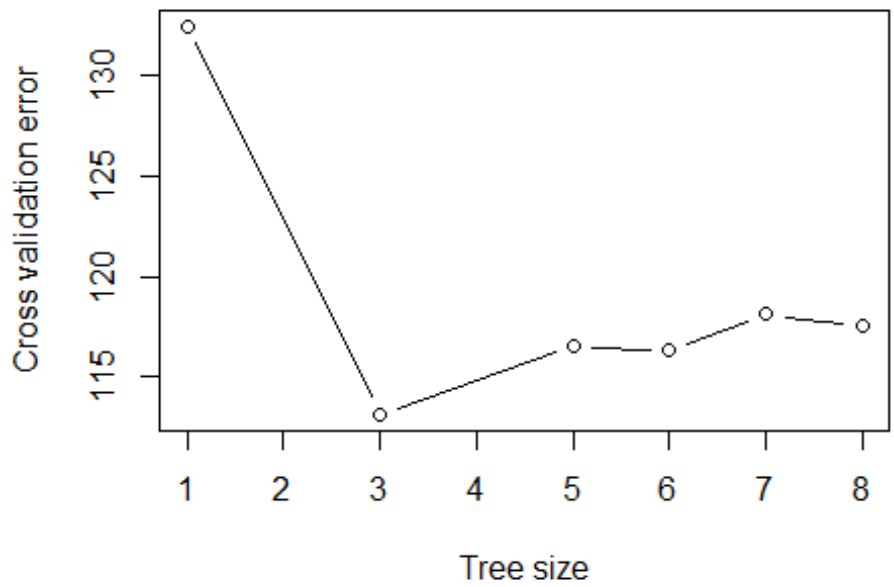
Asian restaurants in Nevada



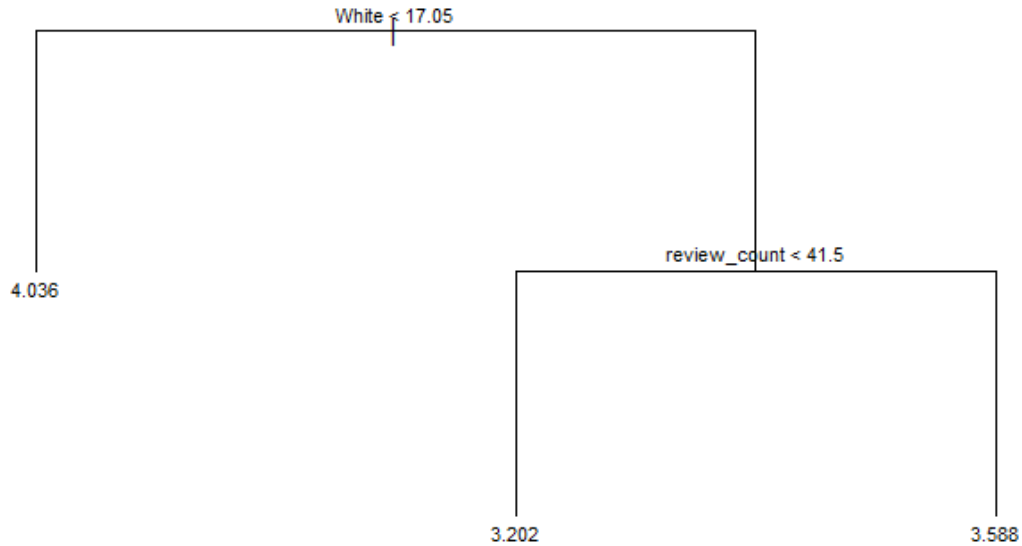

```
## test error: [1] 0.3211486
```

A single regression tree was obtained from the Nevada Asian restaurant training data, and the MSE for this decision tree is about 0.3211. We can see that if the local white population (based on tract ID) is less than 17,050 and there is no outdoor seating, the predicted average restaurant rating is about 4.141 stars. If there is outdoor seating, the predicted star rating falls to about 3.667 (there may be a noise factor). Meanwhile, if the local white population is greater than 17,050 and the review counts are less than 41.5, the average predicted rating is around 3.203. With the same larger white population, if there are greater than 41.5 reviews but less than 232.5 reviews, and the location black population is greater than 9,400, the average star rating will be about 3.420.

```
# CV to find best number nodes
```



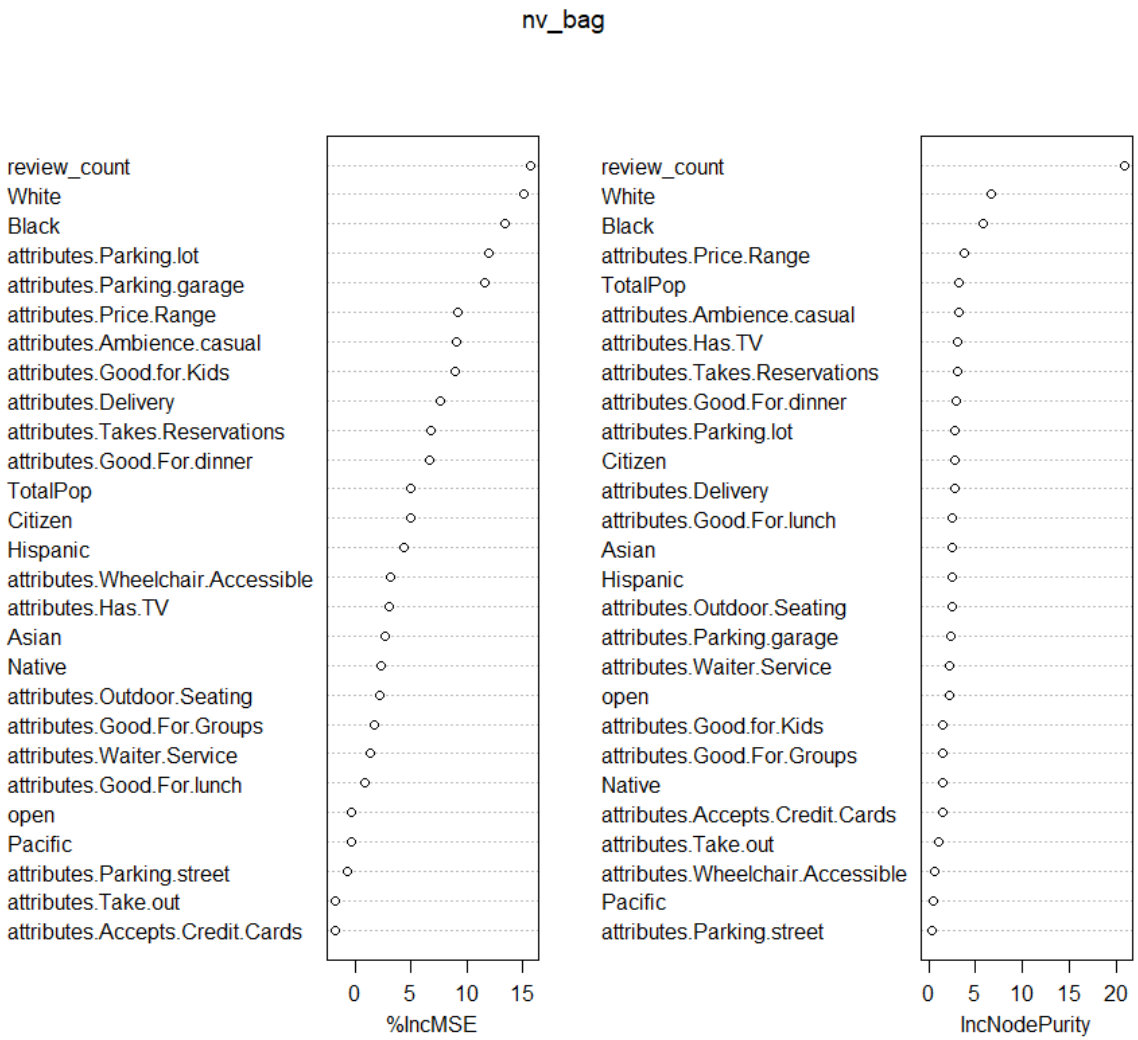

```
# CV errors
## [1] 117.5488 118.1172 116.3407 116.5270 113.1383 132.4697
# test error MSE
## [1] 0.2729385
```



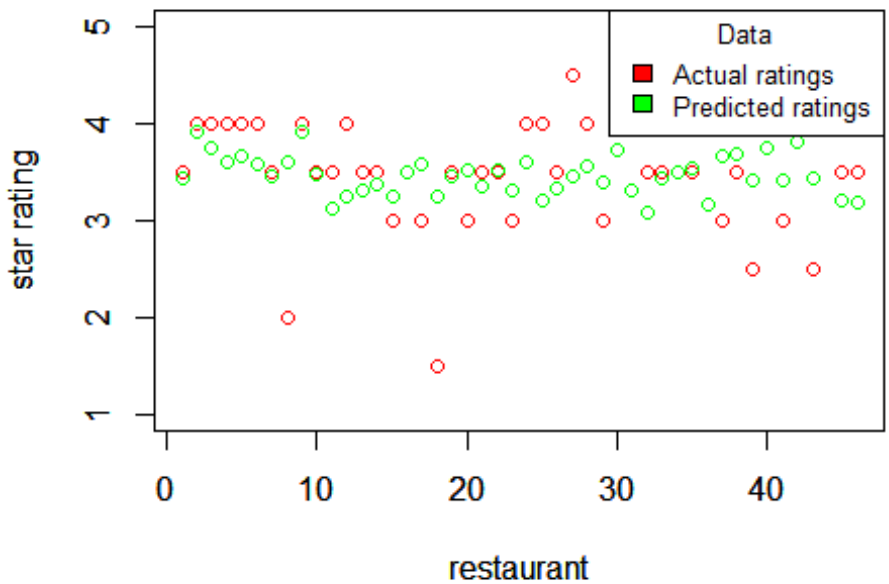
After a 10-fold cross validation, the lowest cross validation error occurs at a tree size of three terminal nodes. Thus the pruned tree shows a much simpler diagram with a test error of about 0.27, an improvement from the previous tree's 0.32. We can see that if the restaurant's local white population is less than 17,050 (based on tract ID), the average rating will be predicted to be about 4.036. If the white population is greater than 17,050 and there are fewer than 41.5 review counts, the average predicted rating will be about 3.202. On the other hand if the area has greater than a 17,050 white population and there are more than 41.5 reviews, then the average star rating increases to about 3.588. It appears that review count and white population are once again important predictors for restaurant rating.

```
# bagging

## test error: [1] 0.3122475

## importance values

##                                %IncMSE    IncNodePurity
## attributes.Outdoor.Seating      2.2514449     2.4465439
## attributes.Parking.lot         11.9631487     2.8016121
## attributes.Waiter.Service       1.4105309     2.2688691
## attributes.Parking.street      -0.7481426     0.3081519
## attributes.Price.Range          9.2011130     3.7934154
## attributes.Parking.garage      11.6083534     2.3825828
## review_count                   15.7765341    20.8743637
```

```
## attributes.Accepts.Credit.Cards  -1.8373757     1.4567157
## attributes.Good.For.lunch         0.8940711     2.5454246
## attributes.Take.out              -1.7976081     0.9962849
## attributes.Takes.Reservations     6.8448037     3.0820381
## attributes.Delivery               7.6831073     2.7275094
## open                             -0.2980499     2.2281958
## TotalPop                          5.0178994     3.1896478
## Hispanic                          4.3705704     2.4935701
## White                            15.1355733     6.6852630
## Black                            13.3910523     5.8579413
## Native                            2.3090485     1.4601023
## Asian                             2.7060223     2.5120024
## Pacific                          -0.3326852     0.4372118
## Citizen                           4.9760518     2.7395918
## attributes.Wheelchair.Accessible  3.1797417     0.6268923
## attributes.Good.For.dinner        6.6318862     2.9225935
## attributes.Good.for.Kids          9.0058850     1.5459147
## attributes.Has.TV                 3.0141216     3.1221572
## attributes.Ambience.casual        9.0830910     3.1511004
## attributes.Good.For.Groups        1.7300035     1.4976756

#plot importance of predictors
```



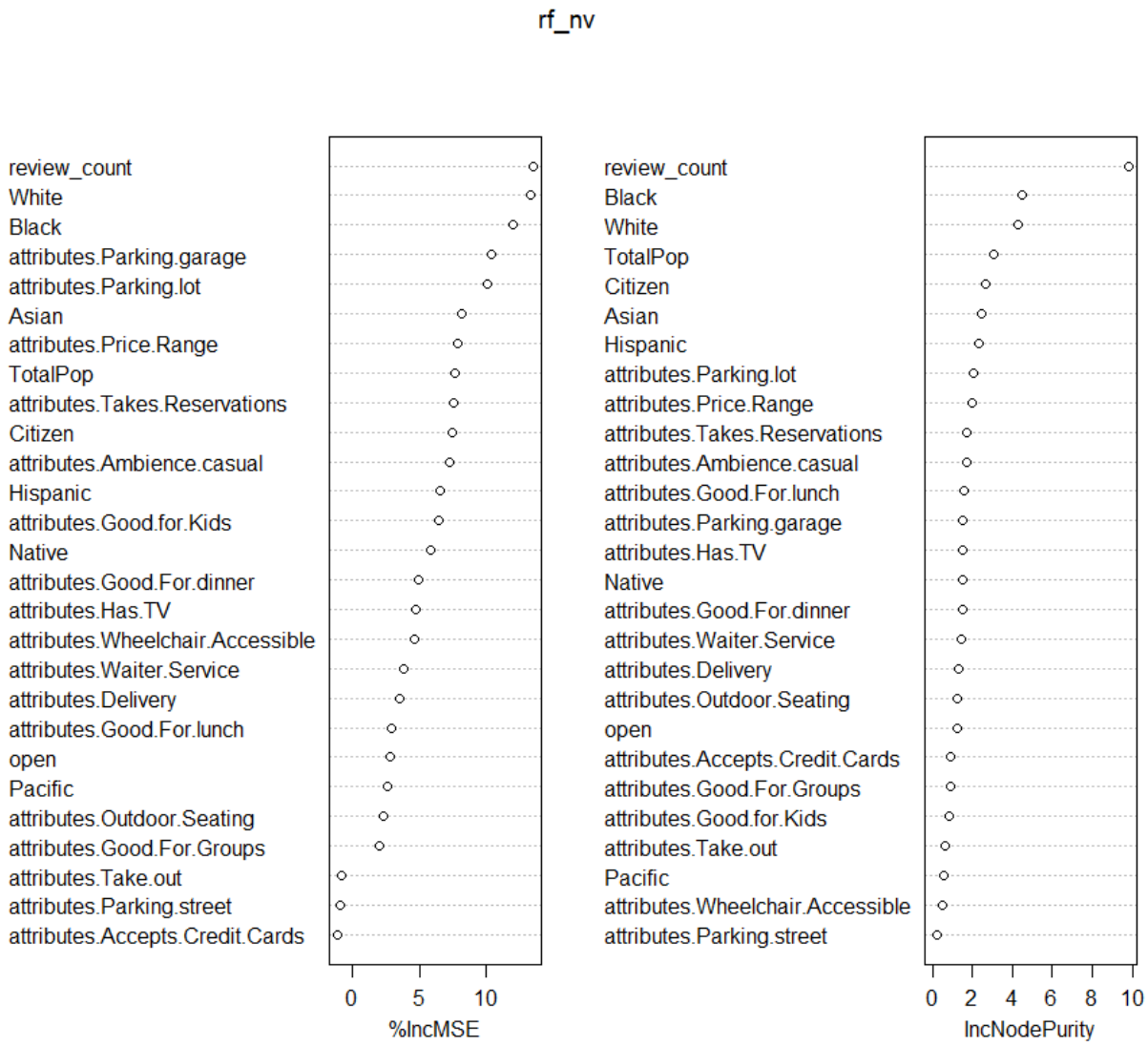
With bagging, we can see that the review count, white and black populations, and other variables such as attributes.Parking.lot and parking garage have larger %IncMSE than other variables. Once again, review count has the largest %IncMSE of about 15.7765341. Looking at model performance, the MSE increases to 0.3122475 from the previous 0.2729385. It is not clear why the bagging method increases the variance as it did similarly for the Arizona Asian restaurant training set. This is an area of possible future investigation.



The scatter plot of actual and predicted star rating against the test set restaurants yield the above graph. This graph is comparable to that of Arizona's test set, except the actual ratings in the test data for Nevada seems to fluctuate a lot more with some extreme low ratings around restaurants #9, #18 and the #40s range. These inaccurate predictions are contributing to the higher test error.

```
# random forest test errors

## test error for varying tree sizes
nv_forest(2)

## [1] 0.3042506

nv_forest(3)

## [1] 0.3061165

nv_forest(4)

## [1] 0.3048472

nv_forest(5)

## [1] 0.3068822

nv_forest(6)
```

```
## [1] 0.3083559

nv_forest(7)

## [1] 0.3058086

nv_forest(8)

## [1] 0.3060131

## importance values and plot

                                %IncMSE        IncNodePurity
## attributes.Outdoor.Seating        2.2496739     1.2737323
## attributes.Parking.lot           10.1189004     2.0313621
## attributes.Waiter.Service         3.8182421     1.4252063
## attributes.Parking.street        -0.9242225     0.2290039
## attributes.Price.Range            7.8999471     1.9795937
## attributes.Parking.garage        10.4656068     1.5286129
## review_count                     13.6296391     9.8425584
## attributes.Accepts.Credit.Cards  -1.2038931     0.9395135
## attributes.Good.For.lunch         2.8788375     1.5698546
## attributes.Take.out              -0.8517775     0.6318545
## attributes.Takes.Reservations     7.5514772     1.7239854
## attributes.Delivery               3.4958358     1.3359213
## open                              2.7777624     1.2571199
## TotalPop                          7.7041015     3.0588640
## Hispanic                          6.5649324     2.3443125
## White                            13.4231366     4.2987432
## Black                            12.0408813     4.4704974
## Native                            5.9038283     1.4961367
## Asian                             8.2183489     2.4901545
## Pacific                           2.5819942     0.5523981
## Citizen                           7.4927675     2.6729974
## attributes.Wheelchair.Accessible  4.6181308     0.5322143
## attributes.Good.For.dinner        4.9920673     1.4939856
## attributes.Good.for.Kids          6.4449238     0.8184747
## attributes.Has.TV                 4.6899280     1.5243544
## attributes.Ambience.casual        7.3055162     1.7065396
## attributes.Good.For.Groups        1.9817047     0.9287555
```



The random forest errors give the lowest error at mtry = 2. Again, it is not clear why the smallest possible tree size gives the highest accuracy. When a random forest is split on mtry = 2, the same variables as the bagging model are given as important predictors of star rating. Review count, white and black populations have %IncMSE of 13.6296391, 13.4231366, and 12.0408813 respectively. These top predictors match with the first decision tree we made for the Nevada data.

In conclusion, the Asian restaurant data from all three states suggest that review count is an important predictor of star rating. There could be many reasons for this correlation. It is possible that response bias exists and only those that feel really strongly one way or another about the restaurant, meaning that they give high or low star ratings, tend to review more often. Therefore, a frequently voted highly ranked restaurant could see more and more positive ratings since more people are drawn to higher rated restaurants. It is also possible that people would tend to rank low rated restaurants low as well to add their voice into the reviews of dissatisfaction and feel heard and seen.

In Nevada, other factors such as local white and black population also appear to contribute to restaurant star rating. However, near the end of the project, we noticed that the population data is probably misappropriated. The population is strictly the population of a group in a particular county. By including population in the model, it misconstrued the model such that it reports that same population each time for every unique restaurant. What would probably be more useful is data on how many white people, asian people, hispanic people, or black people frequent a specific restaurant and then see if that has an impact on the Yelp ratings. An improvement to the model would be to include data on the different ethnic groups that frequent those restaurants and see whether it has any impact on the ratings of the restaurant.

Finally, it is also difficult to draw conclusions from the Wisconsin data because there are so few data points that overfitting was seen on some of the tree models.

Some issues and possible future points of investigation is the increase in MSE when we did bagging with the Arizona and Nevada training data, as well as the very small tree size being the most accurate (lowest test error) parameter for the random forests that were constructed. These values are not normally observed, so we are unsure if they are correct. We did our best to use class methods and interpret them using the information we know.

Initially, our third question was to look at trends in the data over time. However, the data had no time-related data, so we could not perform any time-series analysis. This could be a good topic for future research, where rather than simply looking at a snapshot of whenever the data were collected, one may look at how various patterns in the data vary across time. One may, for example, see how Asian restaurants cluster over time and see if there are any shifts in where they are located. Or perhaps one may look at how ratings for Asian restaurants vary over time and if different predictors are more impactful during different periods. It is really difficult to acquire public data that represents the population well, as census data is just another survey for the public to neglect. Moreover, lots of data is kept private for larger institutions to study. Ideally, it would be interesting to see how Asian restaurants and Asian-owned restaurants fare in comparison to non-Asian restaurants and see if the demographics of the area have an impact on that. It would also be interesting to compare a highly diverse city with a more homogenous town to see if there is a stark difference in the number of Asian restaurants or attitudes towards Asian food and people.